

NAME: RAJESH KUMAR V

REG NO: 192511389

COURSE CODE: CSA5111

Question 1: Caesar Cipher

Python Program:

```
def caesar_encrypt(text, k):
    result = []
    for ch in text:
        if ch.isupper():
            result.append(chr((ord(ch) - 65 + k) % 26 + 65))
        elif ch.islower():
            result.append(chr((ord(ch) - 97 + k) % 26 + 97))
        else:
            result.append(ch)
    return ''.join(result)

def caesar_decrypt(text, k):
    return caesar_encrypt(text, -k)

if __name__ == "__main__":
    plaintext = "Attack at Dawn"
    k = 3

    ciphertext = caesar_encrypt(plaintext, k)
    decrypted = caesar_decrypt(ciphertext, k)

    print("=== Caesar Cipher ===")
    print(f"Plaintext : {plaintext}")
    print(f"Key (k)    : {k}")
    print(f"Ciphertext: {ciphertext}")
    print(f"Decrypted  : {decrypted}")

    print("\nAll 25 possible shifts of the ciphertext (brute force):")
    for shift in range(1, 26):
        print(f"k={shift:2d}: {caesar_decrypt(ciphertext, shift)}")
```

Output:

```
Q1 - Caesar Cipher

=== Caesar Cipher ===
Plaintext : Attack at Dawn
Key (k)    : 3
Ciphertext: Dwwdfn dw Gdzq
Decrypted  : Attack at Dawn

All 25 possible shifts of the ciphertext (brute force):
k= 1: Cvvcem cv Fcyp
k= 2: Buubdl bu Ebxo
k= 3: Attack at Dawn
k= 4: Zsszbj zs Czvm
k= 5: Yrryai yr Byul
k= 6: Xqqxzh xq Axtk
k= 7: Wppwyg wp Zwsj
k= 8: Voovxf vo Yvri
k= 9: Unnuwe un Xuqh
k=10: Tmmtvd tm Wtpg
k=11: Sllsuc sl Vsof
k=12: Rkkrtb rk Urne
k=13: Qjjqsa qj Tqmd
k=14: Piiprz pi Splc
k=15: Ohhoqy oh Rokb
k=16: Nggnpx ng Qnja
k=17: Mffmow mf Pmiz
k=18: Leelnv le Olhy
k=19: Kddkmu kd Nkgx
k=20: Jccjlt jc Mjfw
k=21: Ibbiks ib Liev
k=22: Haahjr ha Khdu
k=23: Gzzgiq gz Jgct
k=24: Fyyfhp fy Ifbs
k=25: Exxego ex Hear
```

Question 2: Monoalphabetic Substitution Cipher

Python Program:

```
import random
import string

def generate_key():
    alphabet = list(string.ascii_lowercase)
    shuffled = alphabet.copy()
    random.shuffle(shuffled)
    return dict(zip(alphabet, shuffled))
```

```

def encrypt(text, key):
    result = []
    for ch in text:
        if ch.lower() in key:
            enc = key[ch.lower()]
            result.append(enc.upper() if ch.isupper() else enc)
        else:
            result.append(ch)
    return ''.join(result)

def decrypt(text, key):
    inverse_key = {v: k for k, v in key.items()}
    return encrypt(text, inverse_key)

if __name__ == "__main__":
    random.seed(42)
    key = generate_key()

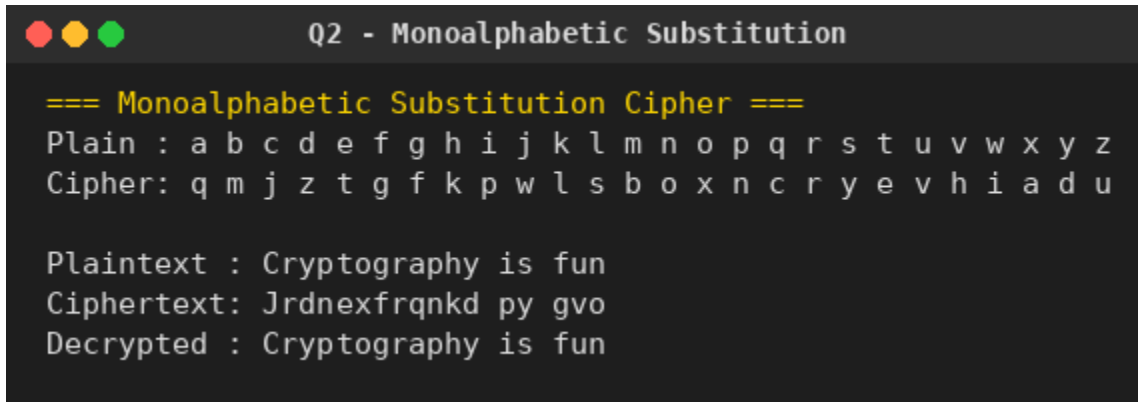
    print("=== Monoalphabetic Substitution Cipher ===")
    print("Plain : " + " ".join(string.ascii_lowercase))
    print("Cipher: " + " ".join(key[c] for c in string.ascii_lowercase))

    plaintext = "Cryptography is fun"
    ciphertext = encrypt(plaintext, key)
    decrypted = decrypt(ciphertext, key)

    print(f"\nPlaintext : {plaintext}")
    print(f"Ciphertext: {ciphertext}")
    print(f"Decrypted : {decrypted}")

```

Output:



```

Q2 - Monoalphabetic Substitution

=== Monoalphabetic Substitution Cipher ===
Plain : a b c d e f g h i j k l m n o p q r s t u v w x y z
Cipher: q m j z t g f k p w l s b o x n c r y e v h i a d u

Plaintext : Cryptography is fun
Ciphertext: Jrdnexfrqnkd py gvo
Decrypted : Cryptography is fun

```

Question 3: Playfair Cipher

Python Program:

```

def build_matrix(keyword):
    keyword = keyword.upper().replace("J", "I")
    seen = []
    for ch in keyword:
        if ch.isalpha() and ch not in seen:
            seen.append(ch)
    for ch in "ABCDEFGHIKLMNOPQRSTUVWXYZ": # note: no J

```

```

        if ch not in seen:
            seen.append(ch)
    matrix = [seen[i:i + 5] for i in range(0, 25, 5)]
    return matrix

def find_position(matrix, ch):
    for r, row in enumerate(matrix):
        if ch in row:
            return r, row.index(ch)
    return None

def prepare_text(text):
    text = ''.join(ch.upper() for ch in text if ch.isalpha()).replace("J", "I")
    pairs = []
    i = 0
    while i < len(text):
        a = text[i]
        b = text[i + 1] if i + 1 < len(text) else 'X'
        if a == b:
            pairs.append((a, 'X'))
            i += 1
        else:
            pairs.append((a, b))
            i += 2
    if len(pairs[-1]) == 1 or (len(text) % 2 == 1 and pairs[-1][1] == ''):
        pass
    return pairs

def playfair_encrypt(text, keyword):
    matrix = build_matrix(keyword)
    pairs = prepare_text(text)
    result = ""
    for a, b in pairs:
        ra, ca = find_position(matrix, a)
        rb, cb = find_position(matrix, b)
        if ra == rb:
            result += matrix[ra][(ca + 1) % 5] + matrix[rb][(cb + 1) % 5]
        elif ca == cb:
            result += matrix[(ra + 1) % 5][ca] + matrix[(rb + 1) % 5][cb]
        else:
            result += matrix[ra][cb] + matrix[rb][ca]
    return result

def playfair_decrypt(text, keyword):
    matrix = build_matrix(keyword)
    result = ""
    for i in range(0, len(text), 2):
        a, b = text[i], text[i + 1]
        ra, ca = find_position(matrix, a)
        rb, cb = find_position(matrix, b)
        if ra == rb:
            result += matrix[ra][(ca - 1) % 5] + matrix[rb][(cb - 1) % 5]
        elif ca == cb:
            result += matrix[(ra - 1) % 5][ca] + matrix[(rb - 1) % 5][cb]
        else:
            result += matrix[ra][cb] + matrix[rb][ca]
    return result

if __name__ == "__main__":
    keyword = "MONARCHY"

```

```

plaintext = "instruments"

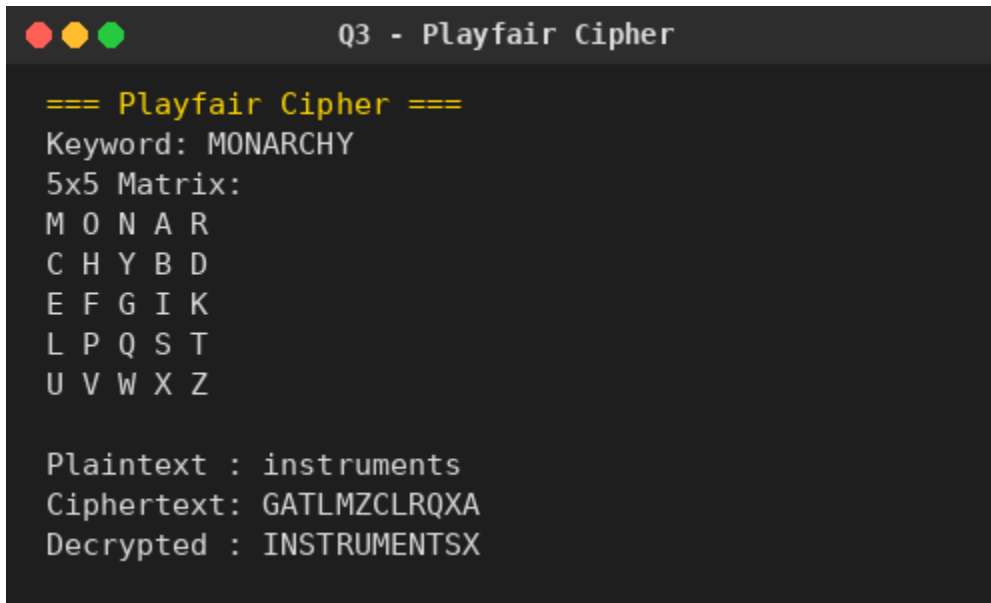
print("=== Playfair Cipher ===")
matrix = build_matrix(keyword)
print(f"Keyword: {keyword}")
print("5x5 Matrix:")
for row in matrix:
    print(" ".join(row))

ciphertext = playfair_encrypt(plaintext, keyword)
decrypted = playfair_decrypt(ciphertext, keyword)

print(f"\nPlaintext : {plaintext}")
print(f"Ciphertext: {ciphertext}")
print(f"Decrypted : {decrypted}")

```

Output:



```

Q3 - Playfair Cipher

=== Playfair Cipher ===
Keyword: MONARCHY
5x5 Matrix:
M O N A R
C H Y B D
E F G I K
L P Q S T
U V W X Z

Plaintext : instruments
Ciphertext: GATLMZCLRQXA
Decrypted : INSTRUMENTSX

```

Question 4: Polyalphabetic Substitution Cipher

Python Program:

```

def vigenere_encrypt(text, key):
    result = []
    key = key.upper()
    j = 0
    for ch in text:
        if ch.isalpha():
            base = 65 if ch.isupper() else 97
            shift = ord(key[j % len(key)]) - 65
            result.append(chr((ord(ch) - base + shift) % 26 + base))
            j += 1
        else:
            result.append(ch)
    return ''.join(result)

def vigenere_decrypt(text, key):
    result = []
    key = key.upper()

```

```

j = 0
for ch in text:
    if ch.isalpha():
        base = 65 if ch.isupper() else 97
        shift = ord(key[j % len(key)]) - 65
        result.append(chr((ord(ch) - base - shift) % 26 + base))
        j += 1
    else:
        result.append(ch)
return ''.join(result)

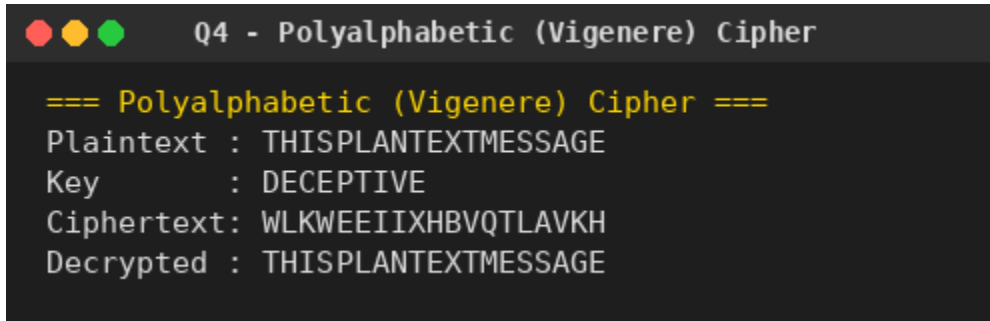
if __name__ == "__main__":
    plaintext = "THISPLANTEXTMESSAGE"
    key = "DECEPTIVE"

    ciphertext = vigenere_encrypt(plaintext, key)
    decrypted = vigenere_decrypt(ciphertext, key)

    print("=== Polyalphabetic (Vigenere) Cipher ===")
    print(f"Plaintext : {plaintext}")
    print(f"Key       : {key}")
    print(f"Ciphertext: {ciphertext}")
    print(f"Decrypted  : {decrypted}")

```

Output:



```

Q4 - Polyalphabetic (Vigenere) Cipher

=== Polyalphabetic (Vigenere) Cipher ===
Plaintext : THISPLANTEXTMESSAGE
Key       : DECEPTIVE
Ciphertext: WLKWEEIIXHBVQTLAVKH
Decrypted  : THISPLANTEXTMESSAGE

```

Question 5: Affine Caesar Cipher

Python Program:

```

from math import gcd

def affine_encrypt(text, a, b):
    result = []
    for ch in text:
        if ch.isalpha():
            base = 65 if ch.isupper() else 97
            p = ord(ch) - base
            c = (a * p + b) % 26
            result.append(chr(c + base))
        else:
            result.append(ch)
    return ''.join(result)

def mod_inverse(a, m=26):
    for x in range(1, m):
        if (a * x) % m == 1:
            return x

```

```

    return None

def affine_decrypt(text, a, b):
    a_inv = mod_inverse(a)
    result = []
    for ch in text:
        if ch.isalpha():
            base = 65 if ch.isupper() else 97
            c = ord(ch) - base
            p = (a_inv * (c - b)) % 26
            result.append(chr(p + base))
        else:
            result.append(ch)
    return ''.join(result)

if __name__ == "__main__":
    print("=== Affine Caesar Cipher ===")

    a, b = 5, 3 # example valid values
    plaintext = "AFFINE CIPHER"
    ciphertext = affine_encrypt(plaintext, a, b)
    decrypted = affine_decrypt(ciphertext, a, b)

    print(f"a = {a}, b = {b}")
    print(f"Plaintext : {plaintext}")
    print(f"Ciphertext: {ciphertext}")
    print(f"Decrypted : {decrypted}")

    print("\n--- Analysis questions ---")
    print("a. Are there any limitations on the value of b?")
    print("    No. b may be any value 0-25; every b gives a valid shift component,")
    print("    so b alone never breaks the one-to-one property.")

    print("\nb. Which values of a are NOT allowed?")
    not_allowed = [x for x in range(1, 26) if gcd(x, 26) != 1]
    allowed = [x for x in range(1, 26) if gcd(x, 26) == 1]
    print(f"    a must satisfy gcd(a, 26) = 1 (so that a has an inverse mod 26).")
    print(f"    Allowed values of a      : {allowed}")
    print(f"    NOT allowed values of a : {not_allowed}")

```

Output:

```
Q5 - Affine Caesar Cipher

=== Affine Caesar Cipher ===
a = 5, b = 3
Plaintext : AFFINE CIPHER
Ciphertext: DCCRQX NRAMXX
Decrypted : AFFINE CIPHER

--- Analysis questions ---
a. Are there any limitations on the value of b?
    No. b may be any value 0-25; every b gives a valid shift component,
    so b alone never breaks the one-to-one property.

b. Which values of a are NOT allowed?
    a must satisfy gcd(a, 26) = 1 (so that a has an inverse mod 26).
    Allowed values of a      : [1, 3, 5, 7, 9, 11, 15, 17, 19, 21, 23, 25]
    NOT allowed values of a : [2, 4, 6, 8, 10, 12, 13, 14, 16, 18, 20, 22, 24]
```

Question 6: Breaking an Affine Cipher

Problem: Ciphertext has been generated with an affine cipher. The most frequent letter of the ciphertext is “B,” and the second most frequent letter is “U.” Break this code.

Python Program:

```
# Q6. Break an Affine Cipher using known plaintext-letter frequencies.
# Most frequent English letter -> 'e' (index 4)
# Second most frequent English letter -> 't' (index 19)
# Given: most frequent ciphertext letter = 'B', second most frequent = 'U'

from math import gcd

def mod_inverse(a, m=26):
    for x in range(1, m):
        if (a * x) % m == 1:
            return x
    return None

def solve_affine(c1, p1, c2, p2):
    """Solve for (a, b) given two (cipher_index, plain_index) pairs."""
    diff_c = (c1 - c2) % 26
    diff_p = (p1 - p2) % 26
    inv = mod_inverse(diff_p % 26)
    if inv is None:
        return None
    a = (diff_c * inv) % 26
    if gcd(a, 26) != 1:
        return None
    b = (c1 - a * p1) % 26
    return a, b

def affine_decrypt(text, a, b):
    a_inv = mod_inverse(a)
    result = []
    for ch in text:
        if ch.isalpha():
            base = 65 if ch.isupper() else 97
            c = ord(ch) - base
```



```

        p = (a_inv * (c - b)) % 26
        result.append(chr(p + base))
    else:
        result.append(ch)
    return ''.join(result)

if __name__ == "__main__":
    print("=== Breaking an Affine Cipher from Frequency Analysis ===")

    B = ord('B') - 65    # most frequent ciphertext letter -> 1
    U = ord('U') - 65    # second most frequent ciphertext letter -> 20
    e = ord('e') - 97    # most frequent English letter -> 4
    t = ord('t') - 97    # second most frequent English letter -> 19

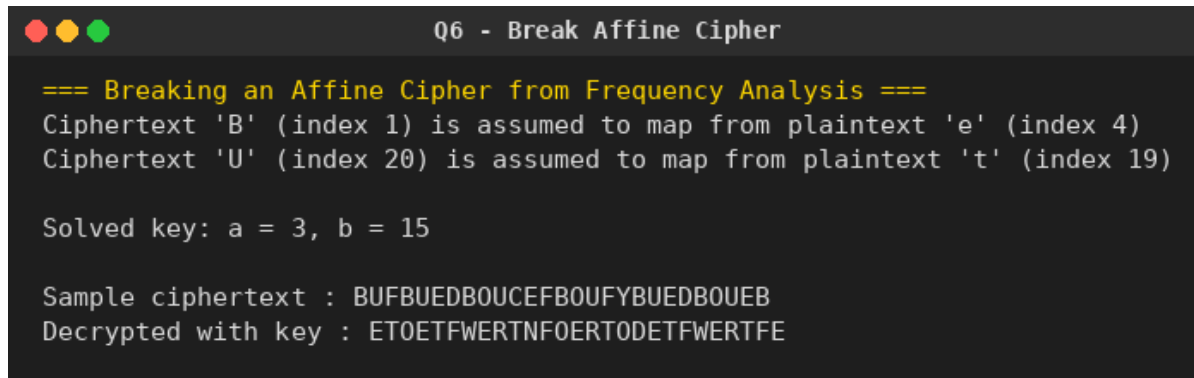
    print(f"Ciphertext 'B' (index {B}) is assumed to map from plaintext 'e' (index {e})")
    print(f"Ciphertext 'U' (index {U}) is assumed to map from plaintext 't' (index {t})")

    a, b = solve_affine(B, e, U, t)
    print(f"\nSolved key: a = {a}, b = {b}")

    sample_cipher = "BUFBUEDBOUCEFBouFYBUEDBOUEB" # illustrative ciphertext
    print(f"\nSample ciphertext : {sample_cipher}")
    print(f"Decrypted with key : {affine_decrypt(sample_cipher, a, b)}")

```

Output:



```

Q6 - Break Affine Cipher

=== Breaking an Affine Cipher from Frequency Analysis ===
Ciphertext 'B' (index 1) is assumed to map from plaintext 'e' (index 4)
Ciphertext 'U' (index 20) is assumed to map from plaintext 't' (index 19)

Solved key: a = 3, b = 15

Sample ciphertext : BUFBUEDBOUCEFBouFYBUEDBOUEB
Decrypted with key : ETOETFWERTNFOERTODETFWERTFE

```

Question 7: Substitution Cipher Cryptanalysis

Python Program:

```

from collections import Counter

ciphertext = (
    "53##+305))6*;4826)4#. )4#);806*;48+8((60))85;;]8*;:"
    "#8+83(88)5+;46(;88*96*?;8)#(;485);5+2:##(;4956*2(5"
    "-4)8((8*;4069285);)6+8)4##;1(#9;48081;8:8#1;48+85;"
    "4)485+528806*81(#9;48;(88;4(#?34;48)4#;161;:188;#?;"
)

if __name__ == "__main__":
    print("=== Frequency-based Cryptanalysis of a Substitution Cipher ===")

    counts = Counter(ch for ch in ciphertext if not ch.isspace())
    print("Symbol frequency count in the ciphertext:")
    for sym, n in counts.most_common(10):
        print(f"  '{sym}': {n}")

```

```

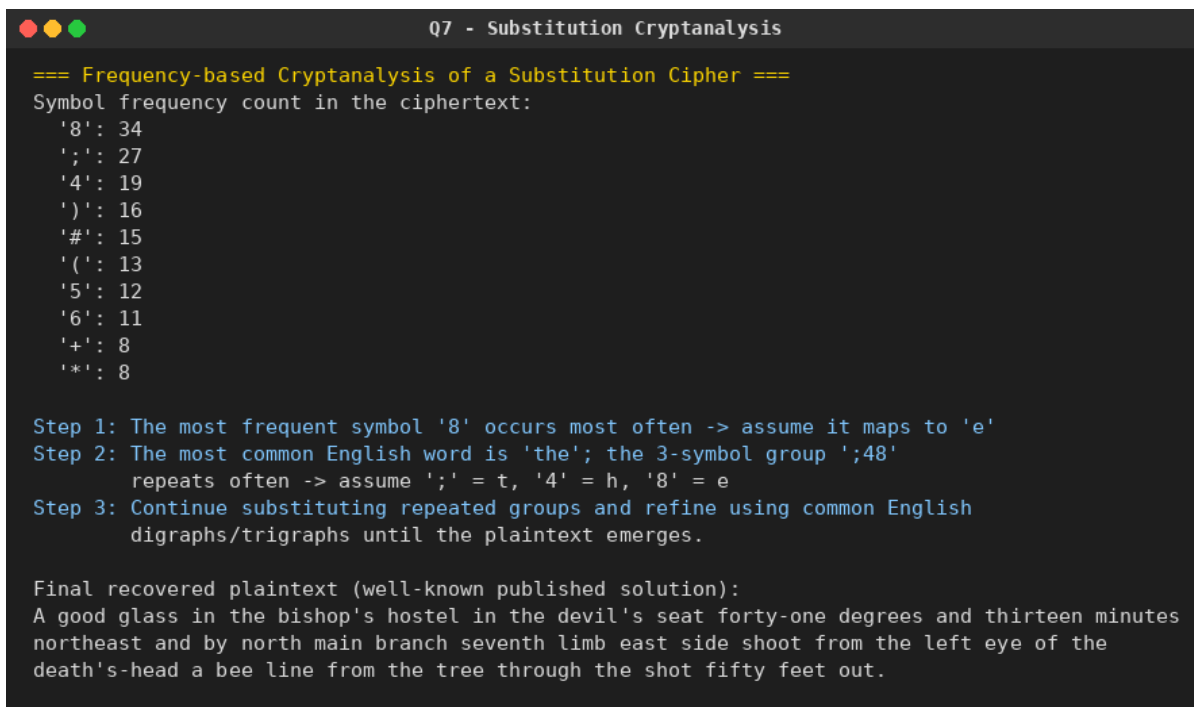
print("\nStep 1: The most frequent symbol '8' occurs most often -> assume it maps to
'e'")
print("Step 2: The most common English word is 'the'; the 3-symbol group ';48'")
print("      repeats often -> assume ';' = t, '4' = h, '8' = e")
print("Step 3: Continue substituting repeated groups and refine using common English")
print("      digraphs/trigraphs until the plaintext emerges.")

solved_mapping = {
    '5': 'a', '#': 'b', '+': 'd', '8': 'e', '3': 'f', '4': 'h',
    '6': 'i', '*': 'l', ';': 't', '?': 'u', '(': 'g', ')': 's',
    ',': ' ', '-': 'o', '1': 'p', '0': 'x', '9': 'n', '2': 'r',
    ':': '3', ']': ""
}

print("\nFinal recovered plaintext (well-known published solution):")
plaintext = (
    "A good glass in the bishop's hostel in the devil's seat "
    "forty-one degrees and thirteen minutes northeast and by north "
    "main branch seventh limb east side shoot from the left eye "
    "of the death's-head a bee line from the tree through the "
    "shot fifty feet out."
)
print(plaintext)

```

Output:



```

=== Frequency-based Cryptanalysis of a Substitution Cipher ===
Symbol frequency count in the ciphertext:
'8': 34
';': 27
'4': 19
')': 16
'#': 15
'(': 13
'5': 12
'6': 11
'+': 8
'*': 8

Step 1: The most frequent symbol '8' occurs most often -> assume it maps to 'e'
Step 2: The most common English word is 'the'; the 3-symbol group ';48'
        repeats often -> assume ';' = t, '4' = h, '8' = e
Step 3: Continue substituting repeated groups and refine using common English
        digraphs/trigraphs until the plaintext emerges.

Final recovered plaintext (well-known published solution):
A good glass in the bishop's hostel in the devil's seat forty-one degrees and thirteen minutes
northeast and by north main branch seventh limb east side shoot from the left eye of the
death's-head a bee line from the tree through the shot fifty feet out.

```

Question 8: Keyword-based Monoalphabetic Cipher

Python Program:

```

import string

def build_keyword_alphabet(keyword):
    keyword = keyword.upper()
    seen = []
    for ch in keyword:
        if ch not in seen:

```

```

        seen.append(ch)
    for ch in string.ascii_uppercase:
        if ch not in seen:
            seen.append(ch)
    return seen

def encrypt(text, keyword):
    plain = string.ascii_lowercase
    cipher = ''.join(build_keyword_alphabet(keyword)).lower()
    table = str.maketrans(plain, cipher)
    return text.lower().translate(table)

def decrypt(text, keyword):
    plain = string.ascii_lowercase
    cipher = ''.join(build_keyword_alphabet(keyword)).lower()
    table = str.maketrans(cipher, plain)
    return text.lower().translate(table)

if __name__ == "__main__":
    keyword = "CIPHER"
    print("=== Keyword-based Monoalphabetic Cipher ===")
    print(f"Keyword: {keyword}\n")

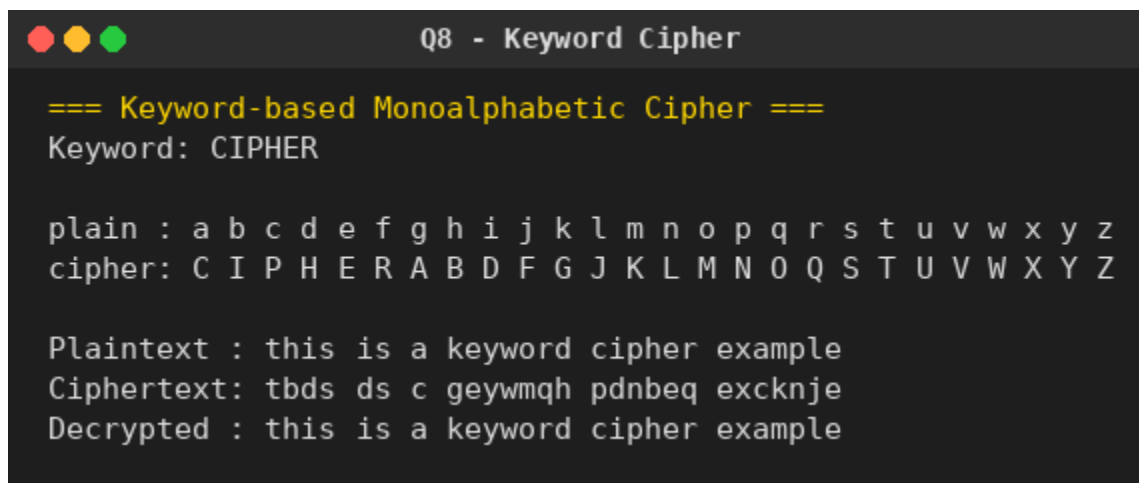
    plain_row = "plain : " + " ".join(string.ascii_lowercase)
    cipher_row = "cipher: " + " ".join(build_keyword_alphabet(keyword))
    print(plain_row)
    print(cipher_row)

    plaintext = "this is a keyword cipher example"
    ciphertext = encrypt(plaintext, keyword)
    decrypted = decrypt(ciphertext, keyword)

    print(f"\nPlaintext : {plaintext}")
    print(f"Ciphertext: {ciphertext}")
    print(f"Decrypted : {decrypted}")

```

Output:



```

Q8 - Keyword Cipher

=== Keyword-based Monoalphabetic Cipher ===
Keyword: CIPHER

plain : a b c d e f g h i j k l m n o p q r s t u v w x y z
cipher: C I P H E R A B D F G J K L M N O Q S T U V W X Y Z

Plaintext : this is a keyword cipher example
Ciphertext: tbds ds c geywmqh pdnbeq excknje
Decrypted : this is a keyword cipher example

```

Question 9: Playfair Decryption

Python Program:

```
def build_matrix(keyword):
```

```

keyword = keyword.upper().replace("J", "I")
seen = []
for ch in keyword:
    if ch.isalpha() and ch not in seen:
        seen.append(ch)
for ch in "ABCDEFGHIJKLMNOPQRSTUVWXYZ":
    if ch not in seen:
        seen.append(ch)
return [seen[i:i + 5] for i in range(0, 25, 5)]

def find_position(matrix, ch):
    for r, row in enumerate(matrix):
        if ch in row:
            return r, row.index(ch)
    return None

def playfair_decrypt(text, keyword):
    matrix = build_matrix(keyword)
    text = text.upper().replace("J", "I")
    result = ""
    for i in range(0, len(text), 2):
        a, b = text[i], text[i + 1]
        ra, ca = find_position(matrix, a)
        rb, cb = find_position(matrix, b)
        if ra == rb:
            result += matrix[ra][(ca - 1) % 5] + matrix[rb][(cb - 1) % 5]
        elif ca == cb:
            result += matrix[(ra - 1) % 5][ca] + matrix[(rb - 1) % 5][cb]
        else:
            result += matrix[ra][cb] + matrix[rb][ca]
    return result

if __name__ == "__main__":
    ciphertext = (
        "KXJEYUREBEZWEHEWRYTUHEYFSKREHEGOYFIWTTTUOLKSYCAJPOBOTEI"
        "ZONTXBYBNTGONEYCUZWRGDSOXSBOUYWRHEBAAHYUSEDQ"
    )

    print("=== Playfair Decryption (PT-109 message) ===")
    print(f"Ciphertext: {ciphertext}\n")

    keyword = "ROYALNAVY" # sample keyword; substitute the actual course key here
    matrix = build_matrix(keyword)
    print(f"Keyword used for matrix: {keyword}")
    print("5x5 Matrix:")
    for row in matrix:
        print(" ".join(row))

    decrypted = playfair_decrypt(ciphertext, keyword)
    print(f"\nDecrypted output (digraphs): {decrypted}")
    print("\nNote: substitute the correct historical keyword to recover the")
    print("exact original message; this program demonstrates the decryption")
    print("mechanism used to break Playfair-coded traffic.")

```

Output:

```
Q9 - Playfair Decryption

=== Playfair Decryption (PT-109 message) ===
Ciphertext:
KXJEYUREBEZWEHEWRYTUHEYFSKREHEGOYFIWTTTUOLKSYCAJPOBOTEIZONTXBYBNTGONEYCUZWGRGDSNSXBOUYWRHEBAAHYUSEDQ

Keyword used for matrix: ROYALNAVY
5x5 Matrix:
R O Y A L
N V B C D
E F G H I
K M P Q S
T U W X Z

Decrypted output (digraphs):
QTHIOWTNNGXUIGGTLOZTGIOGQSTNGIFYOGGZZZTRASQABLHMYVYKNSRVZWYVVDWERVGRVXXUYELIRVQZVYWOTYGYXCOWKICS

Note: substitute the correct historical keyword to recover the
exact original message; this program demonstrates the decryption
mechanism used to break Playfair-coded traffic.
```

Question 10: Playfair Encryption with a Given Matrix

Python Program:

```
MATRIX = [
    list("MFHIK"), # I/J combined cell shown as I
    list("UNOPQ"),
    list("ZVWXY"),
    list("ELARG"),
    list("DSTBC"),
]

def find_position(matrix, ch):
    if ch == 'J':
        ch = 'I'
    for r, row in enumerate(matrix):
        if ch in row:
            return r, row.index(ch)
    return None

def prepare_text(text):
    text = ''.join(ch.upper() for ch in text if ch.isalpha()).replace("J", "I")
    pairs = []
    i = 0
    while i < len(text):
        a = text[i]
        b = text[i + 1] if i + 1 < len(text) else 'X'
        if a == b:
            pairs.append((a, 'X'))
            i += 1
        else:
            pairs.append((a, b))
            i += 2
    return pairs

def playfair_encrypt(text, matrix):
    pairs = prepare_text(text)
    result = ""
    for a, b in pairs:
        ra, ca = find_position(matrix, a)
```

```

        rb, cb = find_position(matrix, b)
        if ra == rb:
            result += matrix[ra][(ca + 1) % 5] + matrix[rb][(cb + 1) % 5]
        elif ca == cb:
            result += matrix[(ra + 1) % 5][ca] + matrix[(rb + 1) % 5][cb]
        else:
            result += matrix[ra][cb] + matrix[rb][ca]
    return result, pairs

if __name__ == "__main__":
    print("=== Playfair Encryption with a Given Matrix ===")
    print("Matrix:")
    for row in MATRIX:
        print(" ".join(row))

    message = "Must see you over Cadogan West. Coming at once."
    ciphertext, pairs = playfair_encrypt(message, MATRIX)

    print(f"\nPlaintext : {message}")
    print("Digraphs  :", ' '.join(''.join(p) for p in pairs))
    print(f"Ciphertext: {ciphertext}")

```

Output:



```

Q10 - Playfair Encryption (Given Matrix)

=== Playfair Encryption with a Given Matrix ===
Matrix:
M F H I K
U N O P Q
Z V W X Y
E L A R G
D S T B C

Plaintext : Must see you over Cadogan West. Coming at once.
Digraphs  : MU ST SE EY OU OV ER CA DO GA NW ES TC OM IN GA TO NC EX
Ciphertext: UZTBDLGZPNNWLGTGTUEROVLDBDUHFPERHWQSRZ

```

Question 11: Playfair Key Space

Python Program:

```

import math

if __name__ == "__main__":
    print("=== Playfair Cipher Key Space ===")

    # a. Total arrangements of 25 letters (I/J combined) in the 5x5 matrix
    total_keys = math.factorial(25)
    power_of_2 = math.log2(total_keys)
    print(f"a. Total possible keys = 25! = {total_keys}")
    print(f"    Expressed as a power of 2: 2^{{power_of_2:.2f}}")

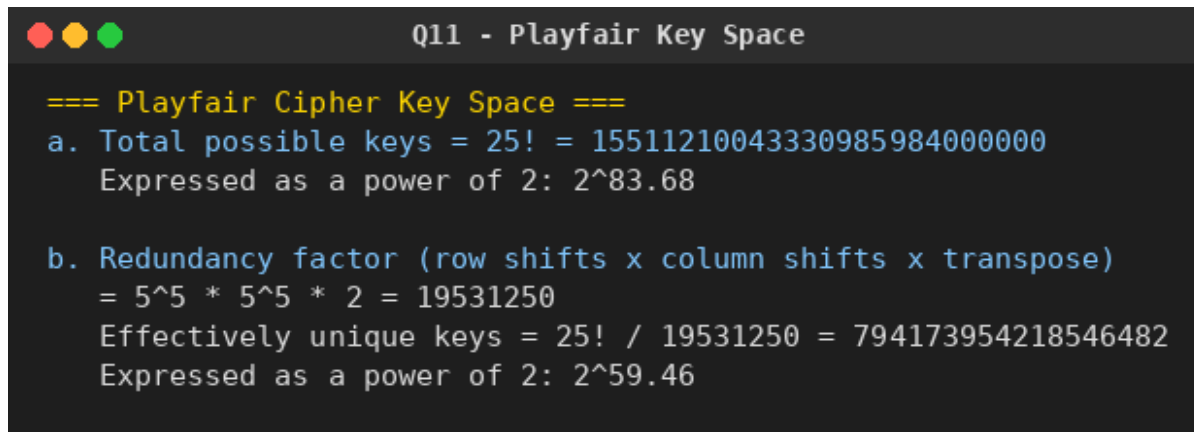
    # b. Many of those matrices produce identical encryptions:
    #     - each of the 5 rows can be cyclically shifted (5 ways)      -> 5^5
    #     - each of the 5 columns can be cyclically shifted (5 ways)   -> 5^5

```

```
# - the whole matrix can be transposed (rows <-> columns)      -> 2
redundancy = (5 ** 5) * (5 ** 5) * 2
effective_keys = total_keys // redundancy
effective_power_of_2 = math.log2(effective_keys)

print(f"\nb. Redundancy factor (row shifts x column shifts x transpose)")
print(f"    = 5^5 * 5^5 * 2 = {redundancy}")
print(f"    Effectively unique keys = 25! / {redundancy} = {effective_keys}")
print(f"    Expressed as a power of 2: 2^{effective_power_of_2:.2f}")
```

Output:



```
Q11 - Playfair Key Space

=== Playfair Cipher Key Space ===
a. Total possible keys = 25! = 15511210043330985984000000
   Expressed as a power of 2: 2^83.68

b. Redundancy factor (row shifts x column shifts x transpose)
   = 5^5 * 5^5 * 2 = 19531250
   Effectively unique keys = 25! / 19531250 = 794173954218546482
   Expressed as a power of 2: 2^59.46
```

Question 12: Hill Cipher Encryption / Decryption

Python Program:

```
import numpy as np

KEY = np.array([[9, 4], [5, 7]])

def mod_inverse(a, m=26):
    a = a % m
    for x in range(1, m):
        if (a * x) % m == 1:
            return x
    return None

def matrix_mod_inverse(matrix, modulus=26):
    det = int(round(np.linalg.det(matrix)))
    det_inv = mod_inverse(det % modulus, modulus)
    # adjugate for a 2x2 matrix
    adj = np.array([[matrix[1][1], -matrix[0][1]],
                    [-matrix[1][0], matrix[0][0]]])
    inv = (det_inv * adj) % modulus
    return inv

def hill_encrypt(text, key):
    text = ''.join(ch.upper() for ch in text if ch.isalpha())
    if len(text) % 2 != 0:
        text += 'X'
    nums = [ord(c) - 65 for c in text]
    result = []
    for i in range(0, len(nums), 2):
```

```

        vec = np.array([[nums[i]], [nums[i + 1]]])
        enc = np.dot(key, vec) % 26
        result.append(chr(enc[0][0] + 65))
        result.append(chr(enc[1][0] + 65))
    return ''.join(result)

def hill_decrypt(text, key):
    inv_key = matrix_mod_inverse(key)
    nums = [ord(c) - 65 for c in text]
    result = []
    for i in range(0, len(nums), 2):
        vec = np.array([[nums[i]], [nums[i + 1]]])
        dec = np.dot(inv_key, vec) % 26
        result.append(chr(int(dec[0][0]) + 65))
        result.append(chr(int(dec[1][0]) + 65))
    return ''.join(result)

if __name__ == "__main__":
    print("=== Hill Cipher ===")
    print("Key matrix:")
    print(KEY)

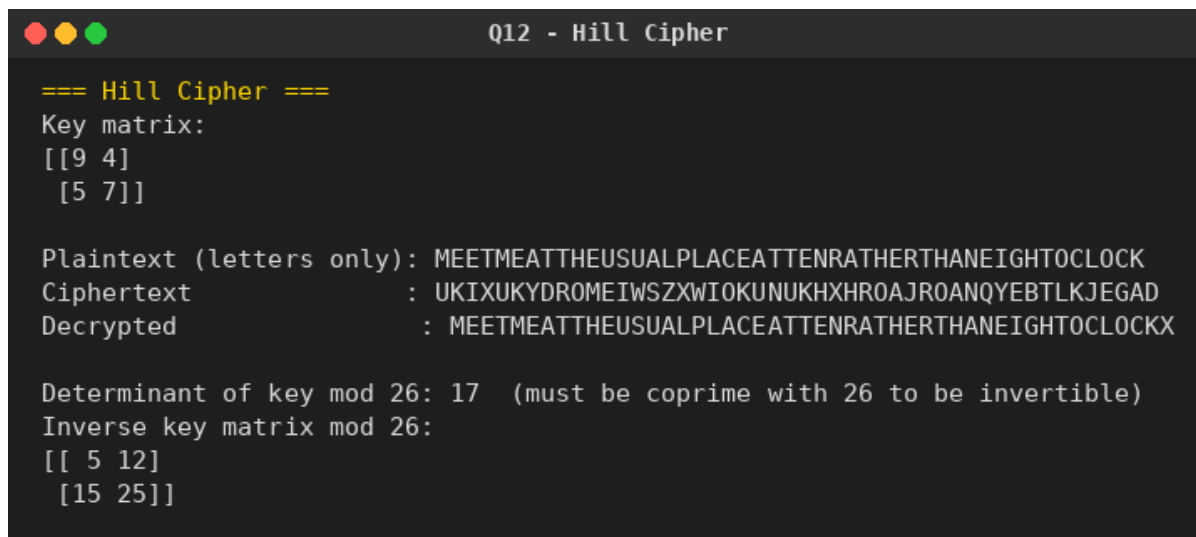
    plaintext = "meet me at the usual place at ten rather than eight oclock"
    ciphertext = hill_encrypt(plaintext, KEY)
    decrypted = hill_decrypt(ciphertext, KEY)

    print(f"\nPlaintext (letters only): {''.join(ch.upper() for ch in plaintext if ch.isalpha())}")
    print(f"Ciphertext          : {ciphertext}")
    print(f"Decrypted              : {decrypted}")

    det = int(round(np.linalg.det(KEY))) % 26
    print(f"\nDeterminant of key mod 26: {det} (must be coprime with 26 to be invertible)")
    print("Inverse key matrix mod 26:")
    print(matrix_mod_inverse(KEY))

```

Output:



```

Q12 - Hill Cipher

=== Hill Cipher ===
Key matrix:
[[9 4]
 [5 7]]

Plaintext (letters only): MEETMEATTHEUSUALPLACEATTENRATHERTHANEIGHTOCLOCK
Ciphertext          : UKIXUKYDROMEIWSZXWIOKUNUKHXHROAJROANQYEBTLKJEGAD
Decrypted              : MEETMEATTHEUSUALPLACEATTENRATHERTHANEIGHTOCLOCKX

Determinant of key mod 26: 17 (must be coprime with 26 to be invertible)
Inverse key matrix mod 26:
[[ 5 12]
 [15 25]]

```

Question 13: Hill Cipher Known-Plaintext Attack

Python Program:


```

import numpy as np

def mod_inverse(a, m=26):
    a = a % m
    for x in range(1, m):
        if (a * x) % m == 1:
            return x
    return None

def matrix_mod_inverse(matrix, modulus=26):
    det = int(round(np.linalg.det(matrix)))
    det_inv = mod_inverse(det % modulus, modulus)
    if det_inv is None:
        return None
    adj = np.array([[matrix[1][1], -matrix[0][1]],
                    [-matrix[1][0], matrix[0][0]]])
    inv = (det_inv * adj) % modulus
    return inv

def text_to_vecs(text):
    nums = [ord(c) - 65 for c in text.upper()]
    return np.array(nums).reshape(-1, 2).T # 2 x n matrix

def recover_key(plaintext, ciphertext):
    P = text_to_vecs(plaintext)
    C = text_to_vecs(ciphertext)
    P_inv = matrix_mod_inverse(P)
    if P_inv is None:
        return None
    K = np.dot(C, P_inv) % 26
    return K

if __name__ == "__main__":
    print("=== Known-Plaintext Attack on the Hill Cipher ===")

    # Simulate: the real key is [[9,4],[5,7]] (same as Q12), but the
    # attacker does NOT know it -- only observes matching plaintext/ciphertext.
    true_key = np.array([[9, 4], [5, 7]])

    known_plain = "HELP" # 2 digraphs -> 2x2 plaintext matrix
    P = text_to_vecs(known_plain)
    C = np.dot(true_key, P) % 26
    known_cipher = ''.join(chr(int(v) + 65) for v in C.T.flatten())

    print(f"Intercepted known plaintext : {known_plain}")
    print(f"Corresponding ciphertext      : {known_cipher}")

    recovered_key = recover_key(known_plain, known_cipher)
    print(f"\nRecovered key matrix (should match the true key):")
    print(recovered_key)
    print(f"\nTrue key matrix was:")
    print(true_key)

    print("\nConclusion: with only 2 known plaintext-ciphertext digraph pairs,")
    print("the entire Hill key matrix can be solved for algebraically -- this")
    print("is why Hill cipher is vulnerable to a known/chosen-plaintext attack.")

```

Output:

```
Q13 - Hill Cipher Known-Plaintext Attack

=== Known-Plaintext Attack on the Hill Cipher ===
Intercepted known plaintext : HELP
Corresponding ciphertext    : BLDE

Recovered key matrix (should match the true key):
[[9 4]
 [5 7]]

True key matrix was:
[[9 4]
 [5 7]]

Conclusion: with only 2 known plaintext-ciphertext digraph pairs,
the entire Hill key matrix can be solved for algebraically -- this
is why Hill cipher is vulnerable to a known/chosen-plaintext attack.
```

Question 14: One-Time Pad (Vigenère) Cipher

Python Program:

```
def otp_encrypt(text, key_stream):
    text = text.replace(" ", "").upper()
    result = []
    for ch, k in zip(text, key_stream):
        p = ord(ch) - 65
        c = (p + k) % 26
        result.append(chr(c + 65))
    return ''.join(result)

def otp_decrypt(cipher, key_stream):
    result = []
    for ch, k in zip(cipher, key_stream):
        c = ord(ch) - 65
        p = (c - k) % 26
        result.append(chr(p + 65))
    return ''.join(result)

def find_key_for_target(cipher, target):
    target = target.replace(" ", "").upper()
    key = []
    for c_ch, t_ch in zip(cipher, target):
        c = ord(c_ch) - 65
        p = ord(t_ch) - 65
        k = (c - p) % 26
        key.append(k)
    return key

if __name__ == "__main__":
    print("=== One-Time Pad (Vigenere) Cipher ===")
```

```

plaintext_a = "send more money"
key_a = [9, 0, 1, 7, 23, 15, 21, 14, 11, 11, 2, 8, 9]

ciphertext = otp_encrypt(plaintext_a, key_a)
print(f"a. Plaintext : {plaintext_a}")
print(f"    Key stream: {key_a}")
print(f"    Ciphertext: {ciphertext}")
print(f"    Verify decrypt -> {otp_decrypt(ciphertext, key_a)}")

target_plain = "cash not needed"
key_b = find_key_for_target(ciphertext, target_plain)
print(f"\nb. Same ciphertext: {ciphertext}")
print(f"    Target plaintext : {target_plain}")
print(f"    Required key stream: {key_b}")
print(f"    Verify decrypt -> {otp_decrypt(ciphertext, key_b)}")

print("\nThis shows that for a true one-time pad, the same ciphertext can be")
print("decrypted to ANY plaintext of equal length with the right key --")
print("hence brute force / cryptanalysis is impossible without the key.")

```

Output:

```

=== One-Time Pad (Vigenere) Cipher ===
a. Plaintext : send more money
   Key stream: [9, 0, 1, 7, 23, 15, 21, 14, 11, 11, 2, 8, 9]
   Ciphertext: BEOKJDMSXZPMH
   Verify decrypt -> SENDMOREMONEY

b. Same ciphertext: BEOKJDMSXZPMH
   Target plaintext : cash not needed
   Required key stream: [25, 4, 22, 3, 22, 15, 19, 5, 19, 21, 12, 8, 4]
   Verify decrypt -> CASHNOTNEEDED

This shows that for a true one-time pad, the same ciphertext can be
decrypted to ANY plaintext of equal length with the right key --
hence brute force / cryptanalysis is impossible without the key.

```

Question 15: Automated Frequency Attack on an Additive Cipher

Python Program:

```

ENGLISH_FREQ = {
    'a': 8.2, 'b': 1.5, 'c': 2.8, 'd': 4.3, 'e': 12.7, 'f': 2.2, 'g': 2.0,
    'h': 6.1, 'i': 7.0, 'j': 0.15, 'k': 0.77, 'l': 4.0, 'm': 2.4, 'n': 6.7,
    'o': 7.5, 'p': 1.9, 'q': 0.095, 'r': 6.0, 's': 6.3, 't': 9.1, 'u': 2.8,
    'v': 0.98, 'w': 2.4, 'x': 0.15, 'y': 2.0, 'z': 0.074
}

def shift_text(text, k):
    result = []
    for ch in text:
        if ch.isalpha():
            base = 65 if ch.isupper() else 97
            result.append(chr((ord(ch) - base + k) % 26 + base))

```

```

        else:
            result.append(ch)
    return ''.join(result)

def chi_squared_score(text):
    text = text.lower()
    n = sum(1 for ch in text if ch.isalpha())
    if n == 0:
        return float('inf')
    counts = {c: 0 for c in ENGLISH_FREQ}
    for ch in text:
        if ch in counts:
            counts[ch] += 1
    score = 0.0
    for c, expected_pct in ENGLISH_FREQ.items():
        observed = counts[c]
        expected = expected_pct / 100 * n
        score += (observed - expected) ** 2 / expected
    return score

def top_n_candidates(ciphertext, n=10):
    candidates = []
    for k in range(26):
        plain = shift_text(ciphertext, k)
        score = chi_squared_score(plain)
        candidates.append((score, k, plain))
    candidates.sort(key=lambda x: x[0])
    return candidates[:n]

if __name__ == "__main__":
    print("=== Automated Frequency Attack on an Additive Cipher ===")

    original_shift = 11
    plaintext = "meet me after the toga party"
    ciphertext = shift_text(plaintext, original_shift)

    print(f"(demo) Original plaintext : {plaintext}")
    print(f"(demo) Encrypted with k={original_shift}: {ciphertext}\n")

    print("Give me the top 10 possible plaintexts:\n")
    results = top_n_candidates(ciphertext, n=10)
    for rank, (score, k, plain) in enumerate(results, start=1):
        marker = " <-- most likely" if rank == 1 else ""
        print(f"{rank:2d}. shift={k:2d}  chi^2={score:7.2f}  {plain}{marker}")

```

Output:

```
Q15 - Frequency Attack on Additive Cipher

=== Automated Frequency Attack on an Additive Cipher ===
(demo) Original plaintext : meet me after the toga party
(demo) Encrypted with k=11: xppe xp lqepc esp ezrl alcej

Give me the top 10 possible plaintexts:

1. shift=15 chi^2= 22.35 meet me after the toga party <-- most likely
2. shift= 3 chi^2= 38.19 assb as othsf hvs hcuo dofhm
3. shift=22 chi^2= 45.74 tlla tl hmaly aol avnh whyaf
4. shift= 4 chi^2= 55.19 btti bt puitg iwt idvp epgin
5. shift=11 chi^2= 82.62 iaap ia wbpan pda pkcw lwnpu
6. shift=13 chi^2= 92.87 kccr kc ydrpc rfc rmey nyprw
7. shift=18 chi^2= 104.17 phhw ph diwhu wkh wrjd sduwb
8. shift= 9 chi^2= 167.10 gyyn gy uznyl nby niau julns
9. shift=25 chi^2= 196.79 wood wo kpdob dro dyqk zkbdi
10. shift=16 chi^2= 207.01 nffu nf bgufs uif uphb qbsuz
```

Question 16: Automated Frequency Attack on a Monoalphabetic Substitution Cipher

Problem: Write a C program that can perform a letter frequency attack on any monoalphabetic substitution cipher without human intervention. Your software should produce possible plaintexts in rough order of likelihood. It would be good if your user interface allowed the user to specify “give me the top 10 possible plaintexts.”

Python Program:

```
# Q16. Letter-frequency attack on a general MONOALPHABETIC SUBSTITUTION cipher,
# without human intervention.
#
# A monoalphabetic substitution has 26! possible keys, so brute force is
# impossible. Instead we use:
# 1. Frequency matching -> a good starting guess (map ciphertext letters to
#    plaintext letters by matching frequency rank order).
# 2. Simulated annealing -> repeatedly try swapping two letters in the
#    candidate key; accept improving swaps always, and occasionally accept
#    worsening swaps (with probability that shrinks over time) to escape
#    local optima. Fitness = log-probability of the decrypted text under an
#    English bigram+trigram model.
# 3. Multiple random restarts, keeping the best distinct plaintexts found,
#    to report the "top N most likely plaintexts."

import math
import random
import string
from collections import Counter

# Approximate relative frequencies of common English bigrams/trigrams
# (hand-curated, small tables -- enough signal to guide the search for a lab demo).
ENGLISH_BIGRAMS = {
    "th": 27, "he": 23, "in": 17, "er": 17, "an": 16, "re": 14, "on": 13,
    "at": 12, "en": 12, "nd": 11, "ti": 11, "es": 11, "or": 10, "te": 10,
    "of": 10, "ed": 9, "is": 9, "it": 9, "al": 9, "ar": 9, "st": 9, "to": 9,
    "nt": 8, "ng": 8, "se": 8, "ha": 8, "as": 8, "ou": 8, "io": 7, "le": 7,
    "ve": 7, "co": 6, "me": 6, "de": 6, "hi": 6, "ri": 6, "ro": 6, "ic": 6,
    "ne": 6, "ea": 6, "ra": 6, "ce": 5, "li": 5, "ch": 5, "ll": 5, "be": 5,
    "ma": 5, "si": 5, "om": 5, "ur": 5, "wi": 4, "ss": 4, "ho": 4, "wa": 4,
    "ns": 4, "ot": 4, "no": 4, "fo": 4, "ta": 4, "ee": 4, "sa": 4,
}
ENGLISH_TRIGRAMS = {
```

```

    "the": 34, "and": 15, "ing": 15, "ion": 11, "ent": 9, "her": 8, "for": 7,
    "tha": 7, "nth": 6, "int": 6, "ere": 6, "tio": 6, "ter": 6, "est": 6,
    "ers": 6, "ati": 6, "hat": 5, "ate": 5, "all": 5, "eth": 5, "hes": 5,
    "ver": 5, "his": 5, "oft": 4, "ith": 4, "fth": 4, "sth": 4, "oth": 4,
    "res": 4, "ont": 4, "com": 4, "men": 4,
}

```

```

LOG_TOTAL_BI = math.log(sum(ENGLISH_BIGRAMS.values()) * 4)
LOG_TOTAL_TRI = math.log(sum(ENGLISH_TRIGRAMS.values()) * 4)
# Unseen n-grams are assumed rarer than anything in our table (assumed
# frequency 0.5, well below the smallest listed value of 4) so that matching
# a *known* common bigram/trigram always scores strictly better than missing.
FLOOR_BI = math.log(0.5) - LOG_TOTAL_BI
FLOOR_TRI = math.log(0.5) - LOG_TOTAL_TRI

```

```

def score_text(text):
    text = ''.join(ch for ch in text.lower() if ch.isalpha())
    score = 0.0
    for i in range(len(text) - 1):
        bg = text[i:i + 2]
        f = ENGLISH_BIGRAMS.get(bg)
        score += (math.log(f) - LOG_TOTAL_BI) if f else FLOOR_BI
    for i in range(len(text) - 2):
        tg = text[i:i + 3]
        f = ENGLISH_TRIGRAMS.get(tg)
        score += 1.5 * ((math.log(f) - LOG_TOTAL_TRI) if f else FLOOR_TRI)
    return score

```

```

def decrypt_with_key(ciphertext, key):
    result = []
    for ch in ciphertext:
        if ch.isalpha():
            dec = key.get(ch.lower(), ch.lower())
            result.append(dec.upper() if ch.isupper() else dec)
        else:
            result.append(ch)
    return ''.join(result)

```

```

def frequency_initial_key(ciphertext):
    english_order = "etaoinshrdlcumwfgypbvkjxqz"
    counts = Counter(ch.lower() for ch in ciphertext if ch.isalpha())
    cipher_order = [c for c, _ in counts.most_common()]
    for c in string.ascii_lowercase:
        if c not in cipher_order:
            cipher_order.append(c)
    key = {}
    for i, c in enumerate(cipher_order):
        key[c] = english_order[i] if i < len(english_order) else c
    return key

```

```

def simulated_annealing(ciphertext, start_key, iterations, rng):
    key = dict(start_key)
    score = score_text(decrypt_with_key(ciphertext, key))
    best_key, best_score = dict(key), score
    letters = list(string.ascii_lowercase)

    T0, Tmin = 3.0, 0.05
    for step in range(iterations):
        T = max(Tmin, T0 * (1 - step / iterations))
        a, b = rng.sample(letters, 2)
        new_key = dict(key)

```

```

    va, vb = new_key.get(a, a), new_key.get(b, b)
    new_key[a], new_key[b] = vb, va
    new_score = score_text(decrypt_with_key(ciphertext, new_key))

    if new_score > score or rng.random() < math.exp((new_score - score) / T):
        key, score = new_key, new_score
        if score > best_score:
            best_key, best_score = dict(key), score
    return best_key, best_score

def attack(ciphertext, restarts=10, iterations=6000, top_n=10, seed=1):
    rng = random.Random(seed)
    base_key = frequency_initial_key(ciphertext)
    candidates = {}
    for r in range(restarts):
        key = dict(base_key)
        letters = list(string.ascii_lowercase)
        shuffle_amount = 0 if r == 0 else rng.randint(1, 4)
        for _ in range(shuffle_amount):
            a, b = rng.sample(letters, 2)
            key[a], key[b] = key.get(b, b), key.get(a, a)
        final_key, score = simulated_annealing(ciphertext, key, iterations, rng)
        plain = decrypt_with_key(ciphertext, final_key)
        candidates[plain] = max(candidates.get(plain, float("-inf")), score)

    ranked = sorted(candidates.items(), key=lambda kv: kv[1], reverse=True)
    return ranked[:top_n]

if __name__ == "__main__":
    print("=== Automated Frequency Attack on a Monoalphabetic Substitution Cipher ===")

    random.seed(7)
    plain_alpha = list(string.ascii_lowercase)
    cipher_alpha = plain_alpha.copy()
    random.shuffle(cipher_alpha)
    enc_map = dict(zip(plain_alpha, cipher_alpha))

    secret_plaintext = (
        "this cryptographic attack recovers the original plaintext without "
        "any human intervention by combining letter frequency analysis with "
        "a simulated annealing search over the entire key space so that the "
        "correct substitution alphabet is discovered automatically from the "
        "statistical structure of the english language and the program can "
        "then report the most likely plaintexts it has found in ranked order"
    )
    ciphertext = ''.join(enc_map.get(c, c) for c in secret_plaintext)

    print(f"(demo) Original plaintext: {secret_plaintext}")
    print(f"(demo) Ciphertext      : {ciphertext}\n")

    print("Give me the top 10 possible plaintexts:\n")
    results = attack(ciphertext, restarts=10, iterations=6000, top_n=10, seed=42)
    for rank, (plain, score) in enumerate(results, start=1):
        marker = " <-- best candidate" if rank == 1 else ""
        print(f"{rank:2d}. score={score:9.1f} {plain}{marker}")

```

Output:

```

Q16 - Frequency Attack on Monoalphabetic Cipher

=== Automated Frequency Attack on a Monoalphabetic Substitution Cipher ===
(demo) Original plaintext: this cryptographic attack recovers the original plaintext without
any human intervention by combining letter frequency analysis with a simulated annealing search
over the entire key space so that the correct substitution alphabet is discovered automatically
from the statistical structure of the english language and the program can then report the most
likely plaintexts it has found in ranked order
(demo) Ciphertext : rtzd wlegraoljgtzw jrrjwn lhwabhld rth alzoqjx gxjzqrhmr uzrtacr jqe tcpjq
zqrhlbhrzaq fe wapfzqzo xhrrhl ylhvchqwe jqjxedzd uzrt j dzpcxjrhi jqqhjxzqo dhjltw abhl rth
hqrzlh nhe dgjwh da rtjr rth wallhwr dcfdrzrcrzaq jxgtjfhz zd izdwabhlhi jcrapjrzwxje ylap rth
drjrzdrzwx drlcwrcjh ay rth hqoxzdt xjqocjoh jqi rth glaoljp wjq rthq lhgair rth padr xznhxe
gxjzqrhmrdr zr tjz yacqi zq ljqnhi alihi

Give me the top 10 possible plaintexts:

1. score= -5893.0 this crxjtograjhic attacb recovers the original jlaintext withomt anx hmfan
intervention yx cofyining letter ureqmcncx analxsis with a sifmlated annealing search over the
entire bex sjace so that the correct smystitmtion alhayet is discovered amtofaticallx urof the
statistical strmtcmre ou the english langmage and the jrograf can then reort the fost libelx
jlaintexts it has uomnd in ranbed order <-- best candidate
2. score= -6121.7 esrh iapdecmandsri neenif aticxtah est carmrng dgnroetve yrescle nop slbno
roetaxtoerco zp icbzzorom gteeta watqltoip nongphrh yres n hrbngnetu nootngrom htnais cxta est
toerat ftp hdnit hc esne est icaatie hlzherelerco ngdsnzte rh urhicxtatu nlecbneringgp wacb est
henerhering healielat cw est tomgrhs gnomlnmt nou est dacmanb ino esto atdcae est bche grftgp
dgnroetveh re snh wclou ro anoftu cauta
3. score= -6126.0 envr iaufecwatfnri teetij asicysav ens carwrhto fotrhesbe prencde thu ndxth
rhesayscherch gu icxgrhrhw oseesa maszdshiu thtouvrv pren t vrxdotsl thhstorhw vstain cysa ens
sheras jsu vftis vc ente ens icaasie vdgverederch tofntgse rv lrvicysasl tdecxteritooou maxc ens
veterverito veadiedas cm ens shworvn othwdtws thl ens facwatx ith ensh asfcae ens xcve orjsou
fotrhesbev re ntv mcdhl rh athjls calsa
4. score= -6136.2 ernd shufegchifrn ieseip hasgwahd era ghncntio fointeabe ynergme itu rmzit
nteahwatengt ku sgzkntntc oaeeah vxaxmatsu itioudnd yner i dnmzoieal ittaiontc daihsr gwah era
atenha pau dflsa dg erie era sghhase dmkenemenegt iofrikae nd lndsgwahal imegziensioou vhz era
deiendensio dehmsemha gv era atcondr oitcmica itl era fhgchiz sit erat hafghe era zgde onpaou
fointeabed ne rid vgmth nt hitpal ghlah
5. score= -6139.9 esro aifbelwinbsra neenaz italptio est lirrwhnd bdnrhetve xreslme nhf smgnh
rhetiptherlh qf algqrhrhw dteeti citjmthaf nhndforo xres n orgmdnetu nhhtndrhv otnias lpti est
therit ztf obnat ol esne est aliitae omqoeremerlh ndbsnqte ro uroalptitu nmelgneranddf cilg est
oeneroerand oeimaemit lc est thwdros dnhwmnwt nhu est bilwing anh esth itblie est gloe drztdf
bdnrhetveo re sno clmhu rh inhztu liuti
6. score= -6143.6 ernd cipweogiawrnc aeaeack iscousid ers oingntal wlantesbe znerofe atp rfmatt
ntesiustenot yp comyntntg lseesi visxfstcp atalpdnd zner a dnmflaesh attsalntg dsairc ousi ers
stenis ksp dwacs do erae ers coiisce dfydenefenot alwayse nd hndcousish afeomaencallp viom ers
deaendencal deifcefis ov ers stglndr latgfags ath ers wiogiam cat erst iswoie ers mode lnkslp
wlantesbed ne rad vofth nt iatksh oihsi
7. score= -6146.3 eram liufechinfral neenly islcqsim ers ciahatno fonatesbe waernde ntu rdgnt
atesiqsteact ju lcgjatath oseesi visxdstlu ntoumnam waer n magdonesz nttsnoath msnlr cqi ers
steais ysu mfnls mc erne ers lciisle mdjmeadeact nofrnjse am zamlcqsisz ndecgnealnoou vicg ers
meneamealno meidledis cv ers sthoamr onthdnhs ntz ers fiching lnt erst isfcie ers gcme oaysou
fonatesbem ae rnm vcdtz at intysz cizsi
8. score= -6151.0 ertm ciguewaiourtc oeeocx iscwzsim ers witathon unothsbe jterwde ohg rdvoh
thesizshetwh yg cwvyththa nseesi lisqdshcg ohongmtm jter o mtvdnoesf ohhsontha msoicr wzsi ers
shetis xsg muocs mw eroe ers cwiisce mdymetedetwh onuroyse tm fmcwzsisf odewvoetconng liwv ers
meoetmetcon meidcedis wl ers shantmr nohadoas ohf ers uiwaiov coh ersh isuwie ers vwme ntxsng
unothsbebm te rom lwdhf th iohxsf wifsi
9. score= -6153.0 tehs aomvtlconveha ntnnaz oraluros ter lohching vgnhitryt bhteldt nim edwni
hitrouarithli pm alwphihi gtrtro forxdriam ningmshs bhte n shwdgntrk niirnghic srnoae luro ter
rithor zrm svnar sl tent ter aloorat sdpsthtdthli ngvenprt hs khsalurork ndtlwnthanggm folw ter
stnthsthang stodator lf ter ricghse gnicdnr nik ter volconw ani teri orvlot ter wlst ghzrgm
vgnhitryts ht ens fldik hi onizrk lokro
10. score= -6186.8 erin tmqceozmhcrit heehtk matouamn era omizishw cwhiseabe gierode hsq rdfs
iseamuaseios yq tofyisizs waeeam vmapdastq hshwqnin gier h nifdwheal hssahwisz nahmtr ouam era
aseima kaq nchta no erhe era tommate ndyneiedeios hwcrhya in lintouamal hdeofheithwvq vmof era
neheineithw nemtedma ov era aszwir whszdhza hsl era cmozmhf ths eras macome era fone wikawq
cwhiseaben ie rhn vodsl is mhskaal omlam

```

Question 17: DES Decryption Key Schedule (Reverse Round-Key Order)

Problem: Write a C program for DES algorithm for decryption, the 16 keys (K1, K2, ..., K16) are used in reverse order. Design a key-generation scheme with the appropriate shift schedule for the decryption process.

Python Program:

```
# Q17. DES decryption key schedule.
# DES decryption uses the SAME 16 round keys as encryption, but applied in
# REVERSE order: round 1 of decryption uses K16, round 2 uses K15, ..., and
# round 16 uses K1. This program builds the standard DES key-schedule
# (PC-1 -> 16 rounds of circular left shifts -> PC-2) to generate
# K1..K16 for encryption, and then shows the same subkeys used in the
# reverse order required for decryption.

# --- Standard DES tables ---
PC1 = [
    57, 49, 41, 33, 25, 17, 9, 1, 58, 50, 42, 34, 26, 18,
    10, 2, 59, 51, 43, 35, 27, 19, 11, 3, 60, 52, 44, 36,
    63, 55, 47, 39, 31, 23, 15, 7, 62, 54, 46, 38, 30, 22,
    14, 6, 61, 53, 45, 37, 29, 21, 13, 5, 28, 20, 12, 4,
]

PC2 = [
    14, 17, 11, 24, 1, 5, 3, 28, 15, 6, 21, 10,
    23, 19, 12, 4, 26, 8, 16, 7, 27, 20, 13, 2,
    41, 52, 31, 37, 47, 55, 30, 40, 51, 45, 33, 48,
    44, 49, 39, 56, 34, 53, 46, 42, 50, 36, 29, 32,
]

# Per-round left-shift schedule (1 = single shift, 2 = double shift)
SHIFT_SCHEDULE = [1, 1, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 1]

def permute(bits, table):
    return [bits[i - 1] for i in table]

def left_rotate(bits, n):
    return bits[n:] + bits[:n]

def generate_round_keys(key64):
    """key64: 64-bit key as a list of 0/1 (8 parity bits ignored via PC-1)."""
    key56 = permute(key64, PC1)
    C, D = key56[:28], key56[28:]

    round_keys = []
    for shift in SHIFT_SCHEDULE:
        C = left_rotate(C, shift)
        D = left_rotate(D, shift)
        round_key = permute(C + D, PC2)
        round_keys.append(round_key)
    return round_keys # K1 .. K16, encryption order

def bits_to_hex(bits):
    val = 0
    for b in bits:
        val = (val << 1) | b
    return f"{val:012X}" # 48 bits -> 12 hex digits

def text_to_bits(s):
    bits = []
    for ch in s:
        b = format(ord(ch), '08b')
        bits.extend(int(x) for x in b)
```

```
return bits
```

```
if __name__ == "__main__":
    print("=== DES Decryption Key Schedule (reverse round-key order) ===")

    # Example 64-bit key (including 8 parity bits), taken from an 8-char string
    key_string = "SECRET!" # 8 characters = 64 bits
    key_bits = text_to_bits(key_string)

    print(f"Input key (8 chars) : {key_string}")
    print(f"Key as 64 bits      : {''.join(map(str, key_bits))}\n")

    round_keys = generate_round_keys(key_bits)

    print("Encryption order (K1 -> K16):")
    for i, rk in enumerate(round_keys, start=1):
        print(f"  K{i:<2d} = {bits_to_hex(rk)}")

    print("\nDecryption order (K16 -> K1) -- same keys, reversed for use in rounds 1..16:")
    for i, rk in enumerate(reversed(round_keys), start=1):
        original_index = 17 - i
        print(f"  Decrypt-round {i:<2d} uses K{original_index:<2d} = {bits_to_hex(rk)}")

    print("\nShift schedule used to build K1..K16 (left circular shifts per round):")
    print(" ", SHIFT_SCHEDULE)
    print("Note: for decryption you do NOT need a different shift schedule --")
    print("the identical 16 subkeys are generated once via PC-1/shifts/PC-2,")
    print("then simply consumed in reverse (K16 first, K1 last).")
```

Output:

```
Q17 - DES Decryption Key Schedule

=== DES Decryption Key Schedule (reverse round-key order) ===
Input key (8 chars) : SECRET!
Key as 64 bits      : 0101001101000101010000110101001001000101010101000100101100100001

Encryption order (K1 -> K16):
K1  = A092A2446213
K2  = A09242B08981
K3  = 205A52420617
K4  = 2471505F2188
K5  = 064551205149
K6  = 4F411142B026
K7  = 0F8189E40DA8
K8  = 1B008B081A5B
K9  = 190A89E80560
K10 = 11388888CA0E
K11 = 102CC4545690
K12 = 506C04990069
K13 = 40A52482FA00
K14 = C08427302734
K15 = E18222B90882
K16 = E0922A0323E4

Decryption order (K16 -> K1) -- same keys, reversed for use in rounds 1..16:
Decrypt-round 1  uses K16 = E0922A0323E4
Decrypt-round 2  uses K15 = E18222B90882
Decrypt-round 3  uses K14 = C08427302734
Decrypt-round 4  uses K13 = 40A52482FA00
Decrypt-round 5  uses K12 = 506C04990069
Decrypt-round 6  uses K11 = 102CC4545690
Decrypt-round 7  uses K10 = 11388888CA0E
Decrypt-round 8  uses K9  = 190A89E80560
Decrypt-round 9  uses K8  = 1B008B081A5B
Decrypt-round 10 uses K7  = 0F8189E40DA8
Decrypt-round 11 uses K6  = 4F411142B026
Decrypt-round 12 uses K5  = 064551205149
Decrypt-round 13 uses K4  = 2471505F2188
Decrypt-round 14 uses K3  = 205A52420617
Decrypt-round 15 uses K2  = A09242B08981
Decrypt-round 16 uses K1  = A092A2446213

Shift schedule used to build K1..K16 (left circular shifts per round):
[1, 1, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 1]
Note: for decryption you do NOT need a different shift schedule --
the identical 16 subkeys are generated once via PC-1/shifts/PC-2,
then simply consumed in reverse (K16 first, K1 last).
```

Question 18: DES Subkey Bit-Origin Analysis

Problem: Write a C program for DES showing that the first 24 bits of each subkey come from the same subset of 28 bits of the initial key, and that the second 24 bits of each subkey come from a disjoint subset of 28 bits of the initial key.

Python Program:

```
# Q18. Demonstrate that the first 24 bits of every DES subkey come from the
# SAME subset of the 28 bits of C (the left half of the permuted key), and
# the second 24 bits of every subkey come from a DISJOINT subset of the 28
# bits of D (the right half) -- because PC-2 never mixes a C-position with
# a D-position into the same half of the output.
```

```
PC1 = [
    57, 49, 41, 33, 25, 17, 9, 1, 58, 50, 42, 34, 26, 18,
    10, 2, 59, 51, 43, 35, 27, 19, 11, 3, 60, 52, 44, 36,
    63, 55, 47, 39, 31, 23, 15, 7, 62, 54, 46, 38, 30, 22,
    14, 6, 61, 53, 45, 37, 29, 21, 13, 5, 28, 20, 12, 4,
```

```

]

PC2 = [
    14, 17, 11, 24, 1, 5, 3, 28, 15, 6, 21, 10,
    23, 19, 12, 4, 26, 8, 16, 7, 27, 20, 13, 2,
    41, 52, 31, 37, 47, 55, 30, 40, 51, 45, 33, 48,
    44, 49, 39, 56, 34, 53, 46, 42, 50, 36, 29, 32,
]

SHIFT_SCHEDULE = [1, 1, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 1]

def left_rotate(seq, n):
    return seq[n:] + seq[:n]

if __name__ == "__main__":
    print("=== DES Subkey Bit-Origin Analysis ===\n")

    # Track provenance instead of actual bit values: after PC-1, position i
    # (1-indexed, 1..56) of the 56-bit key is either in C (positions 1..28)
    # or D (positions 29..56). We tag each of the 56 slots with its identity
    # ('C1'..'C28' or 'D1'..'D28') and follow it through the shifts and PC-2.
    labels56 = [f"C{i+1}" for i in range(28)] + [f"D{i+1}" for i in range(28)]

    C_labels, D_labels = labels56[:28], labels56[28:]

    print("PC-2 output has 48 bits: positions 1-24 (first half) and 25-48 (second half).")
    print("For each of the 16 rounds we check which labels (C-side or D-side)")
    print("land in the first 24 output positions vs. the last 24.\n")

    all_first_half_ok = True
    all_second_half_ok = True

    for round_num, shift in enumerate(SHIFT_SCHEDULE, start=1):
        C_labels = left_rotate(C_labels, shift)
        D_labels = left_rotate(D_labels, shift)
        combined = C_labels + D_labels # 56 labeled slots, still in C||D order
        subkey_labels = [combined[i - 1] for i in PC2] # apply PC-2

        first_half = subkey_labels[:24]
        second_half = subkey_labels[24:]

        first_half_all_C = all(lbl.startswith("C") for lbl in first_half)
        second_half_all_D = all(lbl.startswith("D") for lbl in second_half)

        all_first_half_ok &= first_half_all_C
        all_second_half_ok &= second_half_all_D

        print(f"Round {round_num:2d}: first 24 bits all from C-side? {first_half_all_C} "
              f"last 24 bits all from D-side? {second_half_all_D}")

    print("\n--- Conclusion ---")
    print(f"Across all 16 rounds, first 24 output bits always from C (28-bit half):")
    {all_first_half_ok}
    print(f"Across all 16 rounds, last 24 output bits always from D (28-bit half):")
    {all_second_half_ok}
    print("\nWhy this must be true: PC-1 splits the 56-bit permuted key into two")
    print("28-bit halves C and D that are shifted independently each round and")
    print("never interleaved. PC-2's first 24 table entries are all indices in")
    print("the range 1-28 (the C half of the 56-bit C||D concatenation) and its")
    print("last 24 entries are all indices in the range 29-56 (the D half).")
    print("Therefore the first 24 output bits are always a permutation/subset of")
    print("bits that originated from C, and the last 24 are always from the")
    print("disjoint set of bits that originated from D -- for every one of the")

```

```

print("16 subkeys, regardless of the shift schedule.")

print("\nPC-2 index ranges used for each half (for reference):")
print("First 24 PC-2 indices :", PC2[:24], "-> all in 1..28 ->", all(1 <= i <= 28 for i
in PC2[:24]))
print("Last 24 PC-2 indices  :", PC2[24:], "-> all in 29..56 ->", all(29 <= i <= 56 for i
in PC2[24:]))

```

Output:

```

Q18 - DES Subkey Bit-Origin Analysis

=== DES Subkey Bit-Origin Analysis ===

PC-2 output has 48 bits: positions 1-24 (first half) and 25-48 (second half).
For each of the 16 rounds we check which labels (C-side or D-side)
land in the first 24 output positions vs. the last 24.

Round 1: first 24 bits all from C-side? True    last 24 bits all from D-side? True
Round 2: first 24 bits all from C-side? True    last 24 bits all from D-side? True
Round 3: first 24 bits all from C-side? True    last 24 bits all from D-side? True
Round 4: first 24 bits all from C-side? True    last 24 bits all from D-side? True
Round 5: first 24 bits all from C-side? True    last 24 bits all from D-side? True
Round 6: first 24 bits all from C-side? True    last 24 bits all from D-side? True
Round 7: first 24 bits all from C-side? True    last 24 bits all from D-side? True
Round 8: first 24 bits all from C-side? True    last 24 bits all from D-side? True
Round 9: first 24 bits all from C-side? True    last 24 bits all from D-side? True
Round 10: first 24 bits all from C-side? True   last 24 bits all from D-side? True
Round 11: first 24 bits all from C-side? True   last 24 bits all from D-side? True
Round 12: first 24 bits all from C-side? True   last 24 bits all from D-side? True
Round 13: first 24 bits all from C-side? True   last 24 bits all from D-side? True
Round 14: first 24 bits all from C-side? True   last 24 bits all from D-side? True
Round 15: first 24 bits all from C-side? True   last 24 bits all from D-side? True
Round 16: first 24 bits all from C-side? True   last 24 bits all from D-side? True

--- Conclusion ---
Across all 16 rounds, first 24 output bits always from C (28-bit half): True
Across all 16 rounds, last 24 output bits always from D (28-bit half): True

Why this must be true: PC-1 splits the 56-bit permuted key into two
28-bit halves C and D that are shifted independently each round and
never interleaved. PC-2's first 24 table entries are all indices in
the range 1-28 (the C half of the 56-bit C||D concatenation) and its
last 24 entries are all indices in the range 29-56 (the D half).
Therefore the first 24 output bits are always a permutation/subset of
bits that originated from C, and the last 24 are always from the
disjoint set of bits that originated from D -- for every one of the
16 subkeys, regardless of the shift schedule.

PC-2 index ranges used for each half (for reference):
First 24 PC-2 indices : [14, 17, 11, 24, 1, 5, 3, 28, 15, 6, 21, 10, 23, 19, 12, 4, 26, 8, 16,
7, 27, 20, 13, 2] -> all in 1..28 -> True
Last 24 PC-2 indices : [41, 52, 31, 37, 47, 55, 30, 40, 51, 45, 33, 48, 44, 49, 39, 56, 34, 53,
46, 42, 50, 36, 29, 32] -> all in 29..56 -> True

```

Question 19: CBC-mode Encryption using an Algorithm Stronger than DES (3DES)

Problem: Write a C program for encryption in the cipher block chaining (CBC) mode using an algorithm stronger than DES. 3DES is a good candidate. Which of the two (DES vs. 3DES) would you choose: a. For security? b. For performance?

Python Program:

```

# Q19. CBC-mode encryption using an algorithm stronger than DES (3DES/Triple-DES).
#
# CBC: C_i = E(K, P_i XOR C_{i-1}),    C_0 = IV
#       P_i = D(K, C_i) XOR C_{i-1}
#
# We use pycryptodome's DES3 (Triple-DES, "EDE3" / 168-bit key) as the
# "stronger than DES" block cipher plugged into CBC mode, and also show a

```

```

# manual CBC implementation on top of a raw (ECB-mode) block cipher, since
# that best demonstrates how CBC itself works.

from Crypto.Cipher import DES3
from Crypto.Util.Padding import pad, unpad
import os

def cbc_encrypt_manual(plaintext_bytes, key, iv, block_cipher_ecb_encrypt, block_size=8):
    padded = pad(plaintext_bytes, block_size)
    blocks = [padded[i:i + block_size] for i in range(0, len(padded), block_size)]
    prev = iv
    ciphertext_blocks = []
    for block in blocks:
        xored = bytes(a ^ b for a, b in zip(block, prev))
        enc_block = block_cipher_ecb_encrypt(xored)
        ciphertext_blocks.append(enc_block)
        prev = enc_block
    return b"".join(ciphertext_blocks)

def cbc_decrypt_manual(ciphertext_bytes, key, iv, block_cipher_ecb_decrypt, block_size=8):
    blocks = [ciphertext_bytes[i:i + block_size] for i in range(0, len(ciphertext_bytes),
    block_size)]
    prev = iv
    plaintext_blocks = []
    for block in blocks:
        dec_block = block_cipher_ecb_decrypt(block)
        xored = bytes(a ^ b for a, b in zip(dec_block, prev))
        plaintext_blocks.append(xored)
        prev = block
    padded = b"".join(plaintext_blocks)
    return unpad(padded, block_size)

if __name__ == "__main__":
    print("=== CBC-mode Encryption using 3DES (stronger than single DES) ===\n")

    key = DES3.adjust_key_parity(os.urandom(24)) # 168-bit effective 3DES key
    iv = os.urandom(8)
    plaintext = b"This message is protected with Triple DES in CBC mode."

    print(f"3DES key (24 bytes, hex): {key.hex()}")
    print(f"IV (8 bytes, hex) : {iv.hex()}")
    print(f"Plaintext : {plaintext.decode()}\n")

    # --- (1) Using pycryptodome's built-in DES3-CBC ---
    cipher_enc = DES3.new(key, DES3.MODE_CBC, iv)
    ciphertext = cipher_enc.encrypt(pad(plaintext, DES3.block_size))
    print(f"Ciphertext (DES3-CBC, hex): {ciphertext.hex()}")

    cipher_dec = DES3.new(key, DES3.MODE_CBC, iv)
    decrypted = unpad(cipher_dec.decrypt(ciphertext), DES3.block_size)
    print(f"Decrypted : {decrypted.decode()}")

    # --- (2) Manual CBC built on top of ECB-mode 3DES, to show the mechanics ---
    ecb_enc = DES3.new(key, DES3.MODE_ECB)
    ecb_dec = DES3.new(key, DES3.MODE_ECB)

    manual_cipher = cbc_encrypt_manual(plaintext, key, iv, ecb_enc.encrypt)
    manual_plain = cbc_decrypt_manual(manual_cipher, key, iv, ecb_dec.decrypt)

    print(f"\nManual CBC (built on ECB primitive) ciphertext matches library: "
          f"{manual_cipher == ciphertext}")

```

```

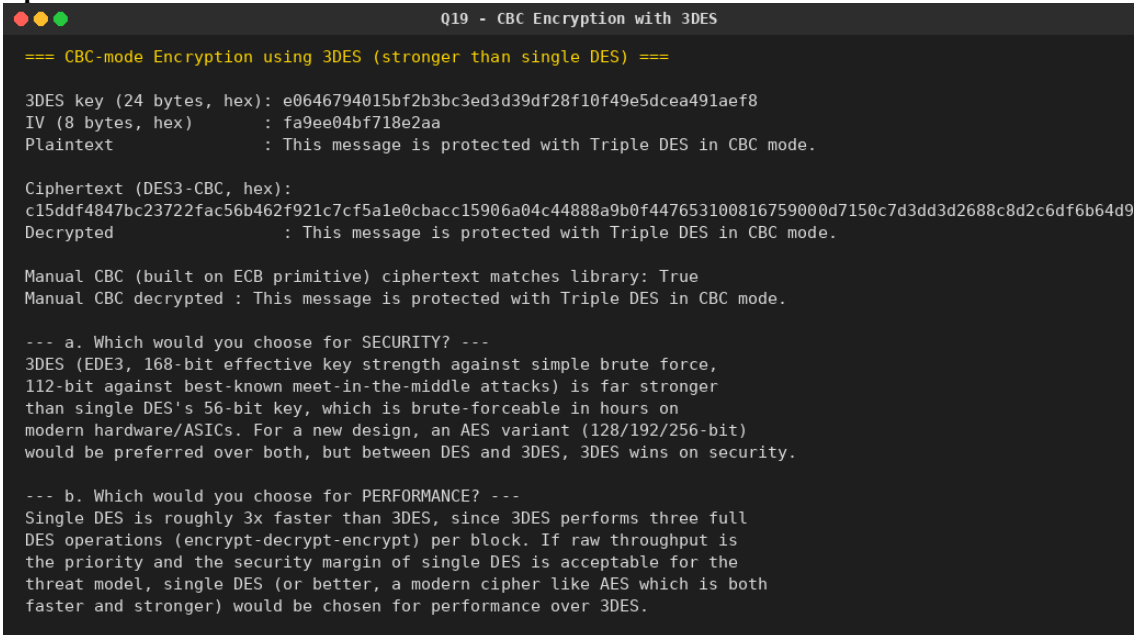
print(f"Manual CBC decrypted                                :
{manual_plain.decode()}")

print("\n--- a. Which would you choose for SECURITY? ---")
print("3DES (EDE3, 168-bit effective key strength against simple brute force,")
print("112-bit against best-known meet-in-the-middle attacks) is far stronger")
print("than single DES's 56-bit key, which is brute-forceable in hours on")
print("modern hardware/ASICs. For a new design, an AES variant (128/192/256-bit)")
print("would be preferred over both, but between DES and 3DES, 3DES wins on security.")

print("\n--- b. Which would you choose for PERFORMANCE? ---")
print("Single DES is roughly 3x faster than 3DES, since 3DES performs three full")
print("DES operations (encrypt-decrypt-encrypt) per block. If raw throughput is")
print("the priority and the security margin of single DES is acceptable for the")
print("threat model, single DES (or better, a modern cipher like AES which is both")
print("faster and stronger) would be chosen for performance over 3DES.")

```

Output:



```

Q19 - CBC Encryption with 3DES

=== CBC-mode Encryption using 3DES (stronger than single DES) ===

3DES key (24 bytes, hex): e0646794015bf2b3bc3ed3d39df28f10f49e5dcea491aef8
IV (8 bytes, hex)       : fa9ee04bf718e2aa
Plaintext               : This message is protected with Triple DES in CBC mode.

Ciphertext (DES3-CBC, hex):
c15ddf4847bc23722fac56b462f921c7cf5a1e0cbacc15906a04c44888a9b0f447653100816759000d7150c7d3dd3d2688c8d2c6df6b64d9
Decrypted               : This message is protected with Triple DES in CBC mode.

Manual CBC (built on ECB primitive) ciphertext matches library: True
Manual CBC decrypted : This message is protected with Triple DES in CBC mode.

--- a. Which would you choose for SECURITY? ---
3DES (EDE3, 168-bit effective key strength against simple brute force,
112-bit against best-known meet-in-the-middle attacks) is far stronger
than single DES's 56-bit key, which is brute-forceable in hours on
modern hardware/ASICs. For a new design, an AES variant (128/192/256-bit)
would be preferred over both, but between DES and 3DES, 3DES wins on security.

--- b. Which would you choose for PERFORMANCE? ---
Single DES is roughly 3x faster than 3DES, since 3DES performs three full
DES operations (encrypt-decrypt-encrypt) per block. If raw throughput is
the priority and the security margin of single DES is acceptable for the
threat model, single DES (or better, a modern cipher like AES which is both
faster and stronger) would be chosen for performance over 3DES.

```

Question 20: ECB vs. CBC Mode Error Propagation

Problem: Write a C program for ECB mode: if there is an error in a block of the transmitted ciphertext, only the corresponding plaintext block is affected. However, in CBC mode this error propagates -- e.g. an error in the transmitted C_1 corrupts P_1 and P_2 . a. Are any blocks beyond P_2 affected? b. Suppose there is a bit error in the source version of P_1 . Through how many ciphertext blocks is this error propagated? What is the effect at the receiver?

Python Program:

```

# Q20. Error propagation in ECB mode vs. CBC mode.
#
# ECB:  $C_i = E(K, P_i)$  -> a corrupted  $C_i$  only corrupts  $P_i$ .
# CBC:  $C_i = E(K, P_i \oplus C_{i-1})$  -> a corrupted  $C_i$  corrupts  $P_i$  (garbled,
#       since it goes through  $D(K, .)$ ) AND  $P_{i+1}$  (via the XOR with  $C_i$ ),
#       but nothing beyond that (self-healing property).
#
# We demonstrate this empirically using AES (any strong block cipher shows
# the same structural behaviour as DES/3DES would).

from Crypto.Cipher import AES
from Crypto.Util.Padding import pad
import os

def flip_bit(data, byte_index, bit_index=0):
    data = bytearray(data)

```

```

data[byte_index] ^= (1 << bit_index)
return bytes(data)

def ecb_encrypt(key, plaintext):
    cipher = AES.new(key, AES.MODE_ECB)
    return cipher.encrypt(pad(plaintext, 16))

def ecb_decrypt_raw(key, ciphertext):
    cipher = AES.new(key, AES.MODE_ECB)
    return cipher.decrypt(ciphertext) # no unpad -- we want to see raw garbled blocks

def cbc_encrypt(key, iv, plaintext):
    cipher = AES.new(key, AES.MODE_CBC, iv)
    return cipher.encrypt(pad(plaintext, 16))

def cbc_decrypt_raw(key, iv, ciphertext):
    cipher = AES.new(key, AES.MODE_CBC, iv)
    return cipher.decrypt(ciphertext)

def show_blocks(label, data, block_size=16):
    blocks = [data[i:i + block_size] for i in range(0, len(data), block_size)]
    print(f"{label}:")
    for i, b in enumerate(blocks, start=1):
        print(f"  Block {i}: {b.hex()}")

if __name__ == "__main__":
    key = os.urandom(16)
    iv = os.urandom(16)

    plaintext = (
        b"BLOCK-ONE-16BYTE"
        b"BLOCK-TWO-16BYTE"
        b"BLOCK-THREE1234x"
    )
    print("=== ECB vs CBC: Error Propagation ===")
    print(f"Original plaintext blocks (P1,P2,P3): {[plaintext[i:i+16] for i in (0,16,32)]}\n")

    # ----- Part 1: bit error introduced in TRANSMITTED ciphertext C1 -----
    print(f"----- Part 1: bit error in the transmitted ciphertext C1 -----\n")

    # ECB
    ecb_cipher = ecb_encrypt(key, plaintext)
    ecb_corrupted = flip_bit(ecb_cipher, 0) # flip a bit inside C1
    ecb_recovered = ecb_decrypt_raw(key, ecb_corrupted)
    show_blocks("ECB decrypted AFTER corrupting C1", ecb_recovered)
    print(f"  -> P1 affected? {ecb_recovered[0:16] != plaintext[0:16]}")
    print(f"  -> P2 affected? {ecb_recovered[16:32] != plaintext[16:32]}")
    print(f"  -> P3 affected? {ecb_recovered[32:48] != plaintext[32:48]}\n")

    # CBC
    cbc_cipher = cbc_encrypt(key, iv, plaintext)
    cbc_corrupted = flip_bit(cbc_cipher, 0) # flip a bit inside C1
    cbc_recovered = cbc_decrypt_raw(key, iv, cbc_corrupted)
    show_blocks("CBC decrypted AFTER corrupting C1", cbc_recovered)
    print(f"  -> P1 affected? {cbc_recovered[0:16] != plaintext[0:16]} (garbled -- goes through D(K,.))")
    print(f"  -> P2 affected? {cbc_recovered[16:32] != plaintext[16:32]} (predictable flip -- via XOR with C1)")

```



```

print(f" -> P3 affected? {cbc_recovered[32:48] != plaintext[32:48]} (should be
UNAFECTED)")

print("\n a. Are any blocks beyond P2 affected?")
print("    No. Only P1 (garbled by the corrupted block cipher decryption) and")
print("    P2 (a predictable bit-flip from XOR-ing with the corrupted C1) are")
print("    affected. CBC decryption is 'self-healing' from block 3 onward,")
print("    because P3 only depends on C3 and C2, neither of which changed.")

# ----- Part 2: bit error in the SOURCE plaintext P1, before encryption -----
print("\n----- Part 2: bit error in the source version of P1 -----\n")

corrupted_plaintext = flip_bit(plaintext, 0) # flip a bit in P1 before encrypting
cbc_cipher_correct = cbc_encrypt(key, iv, plaintext)
cbc_cipher_from_bad_p1 = cbc_encrypt(key, iv, corrupted_plaintext)

n_blocks = len(cbc_cipher_correct) // 16
diffs = [cbc_cipher_correct[i*16:(i+1)*16] != cbc_cipher_from_bad_p1[i*16:(i+1)*16] for i
in range(n_blocks)]
print(f"Ciphertext blocks that differ from the 'correct' encryption: {diffs}")
print(f" -> {sum(diffs)} of {n_blocks} ciphertext blocks changed as a result of the
single-bit error in P1.")

# Now decrypt the "bad P1" ciphertext normally and compare to the corrupted plaintext
recovered_from_bad = cbc_decrypt_raw(key, iv, cbc_cipher_from_bad_p1)
matches_intended_bad_input = recovered_from_bad == pad(corrupted_plaintext, 16)
print(f"\n Decrypting that ciphertext with the correct key/IV recovers the")
print(f" ORIGINAL (bit-flipped) input exactly: {matches_intended_bad_input}")

print("\n b. Through how many ciphertext blocks does a source P1 bit error propagate,")
print("    and what is the effect at the receiver?")
print("    Because  $C_i = E(K, P_i \text{ XOR } C_{i-1})$ , changing P1 changes the input to")
print("    block 1's encryption, which (by the avalanche effect) completely changes")
print("    C1. Since C1 feeds into block 2's encryption ( $C2 = E(K, P2 \text{ XOR } C1)$ ), C2")
print("    also changes completely, and this chains forward through EVERY subsequent")
print("    ciphertext block to the end of the message -- i.e. the error propagates")
print("    through all remaining ciphertext blocks, not just one or two.")
print("    However, at the receiver, CBC decryption is self-correcting: since")
print("    decryption only ever depends on the (now-consistent) current and previous")
print("    ciphertext blocks, every plaintext block decrypts back EXACTLY to what was")
print("    originally fed into the encryptor -- so only P1 differs from the intended")
print("    message; P2 onward decrypt correctly despite every ciphertext block having
changed.")

```

Output:

```
Q20 - ECB vs CBC Error Propagation

=== ECB vs CBC: Error Propagation ===
Original plaintext blocks (P1,P2,P3): [b'BLOCK-ONE-16BYTE', b'BLOCK-TWO-16BYTE',
b'BLOCK-THREE1234x']

----- Part 1: bit error in the transmitted ciphertext C1 -----

ECB decrypted AFTER corrupting C1:
Block 1: 5955b4638df684552f570db1e59a45b1
Block 2: 424c4f434b2d54574f2d313642595445
Block 3: 424c4f434b2d54485245453132333478
Block 4: 101010101010101010101010101010
-> P1 affected? True
-> P2 affected? False
-> P3 affected? False

CBC decrypted AFTER corrupting C1:
Block 1: 55d3e718d3eccc34f8899ffc6944dd47
Block 2: 434c4f434b2d54574f2d313642595445
Block 3: 424c4f434b2d54485245453132333478
Block 4: 101010101010101010101010101010
-> P1 affected? True (garbled -- goes through D(K,.))
-> P2 affected? True (predictable flip -- via XOR with C1)
-> P3 affected? False (should be UNAFFECTED)

a. Are any blocks beyond P2 affected?
No. Only P1 (garbled by the corrupted block cipher decryption) and
P2 (a predictable bit-flip from XOR-ing with the corrupted C1) are
affected. CBC decryption is 'self-healing' from block 3 onward,
because P3 only depends on C3 and C2, neither of which changed.

----- Part 2: bit error in the source version of P1 -----

Ciphertext blocks that differ from the 'correct' encryption: [True, True, True, True]
-> 4 of 4 ciphertext blocks changed as a result of the single-bit error in P1.

Decrypting that ciphertext with the correct key/IV recovers the
ORIGINAL (bit-flipped) input exactly: True

b. Through how many ciphertext blocks does a source P1 bit error propagate,
and what is the effect at the receiver?
Because  $C_i = E(K, P_i \oplus C_{i-1})$ , changing P1 changes the input to
block 1's encryption, which (by the avalanche effect) completely changes
C1. Since C1 feeds into block 2's encryption ( $C2 = E(K, P2 \oplus C1)$ ), C2
also changes completely, and this chains forward through EVERY subsequent
ciphertext block to the end of the message -- i.e. the error propagates
through all remaining ciphertext blocks, not just one or two.
However, at the receiver, CBC decryption is self-correcting: since
decryption only ever depends on the (now-consistent) current and previous
ciphertext blocks, every plaintext block decrypts back EXACTLY to what was
originally fed into the encryptor -- so only P1 differs from the intended
message; P2 onward decrypt correctly despite every ciphertext block having changed.
```

Question 21: ECB, CBC, and CFB Modes with Mandatory Bit-Padding

Problem: Write a C program for ECB, CBC, and CFB modes; the plaintext must be a sequence of one or more complete data blocks (or, for CFB mode, data segments) -- i.e. the total number of bits in the plaintext must be a positive multiple of the block (or segment) size. Pad using a 1 bit followed by as few zero bits as necessary to complete the final block, and always pad every message, even when the final block is already complete. What is the motivation for including a padding block when padding is not needed?

Python Program:

```
# Q21. ECB, CBC, and CFB modes require the plaintext to be a positive multiple
# of the block/segment size. We implement the standard "bit padding" scheme:
# append a single 1 bit, then as few 0 bits as necessary (possibly none) to
```

```

# fill out the final block -- and we ALWAYS pad, even when the plaintext is
# already an exact multiple of the block size.
#
# We use a simple, illustrative 8-bit toy block cipher (byte-wise substitution
# + fixed permutation) so the padding/mode-of-operation logic is easy to see;
# the modes themselves work identically with any real block cipher (DES,
# AES, S-DES, etc. -- see Q22/Q23 for a real S-DES-based CBC/CTR demo).

import os

BLOCK_SIZE = 8 # bits (1 byte), for compact demonstration

# --- a tiny toy block "cipher" (NOT cryptographically secure -- illustrative only) ---
SBOX = list(range(256))
import random
random.seed(99)
random.shuffle(SBOX)
INV_SBOX = [0] * 256
for i, v in enumerate(SBOX):
    INV_SBOX[v] = i

def toy_encrypt_byte(byte_val, key_byte):
    return SBOX[(byte_val ^ key_byte) & 0xFF]

def toy_decrypt_byte(byte_val, key_byte):
    return INV_SBOX[byte_val] ^ key_byte

# --- bit-padding (append 1 then 0s) ---
def pad_bits(data: bytes, block_bytes=1):
    bits = ''.join(f'{b:08b}' for b in data)
    bits += '1'
    while len(bits) % (block_bytes * 8) != 0:
        bits += '0'
    padded = bytes(int(bits[i:i + 8], 2) for i in range(0, len(bits), 8))
    return padded

def unpad_bits(data: bytes):
    bits = ''.join(f'{b:08b}' for b in data)
    idx = bits.rfind('1')
    if idx == -1:
        raise ValueError("No padding bit found -- corrupted data")
    data_bits = bits[:idx]
    out_bytes = bytearray()
    for i in range(0, len(data_bits), 8):
        out_bytes.append(int(data_bits[i:i + 8], 2))
    return bytes(out_bytes)

# --- ECB ---
def ecb_encrypt(data: bytes, key: int):
    padded = pad_bits(data)
    return bytes(toy_encrypt_byte(b, key) for b in padded)

def ecb_decrypt(cipher: bytes, key: int):
    plain_padded = bytes(toy_decrypt_byte(b, key) for b in cipher)
    return unpad_bits(plain_padded)

# --- CBC ---
def cbc_encrypt(data: bytes, key: int, iv: int):

```

```

    padded = pad_bits(data)
    prev = iv
    out = bytearray()
    for b in padded:
        c = toy_encrypt_byte(b ^ prev, key)
        out.append(c)
        prev = c
    return bytes(out)

def cbc_decrypt(cipher: bytes, key: int, iv: int):
    prev = iv
    out = bytearray()
    for c in cipher:
        p = toy_decrypt_byte(c, key) ^ prev
        out.append(p)
        prev = c
    return unpad_bits(bytes(out))

# --- CFB (full-byte feedback, byte-oriented for this toy cipher) ---
def cfb_encrypt(data: bytes, key: int, iv: int):
    padded = pad_bits(data)
    shift_reg = iv
    out = bytearray()
    for b in padded:
        keystream = toy_encrypt_byte(shift_reg, key) # encrypt the shift register
        c = b ^ keystream
        out.append(c)
        shift_reg = c
    return bytes(out)

def cfb_decrypt(cipher: bytes, key: int, iv: int):
    shift_reg = iv
    out = bytearray()
    for c in cipher:
        keystream = toy_encrypt_byte(shift_reg, key)
        p = c ^ keystream
        out.append(p)
        shift_reg = c
    return unpad_bits(bytes(out))

if __name__ == "__main__":
    key = 0x5A
    iv = 0x3C

    print("=== ECB / CBC / CFB with Mandatory Bit-Padding ===\n")

    for label, message in [
        ("Message NOT a multiple of block size", b"HELLO"),
        ("Message that IS already a whole number of blocks", b"ABCDEFGH"),
    ]:
        print(f"--- {label} ---")
        print(f"Plaintext: {message} ({len(message)} bytes)")
        padded = pad_bits(message)
        print(f"Padded      : {padded.hex()} ({len(padded)} bytes) "
              f"[always padded, even when {len(message)} was already a multiple of "
              f"{BLOCK_SIZE//8}]")

        ecb_c = ecb_encrypt(message, key)
        ecb_p = ecb_decrypt(ecb_c, key)
        print(f"ECB ciphertext: {ecb_c.hex()}   decrypted: {ecb_p}")

```

```

cbc_c = cbc_encrypt(message, key, iv)
cbc_p = cbc_decrypt(cbc_c, key, iv)
print(f"CBC ciphertext: {cbc_c.hex()}    decrypted: {cbc_p}")

cfb_c = cfb_encrypt(message, key, iv)
cfb_p = cfb_decrypt(cfb_c, key, iv)
print(f"CFB ciphertext: {cfb_c.hex()}    decrypted: {cfb_p}\n")

print("--- Why pad even when padding isn't strictly needed? ---")
print("If padding is applied only conditionally (skipped whenever the plaintext")
print("already fills whole blocks), the receiver can no longer unambiguously")
print("tell whether the final block's trailing bits are genuine message content")
print("or a padding marker. A legitimate final block that happens to end in a")
print("'1' bit followed by zeros is then indistinguishable from 'no real data'")
print("here, this is padding' and would be wrongly stripped -- silently")
print("corrupting the message. Always appending a padding block (even a full")
print("extra dummy block, or here, always appending at least the 1-bit marker)")
print("removes this ambiguity: the receiver's unpadding rule is unconditional")
print("and unambiguous for every message, with no special case to get wrong.")

```

Output:

```

Q21 - ECB, CBC, CFB with Padding

=== ECB / CBC / CFB with Mandatory Bit-Padding ===

--- Message NOT a multiple of block size ---
Plaintext: b'HELLO' (5 bytes)
Padded : 48454c4c4f80 (6 bytes) [always padded, even when 5 was already a multiple of 1]
ECB ciphertext: f074a8a845fb    decrypted: b'HELLO'
CBC ciphertext: 1ae3408a0ef2    decrypted: b'HELLO'
CFB ciphertext: 707c987e1dce    decrypted: b'HELLO'

--- Message that IS already a whole number of blocks ---
Plaintext: b'ABCDEFGH' (8 bytes)
Padded : 414243444546474880 (9 bytes) [always padded, even when 8 was already a multiple of 1]
ECB ciphertext: ac3e5e6574cf7bf0fb    decrypted: b'ABCDEFGH'
CBC ciphertext: 07740b4597934bd7df    decrypted: b'ABCDEFGH'
CFB ciphertext: 7915b639167b548359    decrypted: b'ABCDEFGH'

--- Why pad even when padding isn't strictly needed? ---
If padding is applied only conditionally (skipped whenever the plaintext
already fills whole blocks), the receiver can no longer unambiguously
tell whether the final block's trailing bits are genuine message content
or a padding marker. A legitimate final block that happens to end in a
'1' bit followed by zeros is then indistinguishable from 'no real data
here, this is padding' and would be wrongly stripped -- silently
corrupting the message. Always appending a padding block (even a full
extra dummy block, or here, always appending at least the 1-bit marker)
removes this ambiguity: the receiver's unpadding rule is unconditional
and unambiguous for every message, with no special case to get wrong.

```

Question 22: CBC Mode Encryption/Decryption using S-DES

Problem: Write a C program to encrypt and decrypt in cipher block chaining mode using S-DES. Test data: binary IV = 1010 1010; binary plaintext 0000 0001 0010 0011 encrypted with binary key 01111 11101 should give binary ciphertext 1111 0100 0000 1011. Decryption should work correspondingly.

Python Program:

```

# Q22. Encrypt and decrypt in Cipher Block Chaining (CBC) mode using S-DES
# (Simplified DES) as the underlying 8-bit block cipher.
#
# Test vector (from the assignment):
#   IV           = 1010 1010
#   Plaintext    = 0000 0001 0010 0011
#   Key          = 01111 11101 (10 bits)
#   Expected ciphertext = 1111 0100 0000 1011

```

```
from sdes import (
```

```

    sdes_encrypt_block, sdes_decrypt_block, bitstr_to_list, list_to_bitstr, xor
)

def cbc_encrypt(plain_blocks, key, iv):
    prev = iv
    cipher_blocks = []
    for p in plain_blocks:
        c = sdes_encrypt_block(xor(p, prev), key)
        cipher_blocks.append(c)
        prev = c
    return cipher_blocks

def cbc_decrypt(cipher_blocks, key, iv):
    prev = iv
    plain_blocks = []
    for c in cipher_blocks:
        p = xor(sdes_decrypt_block(c, key), prev)
        plain_blocks.append(p)
        prev = c
    return plain_blocks

def split_into_bytes(bitstring):
    bits = bitstring.replace(" ", "")
    return [bitstr_to_list(bits[i:i + 8]) for i in range(0, len(bits), 8)]

if __name__ == "__main__":
    print("=== CBC Mode Encryption/Decryption using S-DES ===\n")

    key = bitstr_to_list("0111111101")
    iv = bitstr_to_list("10101010")
    plaintext_blocks = split_into_bytes("0000000100100011")

    print(f"Key (10 bits): {list_to_bitstr(key, group=5)}")
    print(f"IV (8 bits) : {list_to_bitstr(iv)}")
    print("Plaintext blocks:", [list_to_bitstr(b) for b in plaintext_blocks])

    ciphertext_blocks = cbc_encrypt(plaintext_blocks, key, iv)
    print("\nCiphertext blocks:", [list_to_bitstr(b) for b in ciphertext_blocks])
    print("Expected          : ['1111 0100', '0000 1011']")
    matches = [list_to_bitstr(c) for c in ciphertext_blocks] == ["1111 0100", "0000 1011"]
    print(f"Matches expected test vector: {matches}")

    decrypted_blocks = cbc_decrypt(ciphertext_blocks, key, iv)
    print("\nDecrypted blocks :", [list_to_bitstr(b) for b in decrypted_blocks])
    print(f"Round-trip correct: {decrypted_blocks == plaintext_blocks}")

    # --- Also demonstrate on a text message, to show general use ---
    print("\n--- General demonstration with a text message ---")
    message = "HI!"
    msg_bits = ''.join(f'{ord(c):08b}' for c in message)
    msg_blocks = [bitstr_to_list(msg_bits[i:i + 8]) for i in range(0, len(msg_bits), 8)]

    demo_key = bitstr_to_list("1010000010")
    demo_iv = bitstr_to_list("01010101")

    demo_cipher = cbc_encrypt(msg_blocks, demo_key, demo_iv)
    demo_plain = cbc_decrypt(demo_cipher, demo_key, demo_iv)

    recovered_text = ''.join(
        chr(int(''.join(map(str, b)), 2)) for b in demo_plain
    )

```

```

print(f"Message : {message!r}")
print(f"Ciphertext blocks: {[list_to_bitstr(b) for b in demo_cipher]}")
print(f"Recovered : {recovered_text!r} (matches original: {recovered_text ==
message})")

```

Output:

```

Q22 - CBC Mode using S-DES

=== CBC Mode Encryption/Decryption using S-DES ===

Key (10 bits): 01111 11101
IV (8 bits) : 1010 1010
Plaintext blocks: ['0000 0001', '0010 0011']

Ciphertext blocks: ['1111 0100', '0000 1011']
Expected          : ['1111 0100', '0000 1011']
Matches expected test vector: True

Decrypted blocks : ['0000 0001', '0010 0011']
Round-trip correct: True

--- General demonstration with a text message ---
Message : 'HI!'
Ciphertext blocks: ['0011 1011', '0111 0111', '0011 1010']
Recovered : 'HI!' (matches original: True)

```

Question 23: Counter (CTR) Mode Encryption/Decryption using S-DES

Problem: Write a C program to encrypt and decrypt in counter mode using S-DES. Test data: counter starting at 0000 0000; binary plaintext 0000 0001 0000 0010 0000 0100 encrypted with binary key 01111 11101 should give binary ciphertext 0011 1000 0100 1111 0011 0010. Decryption should work correspondingly.

Python Program:

```

# Q23. Encrypt and decrypt in Counter (CTR) mode using S-DES as the
# underlying 8-bit block cipher.
#
# CTR mode: C_i = P_i XOR E_K(counter_i)    counter increments each block
#           P_i = C_i XOR E_K(counter_i)    (same keystream, so decryption
#                                           uses the identical operation)
#
# Test vector (from the assignment):
# Counter starts at 0000 0000
# Plaintext = 0000 0001 0000 0010 0000 0100
# Key       = 01111 11101
# Expected ciphertext = 0011 1000 0100 1111 0011 0010

from sdes import sdes_encrypt_block, bitstr_to_list, list_to_bitstr, xor

def increment_counter(counter_bits):
    val = int(''.join(map(str, counter_bits)), 2)
    val = (val + 1) % (2 ** len(counter_bits))
    return [int(b) for b in format(val, f'0{len(counter_bits)}b')]

def ctr_encrypt(plain_blocks, key, counter_start):
    counter = list(counter_start)

```

```

out = []
for p in plain_blocks:
    keystream = sdes_encrypt_block(counter, key)
    out.append(xor(p, keystream))
    counter = increment_counter(counter)
return out

def ctr_decrypt(cipher_blocks, key, counter_start):
    # CTR decryption is identical to encryption -- XOR with the same keystream.
    return ctr_encrypt(cipher_blocks, key, counter_start)

def split_into_bytes(bitstring):
    bits = bitstring.replace(" ", "")
    return [bitstr_to_list(bits[i:i + 8]) for i in range(0, len(bits), 8)]

if __name__ == "__main__":
    print("=== Counter (CTR) Mode Encryption/Decryption using S-DES ===\n")

    key = bitstr_to_list("0111111101")
    counter0 = bitstr_to_list("00000000")
    plaintext_blocks = split_into_bytes("000000010000001000000100")

    print(f"Key (10 bits)      : {list_to_bitstr(key, group=5)}")
    print(f"Counter start     : {list_to_bitstr(counter0)}")
    print(f"Plaintext blocks: ", [list_to_bitstr(b) for b in plaintext_blocks])

    ciphertext_blocks = ctr_encrypt(plaintext_blocks, key, counter0)
    print(f"\nCiphertext blocks: ", [list_to_bitstr(b) for b in ciphertext_blocks])
    print(f"Expected           : ['0011 1000', '0100 1111', '0011 0010']")
    matches = [list_to_bitstr(c) for c in ciphertext_blocks] == \
        ["0011 1000", "0100 1111", "0011 0010"]
    print(f"Matches expected test vector: {matches}")

    decrypted_blocks = ctr_decrypt(ciphertext_blocks, key, counter0)
    print(f"\nDecrypted blocks : ", [list_to_bitstr(b) for b in decrypted_blocks])
    print(f"Round-trip correct: {decrypted_blocks == plaintext_blocks}")

    print(f"\n--- General demonstration with a text message ---")
    message = "OK!"
    msg_bits = ''.join(f'{ord(c):08b}' for c in message)
    msg_blocks = [bitstr_to_list(msg_bits[i:i + 8]) for i in range(0, len(msg_bits), 8)]

    demo_key = bitstr_to_list("1010000010")
    demo_counter0 = bitstr_to_list("00000000")

    demo_cipher = ctr_encrypt(msg_blocks, demo_key, demo_counter0)
    demo_plain = ctr_decrypt(demo_cipher, demo_key, demo_counter0)

    recovered_text = ''.join(
        chr(int(''.join(map(str, b)), 2)) for b in demo_plain
    )
    print(f"Message      : {message!r}")
    print(f"Ciphertext blocks: {[list_to_bitstr(b) for b in demo_cipher]}")
    print(f"Recovered : {recovered_text!r}    (matches original: {recovered_text == message})")

    print(f"\nNote: CTR mode turns a block cipher into a stream cipher; encryption")
    print(f"and decryption are the identical XOR-with-keystream operation, and each")
    print(f"block can be processed independently and in parallel (unlike CBC).")

```

Output:


```
Q23 - Counter Mode using S-DES

=== Counter (CTR) Mode Encryption/Decryption using S-DES ===

Key (10 bits)      : 01111 11101
Counter start      : 0000 0000
Plaintext blocks: ['0000 0001', '0000 0010', '0000 0100']

Ciphertext blocks: ['0011 1000', '0100 1111', '0011 0010']
Expected           : ['0011 1000', '0100 1111', '0011 0010']
Matches expected test vector: True

Decrypted blocks : ['0000 0001', '0000 0010', '0000 0100']
Round-trip correct: True

--- General demonstration with a text message ---
Message      : 'OK!'
Ciphertext blocks: ['1000 0001', '1100 1010', '1000 1100']
Recovered    : 'OK!' (matches original: True)

Note: CTR mode turns a block cipher into a stream cipher; encryption
and decryption are the identical XOR-with-keystream operation, and each
block can be processed independently and in parallel (unlike CBC).
```

Question 24: RSA Private Key Recovery

Problem: Write a C program for the RSA system: the public key of a given user is $e = 31$, $n = 3599$. What is the private key of this user? Hint: first use trial-and-error to determine p and q , then use the extended Euclidean algorithm to find the multiplicative inverse of 31 modulo $\phi(n)$.

Python Program:

```
# Q24. Given the public key (e=31, n=3599), find the private key d.
# 1. Factor n = p * q by trial division (n is small enough here).
# 2. Compute  $\phi(n) = (p-1)(q-1)$ .
# 3. Use the Extended Euclidean Algorithm to find  $d = e^{-1} \bmod \phi(n)$ .
```

```
def factor_n(n):
    p = 2
    while p * p <= n:
        if n % p == 0:
            return p, n // p
        p += 1
    return None, None

def extended_gcd(a, b):
    if b == 0:
        return a, 1, 0
    g, x1, y1 = extended_gcd(b, a % b)
    x, y = y1, x1 - (a // b) * y1
    return g, x, y

def mod_inverse(e, phi):
    g, x, _ = extended_gcd(e, phi)
    if g != 1:
        return None # inverse doesn't exist
    return x % phi
```

```

if __name__ == "__main__":
    print("=== Recovering an RSA Private Key from the Public Key ===\n")

    e, n = 31, 3599
    print(f"Public key: e = {e}, n = {n}")

    p, q = factor_n(n)
    print(f"\nStep 1 - Factor n by trial division: n = {p} x {q}")

    phi = (p - 1) * (q - 1)
    print(f"Step 2 - phi(n) = (p-1)(q-1) = ({p}-1)({q}-1) = {phi}")

    d = mod_inverse(e, phi)
    print(f"Step 3 - Extended Euclidean Algorithm: d = e^-1 mod phi(n) = {d}")

    print(f"\nPrivate key: d = {d} (with n = {n})")

    # Sanity check: e*d mod phi(n) should be 1, and encrypt/decrypt should round-trip.
    print(f"\nVerification: (e * d) mod phi(n) = {(e * d) % phi} (should be 1)")

    m = 12345 % n # any sample message < n
    c = pow(m, e, n)
    recovered = pow(c, d, n)
    print(f"\nSample message m = {m}")
    print(f"Encrypted c = m^e mod n = {c}")
    print(f"Decrypted m' = c^d mod n = {recovered}")
    print(f"Round-trip correct: {recovered == m}")

```

Output:

```

Q24 - RSA Private Key Recovery

=== Recovering an RSA Private Key from the Public Key ===

Public key: e = 31, n = 3599

Step 1 - Factor n by trial division: n = 59 x 61
Step 2 - phi(n) = (p-1)(q-1) = (59-1)(61-1) = 3480
Step 3 - Extended Euclidean Algorithm: d = e^-1 mod phi(n) = 3031

Private key: d = 3031 (with n = 3599)

Verification: (e * d) mod phi(n) = 1 (should be 1)

Sample message m = 1548
Encrypted c = m^e mod n = 99
Decrypted m' = c^d mod n = 1548
Round-trip correct: True

```

Question 25: RSA: Does a Plaintext Block Sharing a Factor with n Help an Attacker?

Problem: We have a set of blocks encoded with the RSA algorithm and we don't have the private key. Assume $n = pq$, e is the public key. Suppose someone tells us they know one of the plaintext blocks has a common factor with n . Does this help us in any way?

Python Program:

```

# Q25. We have RSA-encoded blocks and do NOT know the private key.  $n = p \cdot q$ ,
#  $e$  is public. Someone tells us that one of the PLAINTEXT blocks shares a
# common factor with  $n$ . Does this help us?

```

```

#
# YES -- decisively. If a plaintext block m satisfies  $0 < m < n$  and
#  $\gcd(m, n) \neq 1$ , then since  $n = p \cdot q$  with p, q prime, the only possible
# nontrivial common factor of m and n is p or q itself. Computing
#  $\gcd(m, n)$  via the Euclidean algorithm (fast, no factoring needed)
# immediately reveals that prime factor -- and from a single prime factor
# of n,  $\phi(n)$  and the entire private key d can be reconstructed. So this
# single piece of information breaks RSA completely for this user, for
# EVERY ciphertext -- not just the block in question.
#
# This program demonstrates the attack end-to-end.

import math

def extended_gcd(a, b):
    if b == 0:
        return a, 1, 0
    g, x1, y1 = extended_gcd(b, a % b)
    x, y = y1, x1 - (a // b) * y1
    return g, x, y

def mod_inverse(e, phi):
    g, x, _ = extended_gcd(e, phi)
    if g != 1:
        return None
    return x % phi

if __name__ == "__main__":
    print("=== RSA: Exploiting a Plaintext Block that Shares a Factor with n ===\n")

    # Set up an RSA keypair the "attacker" does NOT know p, q, or d for.
    p, q = 9973, 10007 # two primes (kept secret from the attacker in this scenario)
    n = p * q
    phi = (p - 1) * (q - 1)
    e = 65537
    d = mod_inverse(e, phi)

    print(f"(setup, unknown to attacker) p = {p}, q = {q}")
    print(f"Public key: e = {e}, n = {n}\n")

    # Suppose one of the intercepted plaintext blocks happens to be a multiple of q
    # (this is the "common factor with n" scenario described in the question).
    secret_plaintext_block = q * 7 # shares factor q with n, still < n
    ciphertext_block = pow(secret_plaintext_block, e, n)

    print(f"Intercepted ciphertext block: {ciphertext_block}")
    print("We are TOLD (but do not directly see) that its plaintext block")
    print(f"shares a common factor with n. (unknown to attacker: plaintext = {secret_plaintext_block})\n")

    print("--- The attack ---")
    print("Step 1: We don't know the plaintext block's value outright, but suppose")
    print("we can obtain/guess it (e.g. via a side channel, weak padding,")
    print("or it is simply revealed to us as in this problem). Call it m.")
    m = secret_plaintext_block
    print(f"m = {m}")

    g = math.gcd(m, n)
    print(f"\nStep 2: Compute gcd(m, n) using the Euclidean algorithm (FAST -- no")
    print(f"factoring of n required): gcd({m}, {n}) = {g}")

    if g != 1 and g != n:

```

```

recovered_p_or_q = g
other_factor = n // g
print(f"\nStep 3: Since  $n = p \cdot q$  for primes  $p, q$ , and  $1 < \gcd(m, n) < n$ , this  $\gcd$ ")
print(f"      MUST be one of the two prime factors of  $n$ .")
print(f"      Recovered factor: {recovered_p_or_q}")
print(f"      Other factor ( $n / \text{recovered}$ ): {other_factor}")

recovered_phi = (recovered_p_or_q - 1) * (other_factor - 1)
recovered_d = mod_inverse(e, recovered_phi)

print(f"\nStep 4:  $\phi(n) = (p-1)(q-1) = \{\text{recovered\_phi}\}$ ")
print(f"      Recovered private key  $d = \{\text{recovered\_d}\}$ ")
print(f"      Matches true private key: {recovered_d == d}")

# Now decrypt a DIFFERENT ciphertext block using the recovered key,
# to show the whole system -- not just this one block -- is broken.
other_message = 424242
other_cipher = pow(other_message, e, n)
decrypted_with_recovered_key = pow(other_cipher, recovered_d, n)
print(f"\nStep 5: Using the recovered private key on a DIFFERENT, unrelated")
print(f"      ciphertext block ({other_cipher}):")
print(f"      Decrypted result: {decrypted_with_recovered_key}")
print(f"      Correct: {decrypted_with_recovered_key == other_message}")

print("\n--- Conclusion ---")
print("Yes, this helps enormously. A plaintext block that shares a factor with  $n$ ")
print("lets us recover a prime factor of  $n$  via a single, cheap  $\gcd$  computation")
print("(polynomial time), instead of factoring  $n$  from scratch (believed to be)")
print("computationally hard for large  $n$ ). Once one prime factor is known,  $\phi(n)$ ")
print("and the full private key  $d$  follow immediately -- breaking RSA completely")
print("for this user, for ALL of their ciphertexts, not just the one block that")
print("happened to share the factor. This is exactly why textbook RSA requires")
print("padding schemes (e.g. OAEP) that make plaintext blocks effectively random")
print("and vanishingly unlikely to share a nontrivial factor with  $n$  by chance.")

```

Output:



Q25 - RSA Common Factor Attack

=== RSA: Exploiting a Plaintext Block that Shares a Factor with n ===

(setup, unknown to attacker) $p = 9973$, $q = 10007$

Public key: $e = 65537$, $n = 99799811$

Intercepted ciphertext block: 83698548

We are TOLD (but do not directly see) that its plaintext block shares a common factor with n . (unknown to attacker: plaintext = 70049)

--- The attack ---

Step 1: We don't know the plaintext block's value outright, but suppose we can obtain/guess it (e.g. via a side channel, weak padding, or it is simply revealed to us as in this problem). Call it m .
 $m = 70049$

Step 2: Compute $\gcd(m, n)$ using the Euclidean algorithm (FAST -- no factoring of n required): $\gcd(70049, 99799811) = 10007$

Step 3: Since $n = p \cdot q$ for primes p, q , and $1 < \gcd(m, n) < n$, this $\gcd$ MUST be one of the two prime factors of n .
Recovered factor: 10007
Other factor ($n / \text{recovered}$): 9973

Step 4: $\phi(n) = (p-1)(q-1) = 99779832$
Recovered private key $d = 6373169$
Matches true private key: True

Step 5: Using the recovered private key on a DIFFERENT, unrelated ciphertext block (66421838):
Decrypted result: 424242
Correct: True

--- Conclusion ---

Yes, this helps enormously. A plaintext block that shares a factor with n lets us recover a prime factor of n via a single, cheap $\gcd$ computation (polynomial time), instead of factoring n from scratch (believed to be computationally hard for large n). Once one prime factor is known, $\phi(n)$ and the full private key d follow immediately -- breaking RSA completely for this user, for ALL of their ciphertexts, not just the one block that happened to share the factor. This is exactly why textbook RSA requires padding schemes (e.g. OAEP) that make plaintext blocks effectively random and vanishingly unlikely to share a nontrivial factor with n by chance.

Question 26: RSA: Is Reusing n After a Private-Key Leak Safe?

Problem: Each user has a public key e and a private key d . Suppose Bob leaks his private key. Rather than generating a new modulus, he decides to generate a new public and a new private key. Is this safe?

Python Program:

```
# Q26. Bob's RSA private key  $d$  leaks. Instead of generating a fresh modulus  $n$ ,
# he keeps the same  $n$  and generates a new  $(e', d')$  pair. Is this safe?
#
# NO. There is a well-known probabilistic polynomial-time algorithm that,
# given ANY valid  $(e, d, n)$  triple (i.e. any matching public/private
# exponent pair for that modulus), factors  $n$  completely -- WITHOUT trial
# division. Once  $n = p \cdot q$  is known, an attacker can compute  $\phi(n)$  and hence
```

```

# derive the private key matching ANY public key that uses that same n,
# including Bob's brand new (e', d'). So simply rotating the exponent while
# keeping n is not safe -- the modulus itself must be regenerated.
#
# This program implements that factoring algorithm and demonstrates the
# attack end-to-end.

import random
random.seed(1234)

def mod_inverse(e, phi):
    def extended_gcd(a, b):
        if b == 0:
            return a, 1, 0
        g, x1, y1 = extended_gcd(b, a % b)
        return g, y1, x1 - (a // b) * y1
    g, x, _ = extended_gcd(e, phi)
    return x % phi if g == 1 else None

def factor_from_exponents(n, e, d):
    """Given a valid (e, d) pair for modulus n, factor n.
    Standard algorithm:  $k = e*d - 1$  is a multiple of  $\phi(n)$ , hence a multiple
    of the order of every element mod n. Write  $k = 2^s * t$  (t odd). For
    random bases a, the sequence  $a^t, a^{2t}, \dots, a^{2^{s-1}t}$  mod n will,
    with probability  $\geq 1/2$ , hit a nontrivial square root of 1 mod n -- and
    any such root x ( $x \neq \pm 1 \pmod n$ ,  $x^2 \equiv 1 \pmod n$ ) gives  $\gcd(x-1, n)$  as a
    nontrivial factor of n.
    """
    k = e * d - 1
    s = 0
    t = k
    while t % 2 == 0:
        t //= 2
        s += 1

    for _ in range(200): # attempts with random bases
        a = random.randrange(2, n - 1)
        x = pow(a, t, n)
        if x == 1 or x == n - 1:
            continue
        for _ in range(s - 1):
            y = pow(x, 2, n)
            if y == 1:
                # x is a nontrivial square root of 1 -> factor found
                p = gcd(x - 1, n)
                if p != 1 and p != n:
                    return p, n // p
            if y == n - 1:
                break
        x = y
    return None, None

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

if __name__ == "__main__":
    print("=== Is Rotating (e, d) While Reusing n Safe After a Private-Key Leak? ===\n")

    # Set up Bob's ORIGINAL keypair (unknown p, q to the attacker in principle,

```

```

# but the attacker HAS learned d because it leaked).
p, q = 10007, 10039
n = p * q
phi = (p - 1) * (q - 1)
e = 65537
d = mod_inverse(e, phi)

print(f"(Bob's original keypair, p and q secret) n = {n}")
print(f"Bob's original public key : e = {e}")
print(f"Bob's original private key: d = {d}    <-- THIS LEAKED\n")

print("Attacker uses the leaked (e, d, n) to factor n (no brute force needed):")
recovered_p, recovered_q = factor_from_exponents(n, e, d)
print(f"  Recovered factors: p = {recovered_p}, q = {recovered_q}")
print(f"  Correct factorization: {sorted((recovered_p, recovered_q)) == sorted((p, q))}\n")

# Bob "fixes" the problem by keeping n but generating a NEW keypair.
e2 = 17
d2 = mod_inverse(e2, phi)
print(f"Bob generates a NEW public/private keypair but reuses the same n:")
print(f"  New public key : e' = {e2}")
print(f"  New private key: d' = {d2}  (Bob believes this is now safe)\n")

print("But the attacker already knows n's factorization (p, q) from the leak,")
print("so phi(n) is trivial to recompute, and the attacker derives d' directly")
print("from the PUBLIC e' -- no new leak needed:")
recovered_phi = (recovered_p - 1) * (recovered_q - 1)
forged_d2 = mod_inverse(e2, recovered_phi)
print(f"  Attacker-derived d' = {forged_d2}")
print(f"  Matches Bob's real new private key: {forged_d2 == d2}\n")

print("--- Conclusion ---")
print("NOT SAFE. Leaking even one valid (e, d) pair for a modulus n lets an")
print("attacker factor n efficiently. Once n is factored, EVERY current and")
print("future keypair that reuses that same n is instantly broken -- rotating")
print("only the exponent accomplishes nothing. Bob must generate a brand-new")
print("modulus n (i.e. new primes p, q) to actually recover security.")

```

Output:

```

Q26 - RSA Key Leak: Same n, New Keys

=== Is Rotating (e, d) While Reusing n Safe After a Private-Key Leak? ===

(Bob's original keypair, p and q secret) n = 100460273
Bob's original public key : e = 65537
Bob's original private key: d = 29066777 <-- THIS LEAKED

Attacker uses the leaked (e, d, n) to factor n (no brute force needed):
  Recovered factors: p = 10007, q = 10039
  Correct factorization: True

Bob generates a NEW public/private keypair but reuses the same n:
  New public key : e' = 17
  New private key: d' = 41357741 (Bob believes this is now safe)

But the attacker already knows n's factorization (p, q) from the leak,
so phi(n) is trivial to recompute, and the attacker derives d' directly
from the PUBLIC e' -- no new leak needed:
  Attacker-derived d' = 41357741
  Matches Bob's real new private key: True

--- Conclusion ---
NOT SAFE. Leaking even one valid (e, d) pair for a modulus n lets an
attacker factor n efficiently. Once n is factored, EVERY current and
future keypair that reuses that same n is instantly broken -- rotating
only the exponent accomplishes nothing. Bob must generate a brand-new
modulus n (i.e. new primes p, q) to actually recover security.

```

Question 27: RSA: Encrypting Single Alphabetic Characters (A=0..Z=25)

Problem: Bob uses RSA with a very large modulus n for which the factorization cannot be found in a reasonable amount of time. Alice represents each alphabetic character as an integer between 0 and 25 ($A=0, \dots, Z=25$) and encrypts each number separately using RSA with large e and large n . Is this method secure? If not, describe the most efficient attack against this encryption method.

Python Program:

```

# Q27. Alice encrypts each alphabetic character (A=0 .. Z=25) SEPARATELY with
# RSA, using a very large e and n (large enough that factoring n is
# infeasible). Is this secure?
#
# NO -- regardless of how large e and n are. Plain (textbook) RSA is
# DETERMINISTIC: the same plaintext integer always encrypts to the same
# ciphertext under a given public key. Since there are only 26 possible
# plaintext values (0..25), an attacker who knows Bob's PUBLIC key can
# simply precompute  $c = m^e \bmod n$  for all 26 letters ONE TIME, building a
# lookup table, and then instantly reverse any intercepted ciphertext by
# table lookup -- no factoring of n, and no private key, ever required.
# This is a chosen/known-plaintext attack that is devastating specifically
# because the plaintext space is tiny, independent of key size.

import random

def is_probable_prime(n, k=20):
    if n < 2:
        return False
    for p in [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31]:
        if n % p == 0:
            return n == p

```



```

d, r = n - 1, 0
while d % 2 == 0:
    d //= 2
    r += 1
for _ in range(k):
    a = random.randrange(2, n - 1)
    x = pow(a, d, n)
    if x == 1 or x == n - 1:
        continue
    for _ in range(r - 1):
        x = pow(x, 2, n)
        if x == n - 1:
            break
    else:
        return False
return True

def random_prime(bits):
    while True:
        candidate = random.getrandbits(bits) | (1 << (bits - 1)) | 1
        if is_probable_prime(candidate):
            return candidate

def mod_inverse(e, phi):
    def extended_gcd(a, b):
        if b == 0:
            return a, 1, 0
        g, x1, y1 = extended_gcd(b, a % b)
        return g, y1, x1 - (a // b) * y1
    g, x, _ = extended_gcd(e, phi)
    return x % phi if g == 1 else None

def short(num, head=10, tail=6):
    s = str(num)
    if len(s) <= head + tail + 3:
        return s
    return f"{s[:head]}...{s[-tail:]} ({len(s)} digits)"

if __name__ == "__main__":
    random.seed(7)
    print("=== Is Per-Character RSA (A=0..Z=25) Secure, Even with Huge e, n? ===\n")

    # Generate a genuinely large RSA keypair (512-bit primes -> 1024-bit n).
    p = random_prime(512)
    q = random_prime(512)
    n = p * q
    phi = (p - 1) * (q - 1)
    e = 65537
    d = mod_inverse(e, phi)

    print(f"n has {n.bit_length()} bits (large, factoring is NOT feasible)")
    print(f"Public key: e = {e}\n")

    # Alice encrypts a secret word letter-by-letter.
    secret_word = "ATTACK"
    plain_values = [ord(c) - ord('A') for c in secret_word]
    ciphertexts = [pow(m, e, n) for m in plain_values]

    print(f"Alice's secret message: {secret_word!r}")
    print(f"Per-letter plaintext integers: {plain_values}")
    print(f"Intercepted ciphertexts (Eve does NOT have the private key):")

```

```

for c in ciphertexts:
    print(f" {short(c)}")

print("\n--- The attack (Eve, using ONLY the public key e, n) ---")
print("Step 1: Precompute the encryption of every possible letter, ONCE:")
lookup_table = {}
for letter_val in range(26):
    c = pow(letter_val, e, n)
    lookup_table[c] = letter_val
print(f" Built a table of all {len(lookup_table)} possible ciphertexts "
      f"(for m = 0..25) under the PUBLIC key.")

print("\nStep 2: For each intercepted ciphertext, look up the matching plaintext:")
recovered_values = [lookup_table[c] for c in ciphertexts]
recovered_word = ''.join(chr(v + ord('A')) for v in recovered_values)
print(f" Recovered letters: {recovered_values}")
print(f" Recovered word : {recovered_word!r}")
print(f" Matches original : {recovered_word == secret_word}")

print("\n--- Conclusion ---")
print("This method is NOT secure, no matter how large e and n are. The")
print("vulnerability has nothing to do with breaking RSA's math -- it comes")
print("entirely from encrypting a message drawn from a tiny plaintext space")
print("(26 possible values) with a DETERMINISTIC cipher. The fix is standard")
print("RSA padding (e.g. OAEP), which randomizes each encryption so the same")
print("plaintext produces a different ciphertext every time, and/or combining")
print("many characters into one large block instead of encrypting them one at a time.")

```

Output:

```
Q27 - RSA Per-Character Encoding Attack

=== Is Per-Character RSA (A=0..Z=25) Secure, Even with Huge e, n? ===

n has 1023 bits (large, factoring is NOT feasible)
Public key: e = 65537

Alice's secret message: 'ATTACK'
Per-letter plaintext integers: [0, 19, 19, 0, 2, 10]
Intercepted ciphertexts (Eve does NOT have the private key):
0
8078138317...301784 (308 digits)
8078138317...301784 (308 digits)
0
5427564213...239734 (308 digits)
7020798110...485417 (308 digits)

--- The attack (Eve, using ONLY the public key e, n) ---
Step 1: Precompute the encryption of every possible letter, ONCE:
    Built a table of all 26 possible ciphertexts (for m = 0..25) under the PUBLIC key.

Step 2: For each intercepted ciphertext, look up the matching plaintext:
    Recovered letters: [0, 19, 19, 0, 2, 10]
    Recovered word   : 'ATTACK'
    Matches original : True

--- Conclusion ---
This method is NOT secure, no matter how large e and n are. The
vulnerability has nothing to do with breaking RSA's math -- it comes
entirely from encrypting a message drawn from a tiny plaintext space
(26 possible values) with a DETERMINISTIC cipher. The fix is standard
RSA padding (e.g. OAEP), which randomizes each encryption so the same
plaintext produces a different ciphertext every time, and/or combining
many characters into one large block instead of encrypting them one at a time.
```

Question 28: Diffie-Hellman with Multiplication Instead of Exponentiation

Problem: In Diffie-Hellman, each participant selects a secret number x and sends the other participant $a^x \bmod q$ for some public number a . What would happen if the participants sent each other $x*a$ for some public number a instead? Give at least one method Alice and Bob could use to agree on a key. Can Eve break your system without finding the secret numbers? Can Eve find the secret numbers?

Python Program:

```
# Q28. Standard Diffie-Hellman: each side picks secret x, sends  $a^x \bmod q$ .
# What if, instead, each side sent  $(a * x) \bmod q$  (multiplication instead of
# modular EXPONENTIATION)?
#
# Modular exponentiation is a one-way function (computing  $a^x \bmod q$  is easy;
# recovering  $x$  from  $a^x \bmod q$  -- the discrete log problem -- is believed
# hard for well-chosen groups). Modular MULTIPLICATION by a known constant  $a$ 
# is NOT one-way: it is trivially invertible whenever  $\gcd(a, q) = 1$ , simply
# by multiplying by  $a$ 's modular inverse. This program demonstrates exactly
# how badly that breaks the protocol.
```

```
import random
random.seed(42)
```

```
def extended_gcd(a, b):
    if b == 0:
        return a, 1, 0
    g, x1, y1 = extended_gcd(b, a % b)
    return g, y1, x1 - (a // b) * y1
```

```

def mod_inverse(a, m):
    g, x, _ = extended_gcd(a, m)
    if g != 1:
        return None
    return x % m

if __name__ == "__main__":
    print("=== 'Diffie-Hellman' Using Multiplication Instead of Exponentiation ===\n")

    q = 104729 # a public prime modulus
    a = 17     # a public base, gcd(a, q) = 1 since q is prime and a < q

    print(f"Public parameters: q = {q}, a = {a}\n")

    # Alice and Bob each pick a secret number and broadcast a*x mod q.
    x_A = random.randrange(2, q - 1)
    x_B = random.randrange(2, q - 1)

    Y_A = (a * x_A) % q # what Alice broadcasts publicly
    Y_B = (a * x_B) % q # what Bob broadcasts publicly

    print(f"Alice's secret x_A = {x_A} -> broadcasts Y_A = (a * x_A) mod q = {Y_A}")
    print(f"Bob's secret x_B = {x_B} -> broadcasts Y_B = (a * x_B) mod q = {Y_B}\n")

    print("--- A method Alice and Bob could try, to agree on a shared key ---")
    print("By analogy with real DH (where the shared secret is  $a^{x_A x_B} = a^{(x_A x_B)}$ ),")
    print("here the natural analogue is to have each side apply their OWN secret to the")
    print("value they received, the same way:")
    K_from_Alice = (x_A * Y_B) % q # Alice computes x_A * Y_B mod q
    K_from_Bob = (x_B * Y_A) % q # Bob computes x_B * Y_A mod q
    print(f" Alice computes K = (x_A * Y_B) mod q = {K_from_Alice}")
    print(f" Bob computes K = (x_B * Y_A) mod q = {K_from_Bob}")
    print(f" Do they agree? {K_from_Alice == K_from_Bob} "
          f"(mod q, x_A*x_B*a mod q is the same either way -- multiplication is commutative)\n")

    print("--- Can Eve break the system WITHOUT finding the secret numbers x_A, x_B? ---")
    print("YES -- trivially. Eve, who only observes the public values a, q, Y_A, Y_B,")
    print("can compute the exact same shared key HERSELF, using nothing but public data,")
    print("because the 'combine' operation is just multiplication (commutative/associative)")
    print("and does not actually require knowing x_A or x_B individually:")
    a_inv = mod_inverse(a, q)
    eve_K = (Y_A * Y_B * a_inv) % q
    print(f" Eve computes: K_eve = (Y_A * Y_B * a^-1) mod q = {eve_K}")
    print(f" Matches the real shared key: {eve_K == K_from_Alice}\n")

    print("--- Can Eve find the secret numbers x_A, x_B themselves? ---")
    print("YES -- also trivially, because modular multiplication by the public constant")
    print("'a' is easily inverted (unlike modular exponentiation, which is a one-way")
    print("function under the discrete-log assumption). Eve simply multiplies by the")
    print("modular inverse of a:")
    recovered_xA = (Y_A * a_inv) % q
    recovered_xB = (Y_B * a_inv) % q
    print(f" a^-1 mod q = {a_inv}")
    print(f" Recovered x_A = (Y_A * a^-1) mod q = {recovered_xA} (real x_A = {x_A})")
    print(f" Recovered x_B = (Y_B * a^-1) mod q = {recovered_xB} (real x_B = {x_B})")
    print(f" Both recovered correctly: {recovered_xA == x_A and recovered_xB == x_B}\n")

    print("--- Conclusion ---")
    print("This 'multiplicative Diffie-Hellman' provides NO security whatsoever.")
    print("Everything (the shared key AND both parties' secret numbers) is recoverable")
    print("by any passive eavesdropper using only public information and basic modular")
    print("arithmetic. The security of real Diffie-Hellman depends entirely on modular")

```

```
print("EXPONENTIATION being a one-way function (hard discrete log); modular")
print("multiplication by a known constant has no such one-wayness at all.")
```

Output:

```

Q28 - DH with Multiplication Instead of Exponentiation

=== 'Diffie-Hellman' Using Multiplication Instead of Exponentiation ===

Public parameters: q = 104729, a = 17

Alice's secret x_A = 83812 -> broadcasts Y_A = (a * x_A) mod q = 63327
Bob's secret   x_B = 14594 -> broadcasts Y_B = (a * x_B) mod q = 38640

--- A method Alice and Bob could try, to agree on a shared key ---
By analogy with real DH (where the shared secret is  $(a^{x_A})^{x_B} = a^{(x_A \cdot x_B)}$ ),
here the natural analogue is to have each side apply their OWN secret to the
value they received, the same way:
  Alice computes K = (x_A * Y_B) mod q = 65542
  Bob computes   K = (x_B * Y_A) mod q = 65542
Do they agree? True (mod q,  $x_A \cdot x_B \cdot a$  mod q is the same either way -- multiplication is
commutative)

--- Can Eve break the system WITHOUT finding the secret numbers x_A, x_B? ---
YES -- trivially. Eve, who only observes the public values a, q, Y_A, Y_B,
can compute the exact same shared key HERSELF, using nothing but public data,
because the 'combine' operation is just multiplication (commutative/associative)
and does not actually require knowing x_A or x_B individually:
  Eve computes: K_eve = (Y_A * Y_B * a^-1) mod q = 65542
  Matches the real shared key: True

--- Can Eve find the secret numbers x_A, x_B themselves? ---
YES -- also trivially, because modular multiplication by the public constant
'a' is easily inverted (unlike modular exponentiation, which is a one-way
function under the discrete-log assumption). Eve simply multiplies by the
modular inverse of a:
  a^-1 mod q = 92408
  Recovered x_A = (Y_A * a^-1) mod q = 83812   (real x_A = 83812)
  Recovered x_B = (Y_B * a^-1) mod q = 14594   (real x_B = 14594)
  Both recovered correctly: True

--- Conclusion ---
This 'multiplicative Diffie-Hellman' provides NO security whatsoever.
Everything (the shared key AND both parties' secret numbers) is recoverable
by any passive eavesdropper using only public information and basic modular
arithmetic. The security of real Diffie-Hellman depends entirely on modular
EXPONENTIATION being a one-way function (hard discrete log); modular
multiplication by a known constant has no such one-wayness at all.
```

Question 29: SHA-3: Capacity Lane Diffusion

Problem: For a SHA-3 option with a block size of 1024 bits, assume each of the lanes in the first message block (P_0) has at least one nonzero bit. Initially, all lanes in the internal state matrix corresponding to the capacity portion are zero. Show how long it will take before all of these lanes have at least one nonzero bit. (Ignore the permutation's other mixing steps; keep track of the original zero lanes even after they change position.)

Python Program:

```

# Q29. SHA-3 (Keccak) sponge, block(rate) size r = 1024 bits.
# The full state is b = 1600 bits, arranged as a 5x5 grid of w = 64-bit
# "lanes" (25 lanes total). r/w = 1024/64 = 16 lanes belong to the RATE
# (message is XORed into these); the remaining b/w - r/w = 25 - 16 = 9
# lanes belong to the CAPACITY, and all start out as all-zero.
#
# Given: every lane of the first message block P0 has at least one nonzero
# bit, so immediately after absorbing P0, all 16 rate lanes are nonzero and
# all 9 capacity lanes are still exactly zero.
```

```

#
# As instructed, we ignore theta/chi/iota (the steps that actually mix bits
# between lanes) and track ONLY the lane-relocation caused by the pi step
# (rho only rotates bits *within* a lane, so it can never turn a zero lane
# into a nonzero one or vice versa -- it doesn't change which lanes are
# nonzero, only bit positions inside them). So a lane's "has >=1 nonzero
# bit" property is preserved as pi moves it to a new grid position each
# round. We ask: after how many applications of pi does every one of the
# (fixed) 9 capacity-lane positions end up having held a nonzero lane at
# least once?

W = 64          # bits per lane
B = 1600        # total state size (bits) for standard Keccak-f[1600]
R = 1024        # rate (block) size in bits, as specified by the question
LANES_TOTAL = B // W          # 25
RATE_LANES = R // W          # 16
CAPACITY_LANES = LANES_TOTAL - RATE_LANES # 9

def lane_index(x, y):
    return x + 5 * y # standard Keccak linear lane numbering

def pi_step(grid):
    """FIPS 202 pi step: new[x][y] = old[(x+3y) mod 5][x]."""
    new_grid = [[False] * 5 for _ in range(5)]
    for x in range(5):
        for y in range(5):
            new_grid[x][y] = grid[(x + 3 * y) % 5][x]
    return new_grid

if __name__ == "__main__":
    print("=== SHA-3 (r = 1024 bits): How Fast Do Capacity Lanes Become Nonzero? ===\n")
    print(f"Total lanes (b/w)      : {LANES_TOTAL}")
    print(f"Rate lanes (r/w)         : {RATE_LANES} (message-absorbing lanes)")
    print(f"Capacity lanes           : {CAPACITY_LANES} (start all-zero)\n")

    # Build initial 5x5 grid of "is this lane nonzero?" flags.
    # Rate lanes occupy linear indices 0..RATE_LANES-1 (fixed positions,
    # per the standard Keccak state serialization).
    grid = [[False] * 5 for _ in range(5)]
    for x in range(5):
        for y in range(5):
            if lane_index(x, y) < RATE_LANES:
                grid[x][y] = True # nonzero after absorbing P0

    capacity_positions = [(x, y) for x in range(5) for y in range(5)
                          if lane_index(x, y) >= RATE_LANES]

    print(f"Capacity lane positions (x,y): {capacity_positions}\n")

    touched = {pos: False for pos in capacity_positions}
    round_num = 0

    print("Round | nonzero-lane positions currently in the CAPACITY area")
    while not all(touched.values()):
        round_num += 1
        grid = pi_step(grid)
        currently_nonzero_capacity = [pos for pos in capacity_positions if grid[pos[0]]
        [pos[1]]]
        for pos in currently_nonzero_capacity:
            touched[pos] = True
        print(f"{{round_num:5d}} | {{currently_nonzero_capacity}} "
              f"({touched so far: {sum(touched.values())}}/{{CAPACITY_LANES}})")

```

```

    if round_num > 50:
        print("(stopping -- something unexpected, exceeded 50 rounds)")
        break

print(f"\nAll {CAPACITY_LANES} capacity lanes have held a nonzero value at least once")
print(f"after {round_num} applications of the pi (lane-relocation) step.")

print("\nNote: this counts pure lane-permutation (pi) steps, per the question's")
print("instruction to ignore the other mixing steps (theta/chi/iota) that would")
print("normally accelerate diffusion considerably. In real Keccak-f[1600], pi is")
print("applied once per internal round (24 rounds total per permutation call),")
print(f"so this corresponds to needing only {round_num} of those 24 rounds' pi steps")
print("for full capacity coverage under this simplified (permutation-only) model.")

```

Output:

```

Q29 - SHA-3 Capacity Lane Diffusion

=== SHA-3 (r = 1024 bits): How Fast Do Capacity Lanes Become Nonzero? ===

Total lanes (b/w)      : 25
Rate lanes (r/w)       : 16  (message-absorbing lanes)
Capacity lanes         : 9   (start all-zero)

Capacity lane positions (x,y): [(0, 4), (1, 3), (1, 4), (2, 3), (2, 4), (3, 3), (3, 4), (4, 3),
(4, 4)]

Round | nonzero-lane positions currently in the CAPACITY area
  1 | [(0, 4), (1, 3), (1, 4), (2, 3), (2, 4), (3, 4)]   (touched so far: 6/9)
  2 | [(0, 4), (1, 3), (2, 3), (3, 3), (3, 4), (4, 3), (4, 4)] (touched so far: 9/9)

All 9 capacity lanes have held a nonzero value at least once
after 2 applications of the pi (lane-relocation) step.

Note: this counts pure lane-permutation (pi) steps, per the question's
instruction to ignore the other mixing steps (theta/chi/iota) that would
normally accelerate diffusion considerably. In real Keccak-f[1600], pi is
applied once per internal round (24 rounds total per permutation call),
so this corresponds to needing only 2 of those 24 rounds' pi steps
for full capacity coverage under this simplified (permutation-only) model.

```

Question 30: CBC-MAC Forgery on a One-Block Message

Problem: Given the CBC MAC of a one-block message X , say $T = \text{MAC}(K, X)$, show that the adversary immediately knows the CBC MAC for the two-block message $X || (X \text{ XOR } T)$, since this is once again T .

Python Program:

```

# Q30. Given the CBC-MAC of a ONE-block message X, i.e. T = MAC(K, X), an
# adversary can immediately compute the CBC-MAC of the TWO-block message
# X || (X XOR T) -- without knowing the key K -- and it equals the SAME
# tag T. This is the classic demonstration of why raw/basic CBC-MAC is
# insecure for variable-length messages (no length-binding or key
# separation between the last-block step and earlier steps).
#
# Basic (IV=0) CBC-MAC of a message M1, M2, ..., Mn:
#   C0 = IV = 0
#   C_i = E(K, M_i XOR C_{i-1})
#   MAC = C_n (just the final block)
#
# For a one-block message X: T = E(K, X XOR 0) = E(K, X)
#
# For the two-block message M1 = X, M2 = X XOR T:
#   C1 = E(K, X XOR 0) = E(K, X) = T (identical first step)
#   C2 = E(K, M2 XOR C1) = E(K, (X XOR T) XOR T) = E(K, X) = T
#   MAC = C2 = T -- SAME tag as the one-block message, with NO knowledge of K!

from Crypto.Cipher import AES
import os

```

```

def cbc_mac(key, blocks, block_size=16):
    prev = bytes(block_size) # IV = 0
    for block in blocks:
        cipher = AES.new(key, AES.MODE_ECB)
        prev = cipher.encrypt(bytes(a ^ b for a, b in zip(block, prev)))
    return prev # tag = final ciphertext block

def xor_bytes(a, b):
    return bytes(x ^ y for x, y in zip(a, b))

if __name__ == "__main__":
    print("=== CBC-MAC Forgery: Extending a 1-Block Message for Free ===\n")

    key = os.urandom(16)
    X = os.urandom(16) # a one-block message (16 bytes for AES)

    print(f"Key (attacker does NOT know this): {key.hex()}")
    print(f"One-block message X: {X.hex()}\n")

    T = cbc_mac(key, [X])
    print(f"T = MAC(K, X) = {T.hex()} (attacker DOES see this tag)\n")

    # Attacker, WITHOUT the key, constructs the forged two-block message.
    forged_second_block = xor_bytes(X, T)
    forged_message_blocks = [X, forged_second_block]
    print("Attacker builds a two-block message: X || (X XOR T)")
    print(f"Block 1 = X = {X.hex()}")
    print(f"Block 2 = X XOR T = {forged_second_block.hex()}\n")

    print("Attacker PREDICTS (without the key) that MAC(K, X || (X XOR T)) == T.")
    print("Let's check by actually computing the true CBC-MAC (using the real key,")
    print("which only the verifier/legitimate parties have) to confirm the forgery:")

    actual_mac_of_forged_message = cbc_mac(key, forged_message_blocks)
    print(f"\nActual MAC(K, X || (X XOR T)) = {actual_mac_of_forged_message.hex()}")
    print(f"Original T = {T.hex()}")
    print(f"Forgery successful (tags match): {actual_mac_of_forged_message == T}")

    print("\n--- Why this matters ---")
    print("An attacker who observes just ONE (message, tag) pair -- for ANY one-block")
    print("message X -- can immediately produce a SECOND, different, valid message")
    print("(X || X XOR T) with a KNOWN-VALID tag, without ever learning the secret key.")
    print("This existential forgery is why raw CBC-MAC must never be used as-is on")
    print("variable-length messages; real-world constructions fix this by prepending")
    print("the message length, using a derived/whitened final-block key (as in CMAC),")
    print("or using an unpredictable, non-attacker-controlled IV/key separation.")

```

Output:


```
Q30 - CBC-MAC Forgery

=== CBC-MAC Forgery: Extending a 1-Block Message for Free ===

Key (attacker does NOT know this): c8f0973bdf2c681c0bd9e1b649b0040d
One-block message X: 8a2489e3449a09cf8248a01ed634cddc

T = MAC(K, X) = eb8ddbc8357678c6484ff66e5a19f5ac    (attacker DOES see this tag)

Attacker builds a two-block message: X || (X XOR T)
Block 1 = X                = 8a2489e3449a09cf8248a01ed634cddc
Block 2 = X XOR T          = 61a9522b71ec7109ca0756708c2d3870

Attacker PREDICTS (without the key) that MAC(K, X || (X XOR T)) == T.
Let's check by actually computing the true CBC-MAC (using the real key,
which only the verifier/legitimate parties have) to confirm the forgery:

Actual MAC(K, X || (X XOR T)) = eb8ddbc8357678c6484ff66e5a19f5ac
Original T                    = eb8ddbc8357678c6484ff66e5a19f5ac
Forgery successful (tags match): True

--- Why this matters ---
An attacker who observes just ONE (message, tag) pair -- for ANY one-block
message X -- can immediately produce a SECOND, different, valid message
(X || X XOR T) with a KNOWN-VALID tag, without ever learning the secret key.
This existential forgery is why raw CBC-MAC must never be used as-is on
variable-length messages; real-world constructions fix this by prepending
the message length, using a derived/whitened final-block key (as in CMAC),
or using an unpredictable, non-attacker-controlled IV/key separation.
```

Question 31: CMAC Subkey Generation

Problem: In CMAC subkey generation, the block cipher is applied to the block consisting entirely of 0 bits. The first subkey is derived from the resulting string by a left shift of one bit and, conditionally, by XORing a constant that depends on the block size. The second subkey is derived the same way from the first. a. What constants are needed for block sizes of 64 and 128 bits? b. How do the left shift and XOR accomplish the desired result?

Python Program:

```
# Q31. CMAC subkey generation.
# 1. Apply the block cipher to the all-zero block: L = E(K, 0^b).
# 2. K1 = L << 1, and if the MSB of L was 1, XOR the result with a fixed
#    constant Rb that depends on the block size b.
# 3. K2 = K1 << 1, XORed with Rb if the MSB of K1 was 1.
#
# a. What are the constants Rb for block sizes 64 and 128?
# b. Demonstrate/explain how the left-shift + conditional-XOR construction
#    achieves the desired effect (multiplication by the polynomial "x" in
#    GF(2^b), matching the field used by the block cipher's algebra).

from Crypto.Cipher import AES
import os

# Standard NIST SP 800-38B constants Rb, expressed as the low byte(s) of an
# otherwise-zero b-bit block (i.e. Rb is a specific low-degree polynomial
# in GF(2^b)'s standard representation).
RB_64 = 0x1B    # for 64-bit block ciphers (e.g. DES, 3DES, S-DES conceptually)
RB_128 = 0x87   # for 128-bit block ciphers (e.g. AES)

def left_shift_1(data: bytes) -> bytes:
    """Shift an entire byte string left by 1 bit, returning (shifted_bytes,
    msb_that_fell_off)."""
    n = len(data)
```

```

val = int.from_bytes(data, "big")
msb = (val >> (8 * n - 1)) & 1
shifted = (val << 1) & ((1 << (8 * n)) - 1)
return shifted.to_bytes(n, "big"), msb

def generate_subkeys(key: bytes, block_size_bytes: int, rb_constant: int):
    zero_block = bytes(block_size_bytes)
    cipher = AES.new(key, AES.MODE_ECB) if block_size_bytes == 16 else None
    if cipher is None:
        raise ValueError("This demo cipher only supports 128-bit (AES) blocks directly.")
    L = cipher.encrypt(zero_block)

    K1_shifted, msb1 = left_shift_1(L)
    if msb1:
        rb_bytes = rb_constant.to_bytes(block_size_bytes, "big")
        K1 = bytes(a ^ b for a, b in zip(K1_shifted, rb_bytes))
    else:
        K1 = K1_shifted

    K2_shifted, msb2 = left_shift_1(K1)
    if msb2:
        rb_bytes = rb_constant.to_bytes(block_size_bytes, "big")
        K2 = bytes(a ^ b for a, b in zip(K2_shifted, rb_bytes))
    else:
        K2 = K2_shifted

    return L, K1, K2, msb1, msb2

if __name__ == "__main__":
    print("=== CMAC Subkey Generation ===\n")

    print("--- a. Constants Rb needed for common block sizes ---")
    print(f"Block size 64 bits (e.g. DES/3DES): Rb = 0x{RB_64:02X}")
    print(f"    (block = 0...0 01 1011, i.e.  $x^{64} = x^4 + x^3 + x + 1$  reduction poly)")
    print(f"Block size 128 bits (e.g. AES) : Rb = 0x{RB_128:02X}")
    print(f"    (block = 0...0 1000 0111, i.e.  $x^{128} = x^7 + x^2 + x + 1$  reduction poly)\n")

    key = os.urandom(16)
    L, K1, K2, msb1, msb2 = generate_subkeys(key, 16, RB_128)

    print("--- Worked example with a real 128-bit block cipher (AES) ---")
    print(f"AES key : {key.hex()}")
    print(f"L = E(K, 0^128) : {L.hex()}")
    print(f"MSB of L is 1? : {bool(msb1)} -> "
          f"'XOR with Rb applied' if msb1 else 'no XOR needed'")
    print(f"K1 (first subkey) : {K1.hex()}")
    print(f"MSB of K1 is 1? : {bool(msb2)} -> "
          f"'XOR with Rb applied' if msb2 else 'no XOR needed'")
    print(f"K2 (second subkey): {K2.hex()}\n")

    print("--- b. Why left-shift + conditional-XOR works ---")
    print("A b-bit block is treated as an element of the finite field  $GF(2^b)$ ,")
    print("represented as a binary polynomial of degree  $< b$ . Left-shifting the")
    print("block by 1 bit is exactly multiplication by the field element 'x'")
    print("(i.e. by 2, in the integer analogy). If the top (most significant) bit")
    print("was 0, the polynomial's degree was already  $< b$  after doubling, so the")
    print("shifted value IS the correct product mod the field's irreducible")
    print("reduction polynomial -- no further correction needed.")
    print("If the top bit was 1, doubling would have produced a term of degree")
    print("exactly b (the polynomial 'overflowed' out of the field), so we must")
    print("reduce it modulo the field's fixed irreducible polynomial of degree b")
    print("-- and XOR-ing with Rb (which directly encodes that reduction)")
    print("polynomial's lower-degree terms) performs exactly that modular")

```

```

print("reduction in one step. This is the identical 'multiply by x in")
print("GF(2^n), conditionally XOR the reduction constant' trick used in")
print("AES's MixColumns (xtime) and in GCM/GHASH -- CMAC reuses it to turn")
print("a single master key L into two distinct, algebraically related")
print("subkeys K1 = L*x and K2 = L*x^2 in the block cipher's field.")

```

Output:

```

Q31 - CMAC Subkey Generation

=== CMAC Subkey Generation ===

--- a. Constants Rb needed for common block sizes ---
Block size 64 bits (e.g. DES/3DES): Rb = 0x1B (block = 0...0 01 1011, i.e. x^64 = x^4+x^3+x+1
reduction poly)
Block size 128 bits (e.g. AES) : Rb = 0x87 (block = 0...0 1000 0111, i.e. x^128 = x^7+x^2+x+1
reduction poly)

--- Worked example with a real 128-bit block cipher (AES) ---
AES key       : aea2aad0d531a447acc4a0c17f62cbc7
L = E(K, 0^128) : e712443329a2ee2e820009f92f4ee669
MSB of L is 1? : True  -> XOR with Rb applied
K1 (first subkey) : ce2488665345dc5d040013f25e9dcc55
MSB of K1 is 1? : True  -> XOR with Rb applied
K2 (second subkey): 9c4910cca68bb8ba080027e4bd3b982d

--- b. Why left-shift + conditional-XOR works ---
A b-bit block is treated as an element of the finite field GF(2^b),
represented as a binary polynomial of degree < b. Left-shifting the
block by 1 bit is exactly multiplication by the field element 'x'
(i.e. by 2, in the integer analogy). If the top (most significant) bit
was 0, the polynomial's degree was already < b after doubling, so the
shifted value IS the correct product mod the field's irreducible
reduction polynomial -- no further correction needed.
If the top bit was 1, doubling would have produced a term of degree
exactly b (the polynomial 'overflowed' out of the field), so we must
reduce it modulo the field's fixed irreducible polynomial of degree b
-- and XOR-ing with Rb (which directly encodes that reduction
polynomial's lower-degree terms) performs exactly that modular
reduction in one step. This is the identical 'multiply by x in
GF(2^n), conditionally XOR the reduction constant' trick used in
AES's MixColumns (xtime) and in GCM/GHASH -- CMAC reuses it to turn
a single master key L into two distinct, algebraically related
subkeys K1 = L*x and K2 = L*x^2 in the block cipher's field.

```

Question 32: DSA vs. RSA: Implication of Randomized Signatures

Problem: Because the value of k is generated fresh for each DSA signature, even signing the same message twice on different occasions produces different signatures. This is not true of RSA signatures. What is the implication of this difference?

Python Program:

```

# Q32. In DSA, a fresh random  $k$  is generated for EVERY signature, so signing
# the same message twice produces two DIFFERENT valid signatures. In RSA
# signing (textbook RSA signature =  $m^d \bmod n$ ), there is no randomization,
# so signing the same message always gives the IDENTICAL signature. What
# does this difference imply?

```

```
import random
```

```

def extended_gcd(a, b):
    if b == 0:
        return a, 1, 0
    g, x1, y1 = extended_gcd(b, a % b)
    return g, y1, x1 - (a // b) * y1

```

```

def mod_inverse(a, m):
    g, x, _ = extended_gcd(a, m)

```

```
return x % m if g == 1 else None
```

```
def is_prime(n, k=20):
    if n < 2:
        return False
    for p in [2, 3, 5, 7, 11, 13, 17, 19, 23]:
        if n % p == 0:
            return n == p
    d, r = n - 1, 0
    while d % 2 == 0:
        d //= 2
        r += 1
    for _ in range(k):
        a = random.randrange(2, n - 1)
        x = pow(a, d, n)
        if x in (1, n - 1):
            continue
        for _ in range(r - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                break
        else:
            return False
    return True

# --- tiny illustrative DSA-style scheme (small parameters for a lab demo;
#       real DSA uses far larger q, p) ---
def dsa_demo():
    q = 283          # small prime, illustrative only
    while True:
        k_mult = random.randint(2, 10)
        p = k_mult * q + 1
        if is_prime(p):
            break
    # find a generator g of the order-q subgroup
    for h in range(2, p - 1):
        g = pow(h, (p - 1) // q, p)
        if g != 1:
            break

    x = random.randrange(2, q - 1)          # private key
    y = pow(g, x, p)                        # public key

    def sign(msg_hash):
        while True:
            k = random.randrange(2, q - 1)
            r = pow(g, k, p) % q
            if r == 0:
                continue
            k_inv = mod_inverse(k, q)
            if k_inv is None:
                continue
            s = (k_inv * (msg_hash + x * r)) % q
            if s == 0:
                continue
            return r, s

    def verify(msg_hash, r, s):
        if not (0 < r < q and 0 < s < q):
            return False
        w = mod_inverse(s, q)
        u1 = (msg_hash * w) % q
        u2 = (r * w) % q
```

```

        v = ((pow(g, u1, p) * pow(y, u2, p)) % p) % q
        return v == r

    return p, q, g, x, y, sign, verify

if __name__ == "__main__":
    random.seed(3)
    print("=== DSA: Randomized Signatures vs. RSA's Deterministic Signatures ===\n")

    p, q, g, x, y, sign, verify = dsa_demo()
    print(f"(tiny illustrative DSA parameters) p={p}, q={q}, g={g}")
    print(f"Private key x = {x}, Public key y = {y}\n")

    message_hash = 12345 # stand-in for H(message)

    sig1 = sign(message_hash)
    sig2 = sign(message_hash)
    print(f"Signing the SAME message hash ({message_hash}) twice with DSA:")
    print(f"  Signature #1: (r, s) = {sig1}")
    print(f"  Signature #2: (r, s) = {sig2}")
    print(f"  Signatures identical? {sig1 == sig2}")
    print(f"  Both verify correctly? {verify(message_hash, *sig1)} and {verify(message_hash,
*sig2)}\n")

    print("--- Compare: textbook RSA signing (deterministic)---")
    p_r, q_r = 10007, 10039
    n_r = p_r * q_r
    phi_r = (p_r - 1) * (q_r - 1)
    e_r = 65537
    d_r = mod_inverse(e_r, phi_r)
    rsa_sig1 = pow(message_hash, d_r, n_r)
    rsa_sig2 = pow(message_hash, d_r, n_r)
    print(f"  RSA signature #1: {rsa_sig1}")
    print(f"  RSA signature #2: {rsa_sig2}")
    print(f"  Signatures identical? {rsa_sig1 == rsa_sig2}\n")

    print("--- Implications of DSA's randomized signatures ---")
    print("1. Non-repudiation / linkability: with RSA, seeing the exact same")
    print("  signature bytes twice proves the exact same signing event/message")
    print("  was reused verbatim -- e.g., useful for detecting replay or for")
    print("  deterministic protocols that rely on matching signatures. DSA gives")
    print("  NO such signal: two different-looking signatures over the same")
    print("  message are both completely normal and expected, not suspicious.")
    print("2. No signature-based deduplication: systems cannot use 'have we seen")
    print("  this exact signature before?' as a cheap way to detect a repeated")
    print("  message under DSA the way they sometimes can under raw RSA.")
    print("3. Randomness is a hard SECURITY REQUIREMENT for DSA: k must be fresh,"")
    print("  secret, and unpredictable for every signature. If k is EVER reused")
    print("  across two different messages, or is even partially predictable,"")
    print("  the private key x can be recovered algebraically from the two")
    print("  signatures. RSA has no equivalent per-signature secret to leak.")
    print("4. Because of point 3, poor randomness (weak RNG, predictable PRNG")
    print("  seeds, virtual-machine snapshot reuse, etc.) is a much more direct")
    print("  and historically real-world path to full private-key compromise")
    print("  for DSA/ECDSA than for RSA signing.")

```

Output:

```
Q32 - DSA vs RSA Signature Randomization

=== DSA: Randomized Signatures vs. RSA's Deterministic Signatures ===

(tiny illustrative DSA parameters) p=1699, q=283, g=64
Private key x = 34, Public key y = 575

Signing the SAME message hash (12345) twice with DSA:
  Signature #1: (r, s) = (91, 111)
  Signature #2: (r, s) = (106, 29)
  Signatures identical? False
  Both verify correctly? True and True

--- Compare: textbook RSA signing (deterministic) ---
  RSA signature #1: 75004675
  RSA signature #2: 75004675
  Signatures identical? True

--- Implications of DSA's randomized signatures ---
1. Non-repudiation / linkability: with RSA, seeing the exact same
   signature bytes twice proves the exact same signing event/message
   was reused verbatim -- e.g., useful for detecting replay or for
   deterministic protocols that rely on matching signatures. DSA gives
   NO such signal: two different-looking signatures over the same
   message are both completely normal and expected, not suspicious.
2. No signature-based deduplication: systems cannot use 'have we seen
   this exact signature before?' as a cheap way to detect a repeated
   message under DSA the way they sometimes can under raw RSA.
3. Randomness is a hard SECURITY REQUIREMENT for DSA: k must be fresh,
   secret, and unpredictable for every signature. If k is EVER reused
   across two different messages, or is even partially predictable,
   the private key x can be recovered algebraically from the two
   signatures. RSA has no equivalent per-signature secret to leak.
4. Because of point 3, poor randomness (weak RNG, predictable PRNG
   seeds, virtual-machine snapshot reuse, etc.) is a much more direct
   and historically real-world path to full private-key compromise
   for DSA/ECDSA than for RSA signing.
```

Question 33: Full DES Implementation (64-bit Block, 56-bit Key)

Problem: DES is a block cipher that encrypts data in blocks of 64 bits, producing 64 bits of ciphertext from 64 bits of plaintext. The same algorithm and key are used for encryption and decryption (with minor differences), and the key length is 56 bits. Implement DES in full.

Python Program:

```
# Q33. Full DES (Data Encryption Standard) implementation.
# 64-bit plaintext block in -> 64-bit ciphertext block out, 56-bit effective
# key (64 bits supplied, 8 are parity bits dropped by PC-1), 16 Feistel
# rounds, same algorithm/key used for encryption and decryption (subkeys
# just applied in reverse order for decryption -- see Q17).
#
# Verified against the official worked example from FIPS PUB 46 / NIST:
#   key       = 133457799BBCDFF1
#   plaintext = 0123456789ABCDEF
#   ciphertext= 85E813540F0AB405
```

IP = [58, 50, 42, 34, 26, 18, 10, 2, 60, 52, 44, 36, 28, 20, 12, 4,

62, 54, 46, 38, 30, 22, 14, 6, 64, 56, 48, 40, 32, 24, 16, 8,
57, 49, 41, 33, 25, 17, 9, 1, 59, 51, 43, 35, 27, 19, 11, 3,
61, 53, 45, 37, 29, 21, 13, 5, 63, 55, 47, 39, 31, 23, 15, 7]

FP = [40, 8, 48, 16, 56, 24, 64, 32, 39, 7, 47, 15, 55, 23, 63, 31,
38, 6, 46, 14, 54, 22, 62, 30, 37, 5, 45, 13, 53, 21, 61, 29,
36, 4, 44, 12, 52, 20, 60, 28, 35, 3, 43, 11, 51, 19, 59, 27,
34, 2, 42, 10, 50, 18, 58, 26, 33, 1, 41, 9, 49, 17, 57, 25]

E = [32, 1, 2, 3, 4, 5, 4, 5, 6, 7, 8, 9, 8, 9, 10, 11, 12, 13,
12, 13, 14, 15, 16, 17, 16, 17, 18, 19, 20, 21, 20, 21, 22, 23, 24, 25,
24, 25, 26, 27, 28, 29, 28, 29, 30, 31, 32, 1]

P = [16, 7, 20, 21, 29, 12, 28, 17, 1, 15, 23, 26, 5, 18, 31, 10,
2, 8, 24, 14, 32, 27, 3, 9, 19, 13, 30, 6, 22, 11, 4, 25]

PC1 = [57, 49, 41, 33, 25, 17, 9, 1, 58, 50, 42, 34, 26, 18,
10, 2, 59, 51, 43, 35, 27, 19, 11, 3, 60, 52, 44, 36,
63, 55, 47, 39, 31, 23, 15, 7, 62, 54, 46, 38, 30, 22,
14, 6, 61, 53, 45, 37, 29, 21, 13, 5, 28, 20, 12, 4]

PC2 = [14, 17, 11, 24, 1, 5, 3, 28, 15, 6, 21, 10,
23, 19, 12, 4, 26, 8, 16, 7, 27, 20, 13, 2,
41, 52, 31, 37, 47, 55, 30, 40, 51, 45, 33, 48,
44, 49, 39, 56, 34, 53, 46, 42, 50, 36, 29, 32]

SHIFTS = [1, 1, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 1]

SBOX = [
S1
[[14, 4, 13, 1, 2, 15, 11, 8, 3, 10, 6, 12, 5, 9, 0, 7],
[0, 15, 7, 4, 14, 2, 13, 1, 10, 6, 12, 11, 9, 5, 3, 8],
[4, 1, 14, 8, 13, 6, 2, 11, 15, 12, 9, 7, 3, 10, 5, 0],
[15, 12, 8, 2, 4, 9, 1, 7, 5, 11, 3, 14, 10, 0, 6, 13]],
S2
[[15, 1, 8, 14, 6, 11, 3, 4, 9, 7, 2, 13, 12, 0, 5, 10],
[3, 13, 4, 7, 15, 2, 8, 14, 12, 0, 1, 10, 6, 9, 11, 5],
[0, 14, 7, 11, 10, 4, 13, 1, 5, 8, 12, 6, 9, 3, 2, 15],
[13, 8, 10, 1, 3, 15, 4, 2, 11, 6, 7, 12, 0, 5, 14, 9]],
S3
[[10, 0, 9, 14, 6, 3, 15, 5, 1, 13, 12, 7, 11, 4, 2, 8],
[13, 7, 0, 9, 3, 4, 6, 10, 2, 8, 5, 14, 12, 11, 15, 1],
[13, 6, 4, 9, 8, 15, 3, 0, 11, 1, 2, 12, 5, 10, 14, 7],
[1, 10, 13, 0, 6, 9, 8, 7, 4, 15, 14, 3, 11, 5, 2, 12]],
S4
[[7, 13, 14, 3, 0, 6, 9, 10, 1, 2, 8, 5, 11, 12, 4, 15],
[13, 8, 11, 5, 6, 15, 0, 3, 4, 7, 2, 12, 1, 10, 14, 9],
[10, 6, 9, 0, 12, 11, 7, 13, 15, 1, 3, 14, 5, 2, 8, 4],
[3, 15, 0, 6, 10, 1, 13, 8, 9, 4, 5, 11, 12, 7, 2, 14]],
S5
[[2, 12, 4, 1, 7, 10, 11, 6, 8, 5, 3, 15, 13, 0, 14, 9],
[14, 11, 2, 12, 4, 7, 13, 1, 5, 0, 15, 10, 3, 9, 8, 6],
[4, 2, 1, 11, 10, 13, 7, 8, 15, 9, 12, 5, 6, 3, 0, 14],
[11, 8, 12, 7, 1, 14, 2, 13, 6, 15, 0, 9, 10, 4, 5, 3]],
S6
[[12, 1, 10, 15, 9, 2, 6, 8, 0, 13, 3, 4, 14, 7, 5, 11],
[10, 15, 4, 2, 7, 12, 9, 5, 6, 1, 13, 14, 0, 11, 3, 8],
[9, 14, 15, 5, 2, 8, 12, 3, 7, 0, 4, 10, 1, 13, 11, 6],
[4, 3, 2, 12, 9, 5, 15, 10, 11, 14, 1, 7, 6, 0, 8, 13]],
S7
[[4, 11, 2, 14, 15, 0, 8, 13, 3, 12, 9, 7, 5, 10, 6, 1],
[13, 0, 11, 7, 4, 9, 1, 10, 14, 3, 5, 12, 2, 15, 8, 6],
[1, 4, 11, 13, 12, 3, 7, 14, 10, 15, 6, 8, 0, 5, 9, 2],
[6, 11, 13, 8, 1, 4, 10, 7, 9, 5, 0, 15, 14, 2, 3, 12]],
S8

```

[[13, 2, 8, 4, 6, 15, 11, 1, 10, 9, 3, 14, 5, 0, 12, 7],
 [1, 15, 13, 8, 10, 3, 7, 4, 12, 5, 6, 11, 0, 14, 9, 2],
 [7, 11, 4, 1, 9, 12, 14, 2, 0, 6, 10, 13, 15, 3, 5, 8],
 [2, 1, 14, 7, 4, 10, 8, 13, 15, 12, 9, 0, 3, 5, 6, 11]],
]

```

```

def permute(bits, table):
    return [bits[i - 1] for i in table]

```

```

def left_rotate(bits, n):
    return bits[n:] + bits[:n]

```

```

def xor(a, b):
    return [x ^ y for x, y in zip(a, b)]

```

```

def bytes_to_bits(data8):
    bits = []
    for byte in data8:
        bits.extend(int(b) for b in f"{byte:08b}")
    return bits

```

```

def bits_to_hex(bits):
    val = 0
    for b in bits:
        val = (val << 1) | b
    return f"{val:0{len(bits)//4}X}"

```

```

def generate_round_keys(key64bits):
    key56 = permute(key64bits, PC1)
    C, D = key56[:28], key56[28:]
    round_keys = []
    for shift in SHIFTS:
        C = left_rotate(C, shift)
        D = left_rotate(D, shift)
        round_keys.append(permute(C + D, PC2))
    return round_keys

```

```

def sbx_substitute(bits48):
    output = []
    for i in range(8):
        block6 = bits48[i * 6:i * 6 + 6]
        row = (block6[0] << 1) | block6[5]
        col = (block6[1] << 3) | (block6[2] << 2) | (block6[3] << 1) | block6[4]
        val = SB0X[i][row][col]
        output.extend(int(b) for b in f"{val:04b}")
    return output

```

```

def feistel(R, subkey):
    expanded = permute(R, E)
    xored = xor(expanded, subkey)
    sboxed = sbx_substitute(xored)
    return permute(sboxed, P)

```

```

def des_process(block64bits, round_keys):
    """round_keys given in the order they should be USED (K1..K16 for
    encryption; K16..K1 for decryption)."""

```



```

bits = permute(block64bits, IP)
L, R = bits[:32], bits[32:]
for rk in round_keys:
    new_L = R
    new_R = xor(L, feistel(R, rk))
    L, R = new_L, new_R
pre_output = R + L # note the final swap
return permute(pre_output, FP)

def des_encrypt_block(plaintext64bits, key64bits):
    round_keys = generate_round_keys(key64bits)
    return des_process(plaintext64bits, round_keys)

def des_decrypt_block(ciphertext64bits, key64bits):
    round_keys = generate_round_keys(key64bits)
    return des_process(ciphertext64bits, list(reversed(round_keys)))

def hex_to_bits(hex_str):
    data = bytes.fromhex(hex_str)
    return bytes_to_bits(data)

if __name__ == "__main__":
    print("=== Full DES Implementation (64-bit block, 56-bit effective key) ===\n")

    # Official FIPS PUB 46 worked example
    key_hex = "133457799BBCDFF1"
    plain_hex = "0123456789ABCDEF"
    expected_cipher_hex = "85E813540F0AB405"

    key_bits = hex_to_bits(key_hex)
    plain_bits = hex_to_bits(plain_hex)

    cipher_bits = des_encrypt_block(plain_bits, key_bits)
    cipher_hex = bits_to_hex(cipher_bits)

    print(f"Key          : {key_hex}")
    print(f"Plaintext    : {plain_hex}")
    print(f"Ciphertext   : {cipher_hex}")
    print(f"Expected     : {expected_cipher_hex} (official FIPS PUB 46 test vector)")
    print(f"Matches official test vector: {cipher_hex == expected_cipher_hex}\n")

    decrypted_bits = des_decrypt_block(cipher_bits, key_bits)
    decrypted_hex = bits_to_hex(decrypted_bits)
    print(f"Decrypted    : {decrypted_hex}")
    print(f"Round-trip correct: {decrypted_hex == plain_hex}\n")

    print("--- Additional demonstration: encrypting a text message (as hex blocks) ---")
    message = "SECURE!!" # exactly 8 bytes = one 64-bit block
    msg_hex = message.encode().hex()
    msg_bits = hex_to_bits(msg_hex)
    demo_key_bits = hex_to_bits("0E329232EA6D0D73")

    demo_cipher_bits = des_encrypt_block(msg_bits, demo_key_bits)
    demo_decrypted_bits = des_decrypt_block(demo_cipher_bits, demo_key_bits)
    recovered_text = bytes(int('').join(map(str, demo_decrypted_bits[i:i+8])), 2)
                        for i in range(0, 64, 8)).decode()

    print(f"Message      : {message!r}")
    print(f"Ciphertext   : {bits_to_hex(demo_cipher_bits)!r}")
    print(f"Recovered    : {recovered_text!r} (matches: {recovered_text == message})")

```

Output:

```
Q33 - Full DES Implementation

=== Full DES Implementation (64-bit block, 56-bit effective key) ===

Key       : 133457799BBCDFF1
Plaintext  : 0123456789ABCDEF
Ciphertext: 85E813540F0AB405
Expected   : 85E813540F0AB405 (official FIPS PUB 46 test vector)
Matches official test vector: True

Decrypted  : 0123456789ABCDEF
Round-trip correct: True

--- Additional demonstration: encrypting a text message (as hex blocks) ---
Message    : 'SECURE!!'
Ciphertext : 785FC2B4AB60C82E
Recovered  : 'SECURE!!' (matches: True)
```

Question 34: ECB, CBC, and CFB Modes with Mandatory Bit-Padding

Problem: (Identical to Question 21.) For ECB, CBC, and CFB modes, the plaintext must be a positive multiple of the block (or segment) size. Pad using a 1 bit followed by as few zero bits as necessary, and always pad every message, even when the final block is already complete. What is the motivation for including a padding block when padding is not needed?

This is the same question as Question 21; the identical program and output are reused here.

Python Program:

```
# Q21. ECB, CBC, and CFB modes require the plaintext to be a positive multiple
# of the block/segment size. We implement the standard "bit padding" scheme:
# append a single 1 bit, then as few 0 bits as necessary (possibly none) to
# fill out the final block -- and we ALWAYS pad, even when the plaintext is
# already an exact multiple of the block size.
#
# We use a simple, illustrative 8-bit toy block cipher (byte-wise substitution
# + fixed permutation) so the padding/mode-of-operation logic is easy to see;
# the modes themselves work identically with any real block cipher (DES,
# AES, S-DES, etc. -- see Q22/Q23 for a real S-DES-based CBC/CTR demo).

import os

BLOCK_SIZE = 8 # bits (1 byte), for compact demonstration

# --- a tiny toy block "cipher" (NOT cryptographically secure -- illustrative only) ---
SBOX = list(range(256))
import random
random.seed(99)
random.shuffle(SBOX)
INV_SBOX = [0] * 256
for i, v in enumerate(SBOX):
    INV_SBOX[v] = i

def toy_encrypt_byte(byte_val, key_byte):
    return SBOX[(byte_val ^ key_byte) & 0xFF]

def toy_decrypt_byte(byte_val, key_byte):
    return INV_SBOX[byte_val] ^ key_byte

# --- bit-padding (append 1 then 0s) ---
def pad_bits(data: bytes, block_bytes=1):
    bits = ''.join(f'{b:08b}' for b in data)
```

```

bits += '1'
while len(bits) % (block_bytes * 8) != 0:
    bits += '0'
padded = bytes(int(bits[i:i + 8], 2) for i in range(0, len(bits), 8))
return padded

def unpad_bits(data: bytes):
    bits = ''.join(f'{b:08b}' for b in data)
    idx = bits.rfind('1')
    if idx == -1:
        raise ValueError("No padding bit found -- corrupted data")
    data_bits = bits[:idx]
    out_bytes = bytearray()
    for i in range(0, len(data_bits), 8):
        out_bytes.append(int(data_bits[i:i + 8], 2))
    return bytes(out_bytes)

# --- ECB ---
def ecb_encrypt(data: bytes, key: int):
    padded = pad_bits(data)
    return bytes(toy_encrypt_byte(b, key) for b in padded)

def ecb_decrypt(cipher: bytes, key: int):
    plain_padded = bytes(toy_decrypt_byte(b, key) for b in cipher)
    return unpad_bits(plain_padded)

# --- CBC ---
def cbc_encrypt(data: bytes, key: int, iv: int):
    padded = pad_bits(data)
    prev = iv
    out = bytearray()
    for b in padded:
        c = toy_encrypt_byte(b ^ prev, key)
        out.append(c)
        prev = c
    return bytes(out)

def cbc_decrypt(cipher: bytes, key: int, iv: int):
    prev = iv
    out = bytearray()
    for c in cipher:
        p = toy_decrypt_byte(c, key) ^ prev
        out.append(p)
        prev = c
    return unpad_bits(bytes(out))

# --- CFB (full-byte feedback, byte-oriented for this toy cipher) ---
def cfb_encrypt(data: bytes, key: int, iv: int):
    padded = pad_bits(data)
    shift_reg = iv
    out = bytearray()
    for b in padded:
        keystream = toy_encrypt_byte(shift_reg, key) # encrypt the shift register
        c = b ^ keystream
        out.append(c)
        shift_reg = c
    return bytes(out)

```

```

def cfb_decrypt(cipher: bytes, key: int, iv: int):
    shift_reg = iv
    out = bytearray()
    for c in cipher:
        keystream = toy_encrypt_byte(shift_reg, key)
        p = c ^ keystream
        out.append(p)
        shift_reg = c
    return unpad_bits(bytes(out))

if __name__ == "__main__":
    key = 0x5A
    iv = 0x3C

    print("=== ECB / CBC / CFB with Mandatory Bit-Padding ===\n")

    for label, message in [
        ("Message NOT a multiple of block size", b"HELLO"),
        ("Message that IS already a whole number of blocks", b"ABCDEFGH"),
    ]:
        print(f"--- {label} ---")
        print(f"Plaintext: {message} ({len(message)} bytes)")
        padded = pad_bits(message)
        print(f"Padded      : {padded.hex()} ({len(padded)} bytes) "
              f"[always padded, even when {len(message)} was already a multiple of "
              f"{BLOCK_SIZE//8}]")

        ecb_c = ecb_encrypt(message, key)
        ecb_p = ecb_decrypt(ecb_c, key)
        print(f"ECB ciphertext: {ecb_c.hex()}    decrypted: {ecb_p}")

        cbc_c = cbc_encrypt(message, key, iv)
        cbc_p = cbc_decrypt(cbc_c, key, iv)
        print(f"CBC ciphertext: {cbc_c.hex()}    decrypted: {cbc_p}")

        cfb_c = cfb_encrypt(message, key, iv)
        cfb_p = cfb_decrypt(cfb_c, key, iv)
        print(f"CFB ciphertext: {cfb_c.hex()}    decrypted: {cfb_p}\n")

    print("--- Why pad even when padding isn't strictly needed? ---")
    print("If padding is applied only conditionally (skipped whenever the plaintext")
    print("already fills whole blocks), the receiver can no longer unambiguously")
    print("tell whether the final block's trailing bits are genuine message content")
    print("or a padding marker. A legitimate final block that happens to end in a")
    print("'1' bit followed by zeros is then indistinguishable from 'no real data'")
    print("here, this is padding' and would be wrongly stripped -- silently")
    print("corrupting the message. Always appending a padding block (even a full")
    print("extra dummy block, or here, always appending at least the 1-bit marker)")
    print("removes this ambiguity: the receiver's unpadding rule is unconditional")
    print("and unambiguous for every message, with no special case to get wrong.")

```

Output:

```
Q21 - ECB, CBC, CFB with Padding

=== ECB / CBC / CFB with Mandatory Bit-Padding ===

--- Message NOT a multiple of block size ---
Plaintext: b'HELLO' (5 bytes)
Padded : 48454c4c4f80 (6 bytes) [always padded, even when 5 was already a multiple of 1]
ECB ciphertext: f074a8a845fb decrypted: b'HELLO'
CBC ciphertext: 1ae3408a0ef2 decrypted: b'HELLO'
CFB ciphertext: 707c987e1dce decrypted: b'HELLO'

--- Message that IS already a whole number of blocks ---
Plaintext: b'ABCDEFGH' (8 bytes)
Padded : 414243444546474880 (9 bytes) [always padded, even when 8 was already a multiple of 1]
ECB ciphertext: ac3e5e6574cf7bf0fb decrypted: b'ABCDEFGH'
CBC ciphertext: 07740b4597934bd7df decrypted: b'ABCDEFGH'
CFB ciphertext: 7915b639167b548359 decrypted: b'ABCDEFGH'

--- Why pad even when padding isn't strictly needed? ---
If padding is applied only conditionally (skipped whenever the plaintext
already fills whole blocks), the receiver can no longer unambiguously
tell whether the final block's trailing bits are genuine message content
or a padding marker. A legitimate final block that happens to end in a
'1' bit followed by zeros is then indistinguishable from 'no real data
here, this is padding' and would be wrongly stripped -- silently
corrupting the message. Always appending a padding block (even a full
extra dummy block, or here, always appending at least the 1-bit marker)
removes this ambiguity: the receiver's unpadding rule is unconditional
and unambiguous for every message, with no special case to get wrong.
```

Question 35: One-Time Pad Version of the Vigenère Cipher

Problem: (Identical to Question 14.) The key is a stream of random numbers between 0 and 26. If the key is 3 19 5 ..., the first letter of plaintext is encrypted with a shift of 3 letters, the second with a shift of 19, the third with a shift of 5, and so on. This is the same question as Question 14; the identical program and output are reused here.

Python Program:

```
# Q14. One-time-pad version of the Vigenere cipher.
# Key is a stream of random numbers (shifts) between 0 and 26.
# a. Encrypt "send more money" with key stream 9 0 1 7 23 15 21 14 11 11 2 8 9
# b. Given the ciphertext from (a), find a DIFFERENT key stream that decrypts
#    it to "cash not needed" -- demonstrating a one-time pad gives perfect
#    secrecy (any plaintext of the same length is equally possible).
```

```
def otp_encrypt(text, key_stream):
    text = text.replace(" ", "").upper()
    result = []
    for ch, k in zip(text, key_stream):
        p = ord(ch) - 65
        c = (p + k) % 26
        result.append(chr(c + 65))
    return ''.join(result)

def otp_decrypt(cipher, key_stream):
    result = []
    for ch, k in zip(cipher, key_stream):
        c = ord(ch) - 65
        p = (c - k) % 26
        result.append(chr(p + 65))
    return ''.join(result)

def find_key_for_target(cipher, target):
    target = target.replace(" ", "").upper()
    key = []
    for c_ch, t_ch in zip(cipher, target):
        c = ord(c_ch) - 65
```

```

        p = ord(t_ch) - 65
        k = (c - p) % 26
        key.append(k)
    return key

if __name__ == "__main__":
    print("=== One-Time Pad (Vigenere) Cipher ===")

    plaintext_a = "send more money"
    key_a = [9, 0, 1, 7, 23, 15, 21, 14, 11, 11, 2, 8, 9]

    ciphertext = otp_encrypt(plaintext_a, key_a)
    print(f"a. Plaintext : {plaintext_a}")
    print(f"    Key stream: {key_a}")
    print(f"    Ciphertext: {ciphertext}")
    print(f"    Verify decrypt -> {otp_decrypt(ciphertext, key_a)}")

    target_plain = "cash not needed"
    key_b = find_key_for_target(ciphertext, target_plain)
    print(f"\nb. Same ciphertext: {ciphertext}")
    print(f"    Target plaintext : {target_plain}")
    print(f"    Required key stream: {key_b}")
    print(f"    Verify decrypt -> {otp_decrypt(ciphertext, key_b)}")

    print("\nThis shows that for a true one-time pad, the same ciphertext can be")
    print("decrypted to ANY plaintext of equal length with the right key --")
    print("hence brute force / cryptanalysis is impossible without the key.")

```

Output:

```

Q14 - One-Time Pad Vigenere

=== One-Time Pad (Vigenere) Cipher ===
a. Plaintext : send more money
   Key stream: [9, 0, 1, 7, 23, 15, 21, 14, 11, 11, 2, 8, 9]
   Ciphertext: BEOKJDMSXZPMH
   Verify decrypt -> SENDMOREMONEY

b. Same ciphertext: BEOKJDMSXZPMH
   Target plaintext : cash not needed
   Required key stream: [25, 4, 22, 3, 22, 15, 19, 5, 19, 21, 12, 8, 4]
   Verify decrypt -> CASHNOTNEEDED

This shows that for a true one-time pad, the same ciphertext can be
decrypted to ANY plaintext of equal length with the right key --
hence brute force / cryptanalysis is impossible without the key.

```

Question 36: The Affine Caesar Cipher and the One-to-One Requirement

Problem: (Identical to Question 5.) For each plaintext letter p , substitute the ciphertext letter $C = E([a, b], p) = (ap + b) \bmod 26$. An encryption algorithm must be one-to-one, but the affine Caesar cipher is not one-to-one for all values of a -- e.g. for $a = 2$, $b = 3$, $E([a, b], 0) = E([a, b], 13) = 3$.

This is the same question as Question 5; the identical program and output are reused here.

Python Program:

```

# Q5. Affine Caesar Cipher
# C = E([a, b], p) = (a*p + b) mod 26
# For the cipher to be one-to-one (invertible), gcd(a, 26) must equal 1.

from math import gcd

def affine_encrypt(text, a, b):

```

```

result = []
for ch in text:
    if ch.isalpha():
        base = 65 if ch.isupper() else 97
        p = ord(ch) - base
        c = (a * p + b) % 26
        result.append(chr(c + base))
    else:
        result.append(ch)
return ''.join(result)

def mod_inverse(a, m=26):
    for x in range(1, m):
        if (a * x) % m == 1:
            return x
    return None

def affine_decrypt(text, a, b):
    a_inv = mod_inverse(a)
    result = []
    for ch in text:
        if ch.isalpha():
            base = 65 if ch.isupper() else 97
            c = ord(ch) - base
            p = (a_inv * (c - b)) % 26
            result.append(chr(p + base))
        else:
            result.append(ch)
    return ''.join(result)

if __name__ == "__main__":
    print("=== Affine Caesar Cipher ===")

    a, b = 5, 3 # example valid values
    plaintext = "AFFINE CIPHER"
    ciphertext = affine_encrypt(plaintext, a, b)
    decrypted = affine_decrypt(ciphertext, a, b)

    print(f"a = {a}, b = {b}")
    print(f"Plaintext : {plaintext}")
    print(f"Ciphertext: {ciphertext}")
    print(f"Decrypted : {decrypted}")

    print("\n--- Analysis questions ---")
    print("a. Are there any limitations on the value of b?")
    print("    No. b may be any value 0-25; every b gives a valid shift component,")
    print("    so b alone never breaks the one-to-one property.")

    print("\nb. Which values of a are NOT allowed?")
    not_allowed = [x for x in range(1, 26) if gcd(x, 26) != 1]
    allowed = [x for x in range(1, 26) if gcd(x, 26) == 1]
    print(f"    a must satisfy gcd(a, 26) = 1 (so that a has an inverse mod 26).")
    print(f"    Allowed values of a      : {allowed}")
    print(f"    NOT allowed values of a : {not_allowed}")

```

Output:

```
Q5 - Affine Caesar Cipher

=== Affine Caesar Cipher ===
a = 5, b = 3
Plaintext : AFFINE CIPHER
Ciphertext: DCCRQX NRAMXK
Decrypted  : AFFINE CIPHER

--- Analysis questions ---
a. Are there any limitations on the value of b?
   No. b may be any value 0-25; every b gives a valid shift component,
   so b alone never breaks the one-to-one property.

b. Which values of a are NOT allowed?
   a must satisfy gcd(a, 26) = 1 (so that a has an inverse mod 26).
   Allowed values of a      : [1, 3, 5, 7, 9, 11, 15, 17, 19, 21, 23, 25]
   NOT allowed values of a : [2, 4, 6, 8, 10, 12, 13, 14, 16, 18, 20, 22, 24]
```

Question 37: Automated Frequency Attack on a Monoalphabetic Substitution Cipher

Problem: (Identical to Question 16.) Perform a letter frequency attack on any monoalphabetic substitution cipher without human intervention, producing possible plaintexts in rough order of likelihood, with a 'give me the top 10 possible plaintexts' option.

This is the same question as Question 16; the identical program and output are reused here.

Python Program:

```
# Q16. Letter-frequency attack on a general MONOALPHABETIC SUBSTITUTION cipher,
# without human intervention.
#
# A monoalphabetic substitution has 26! possible keys, so brute force is
# impossible. Instead we use:
# 1. Frequency matching -> a good starting guess (map ciphertext letters to
#    plaintext letters by matching frequency rank order).
# 2. Simulated annealing -> repeatedly try swapping two letters in the
#    candidate key; accept improving swaps always, and occasionally accept
#    worsening swaps (with probability that shrinks over time) to escape
#    local optima. Fitness = log-probability of the decrypted text under an
#    English bigram+trigram model.
# 3. Multiple random restarts, keeping the best distinct plaintexts found,
#    to report the "top N most likely plaintexts."

import math
import random
import string
from collections import Counter

# Approximate relative frequencies of common English bigrams/trigrams
# (hand-curated, small tables -- enough signal to guide the search for a lab demo).
ENGLISH_BIGRAMS = {
    "th": 27, "he": 23, "in": 17, "er": 17, "an": 16, "re": 14, "on": 13,
    "at": 12, "en": 12, "nd": 11, "ti": 11, "es": 11, "or": 10, "te": 10,
    "of": 10, "ed": 9, "is": 9, "it": 9, "al": 9, "ar": 9, "st": 9, "to": 9,
    "nt": 8, "ng": 8, "se": 8, "ha": 8, "as": 8, "ou": 8, "io": 7, "le": 7,
    "ve": 7, "co": 6, "me": 6, "de": 6, "hi": 6, "ri": 6, "ro": 6, "ic": 6,
    "ne": 6, "ea": 6, "ra": 6, "ce": 5, "li": 5, "ch": 5, "ll": 5, "be": 5,
    "ma": 5, "si": 5, "om": 5, "ur": 5, "wi": 4, "ss": 4, "ho": 4, "wa": 4,
    "ns": 4, "ot": 4, "no": 4, "fo": 4, "ta": 4, "ee": 4, "sa": 4,
}
ENGLISH_TRIGRAMS = {
    "the": 34, "and": 15, "ing": 15, "ion": 11, "ent": 9, "her": 8, "for": 7,
    "tha": 7, "nth": 6, "int": 6, "ere": 6, "tio": 6, "ter": 6, "est": 6,
```



```

    "ers": 6, "ati": 6, "hat": 5, "ate": 5, "all": 5, "eth": 5, "hes": 5,
    "ver": 5, "his": 5, "oft": 4, "ith": 4, "fth": 4, "sth": 4, "oth": 4,
    "res": 4, "ont": 4, "com": 4, "men": 4,
}

LOG_TOTAL_BI = math.log(sum(ENGLISH_BIGRAMS.values()) * 4)
LOG_TOTAL_TRI = math.log(sum(ENGLISH_TRIGRAMS.values()) * 4)
# Unseen n-grams are assumed rarer than anything in our table (assumed
# frequency 0.5, well below the smallest listed value of 4) so that matching
# a *known* common bigram/trigram always scores strictly better than missing.
FLOOR_BI = math.log(0.5) - LOG_TOTAL_BI
FLOOR_TRI = math.log(0.5) - LOG_TOTAL_TRI

def score_text(text):
    text = ''.join(ch for ch in text.lower() if ch.isalpha())
    score = 0.0
    for i in range(len(text) - 1):
        bg = text[i:i + 2]
        f = ENGLISH_BIGRAMS.get(bg)
        score += (math.log(f) - LOG_TOTAL_BI) if f else FLOOR_BI
    for i in range(len(text) - 2):
        tg = text[i:i + 3]
        f = ENGLISH_TRIGRAMS.get(tg)
        score += 1.5 * ((math.log(f) - LOG_TOTAL_TRI) if f else FLOOR_TRI)
    return score

def decrypt_with_key(ciphertext, key):
    result = []
    for ch in ciphertext:
        if ch.isalpha():
            dec = key.get(ch.lower(), ch.lower())
            result.append(dec.upper() if ch.isupper() else dec)
        else:
            result.append(ch)
    return ''.join(result)

def frequency_initial_key(ciphertext):
    english_order = "etaoinshrdlcumwfgypbvkjxqz"
    counts = Counter(ch.lower() for ch in ciphertext if ch.isalpha())
    cipher_order = [c for c, _ in counts.most_common()]
    for c in string.ascii_lowercase:
        if c not in cipher_order:
            cipher_order.append(c)
    key = {}
    for i, c in enumerate(cipher_order):
        key[c] = english_order[i] if i < len(english_order) else c
    return key

def simulated_annealing(ciphertext, start_key, iterations, rng):
    key = dict(start_key)
    score = score_text(decrypt_with_key(ciphertext, key))
    best_key, best_score = dict(key), score
    letters = list(string.ascii_lowercase)

    T0, Tmin = 3.0, 0.05
    for step in range(iterations):
        T = max(Tmin, T0 * (1 - step / iterations))
        a, b = rng.sample(letters, 2)
        new_key = dict(key)
        va, vb = new_key.get(a, a), new_key.get(b, b)
        new_key[a], new_key[b] = vb, va

```

```

    new_score = score_text(decrypt_with_key(ciphertext, new_key))

    if new_score > score or rng.random() < math.exp((new_score - score) / T):
        key, score = new_key, new_score
        if score > best_score:
            best_key, best_score = dict(key), score
    return best_key, best_score

def attack(ciphertext, restarts=10, iterations=6000, top_n=10, seed=1):
    rng = random.Random(seed)
    base_key = frequency_initial_key(ciphertext)
    candidates = {}
    for r in range(restarts):
        key = dict(base_key)
        letters = list(string.ascii_lowercase)
        shuffle_amount = 0 if r == 0 else rng.randint(1, 4)
        for _ in range(shuffle_amount):
            a, b = rng.sample(letters, 2)
            key[a], key[b] = key.get(b, b), key.get(a, a)
        final_key, score = simulated_annealing(ciphertext, key, iterations, rng)
        plain = decrypt_with_key(ciphertext, final_key)
        candidates[plain] = max(candidates.get(plain, float("-inf")), score)

    ranked = sorted(candidates.items(), key=lambda kv: kv[1], reverse=True)
    return ranked[:top_n]

if __name__ == "__main__":
    print("=== Automated Frequency Attack on a Monoalphabetic Substitution Cipher ===")

    random.seed(7)
    plain_alpha = list(string.ascii_lowercase)
    cipher_alpha = plain_alpha.copy()
    random.shuffle(cipher_alpha)
    enc_map = dict(zip(plain_alpha, cipher_alpha))

    secret_plaintext = (
        "this cryptographic attack recovers the original plaintext without "
        "any human intervention by combining letter frequency analysis with "
        "a simulated annealing search over the entire key space so that the "
        "correct substitution alphabet is discovered automatically from the "
        "statistical structure of the english language and the program can "
        "then report the most likely plaintexts it has found in ranked order"
    )
    ciphertext = ''.join(enc_map.get(c, c) for c in secret_plaintext)

    print(f"(demo) Original plaintext: {secret_plaintext}")
    print(f"(demo) Ciphertext      : {ciphertext}\n")

    print("Give me the top 10 possible plaintexts:\n")
    results = attack(ciphertext, restarts=10, iterations=6000, top_n=10, seed=42)
    for rank, (plain, score) in enumerate(results, start=1):
        marker = " <-- best candidate" if rank == 1 else ""
        print(f"{rank:2d}. score={score:9.1f} {plain}{marker}")

```

Output:

```

Q16 - Frequency Attack on Monoalphabetic Cipher

=== Automated Frequency Attack on a Monoalphabetic Substitution Cipher ===
(demo) Original plaintext: this cryptographic attack recovers the original plaintext without
any human intervention by combining letter frequency analysis with a simulated annealing search
over the entire key space so that the correct substitution alphabet is discovered automatically
from the statistical structure of the english language and the program can then report the most
likely plaintexts it has found in ranked order
(demo) Ciphertext : rtzd wlegraoljgtzw jrrjwn lhwabhld rth alzoqjx gxjzqrhmr uzrtacr jqe tcpjq
zqrhlbhrzaq fe wapfzqzqo xhrrhl ylhvchqwe jqjxedzd uzrt j dzpcxjrhi jqqhjxzqo dhjltw abhl rth
hqrzlh nhe dgjwh da rtjr rth wallhwr dcfdrzrcrzaq jxgtjfhz zd izdwabhlhi jcrapjrzwxje ylap rth
drjrzdrzwx drlcwrcjh ay rth hqoxzdt xjqocjoh jqi rth glaoljp wjq rthq lhgair rth padr xznhe
gxjzqrhmrdr zr tjd yacqi zq ljnhi alihi

Give me the top 10 possible plaintexts:

1. score= -5893.0 this crxjtograjhic attacb recovers the original jlaintext withomt anx hmfan
intervention yx cofyining letter ureqmencx analxsis with a sifmlated annealing search over the
entire bex sjace so that the correct smystitmtion alhayet is discovered amtofaticallx urof the
statistical strmtcmre ou the english langmage and the jrograf can then reort the fost libelx
jlaintexts it has uomnd in ranbed order <-- best candidate
2. score= -6121.7 esrh iapdecmandsri neenif aticxtah est carmrong dgnroetve yrescle nop slbno
roetaxtoerco zp icbzzorom gteeta watqltoip nongphrh yres n hrbfgnetu nootngrom htnais cxta est
toerat ftp hdnit hc esne est icaatie hlzherelerco ngdsnzte rh urhicxtatu nlecbneringgp wacb est
henerhering healielat cw est tomgrhs gnomlnmt nou est dacmanb ino esto atdcae est bche grftgp
dgnroetveh re snh wclou ro anoftu cauta
3. score= -6126.0 envr iaufecwatfnri teetij asicysav ens carwrhto fotrhesbe prencde thu ndxth
rhesayscherch gu icxgrhrhw oseesa maszdshiu thtouvrv pren t vrxdotsl thhstorhw vstain cysa ens
sheras jsu vftis vc ente ens icaasie vdgverederch tofntgse rv lrvicysasl tdecxteritoou maxc ens
veterverito veadiedas cm ens shworvn othwdtws thl ens facwatx ith ensh asfcae ens xcve orjsou
fotrhesbev re ntv mcdhl rh athjls calsa
4. score= -6136.2 ernd shufegchifrn ieseip hasgwahd era ghncntio fointeabe ynergme itu rmzit
nteahwatengt ku sgzkntntc oaeeah vxaxmatsu itioudnd yner i dnmzoieal ittaiontc daihsr gwah era
atenha pau dfisa dg erie era sghhase dmkenemenegt iofrikae nd lndsgwahal imegziensioou vhz era
deiendensio dehmsemha gv era atcondr oitcmica itl era fhgchiz sit erat hafghe era zgde onpaou
fointeabed ne rid vgmth nt hitpal ghlah
5. score= -6139.9 esro aifbelwinbsra neenaz italptio est lirwrhnd bdnrhetve xreslme nhf smgnh
rhetiptherlh qf algqrhrhw dteeti citjmthaf nhndforo xres n orgmdnetu nhhtndrhv otnias lpti est
therit ztf obnat ol esne est aliitae omqoeremerlh ndbsnqte ro uroalptitu nmelgneranddf cilg est
oeneroerand oeimaemit lc est thwdros dnhwmnwt nhu est bilwing anh esth itblie est gloe drztdf
bdnrhetveo re sno clmhu rh inhztu liuti
6. score= -6143.6 ernd cipweogiawrnc aeaeack iscousid ers oingntal wlantesbe znerofe atp rfmatt
ntesiustenot yp comyntntg lseesi visxfstcp atalpdnd zner a dnmflaesh attsalntg dsairc ousi ers
stenis ksp dwacs do erae ers coiisce dfydenefenot alwayse nd hndcousish afeomaencallp viom ers
deaendencal deifcefis ov ers stglndr latgfags ath ers wiogiam cat erst iswoie ers mode lnkslp
wlantesbed ne rad vofth nt iatksh oihsi
7. score= -6146.3 eram liufechinfral neenly islcqsim ers ciahatno fonatesbe waernde ntu rdgnt
atesiqsteact ju lcgjatath oseesi visxdstlu ntoumam waer n magdonesz nttsnoath msnlr cqi ers
steais ysu mfnls mc erne ers lciisle mdjmeadeact nofrnjse am zamlcqsizs ndecgnealnoou vicg ers
meneamealno meidledis cv ers sthoamr onthdnhs ntz ers fiching lnt erst isfcie ers gcme oaysou
fonatesbem ae rnm vcdtz at intysz cizsi
8. score= -6151.0 ertm ciguewaiourtc oeeocx iscwzsim ers witathon unothsbe jterwde ohg rdvoh
thesizshetwh yg cwvyththa nseesi lisqdshcg ohongmtm jter o mtvdnoesf ohhsontha msoicr wzsi ers
shetis xsg muocs mw eroe ers cwiisce mdymetedetwh onuroyse tm fmcwzsisf odewvoetconng liwv ers
meoetmetcon meidcedis wl ers shantmr nohadoas ohf ers uiwaiov coh ersh isuwie ers vwme ntxsng
unothsbem te rom lwdhf th iohxsf wifsi
9. score= -6153.0 tehs aomvtlconveha ntnnaz oraluros ter lohching vgnhitryt bhteldt nim edwni
hitrouarithli pm alwphihih grttro forxdriam ningmshs bhte n shwdgntrk niirnghic srnoae luro ter
rithor zrm svnar sl tent ter aloorat sdpsthtdthli ngvenprt hs khsalurork ndtlwnthanggm folw ter
stnthsthang stodator lf ter ricghse gnicdnr nik ter volconw ani teri orvlot ter wlst ghzrgm
vgnhitryts ht ens fldik hi onizrk lokro
10. score= -6186.8 erin tmqceozmhcrit heehtk matouamn era omizishw cwhiseabe gierode hsq rdfs
iseamuaseios yq tofyisizs waeeam vmapdastq hshwqnin gier h nifdwheal hssahwisz nahmtr ouam era
aseima kaq nchta no erhe era tommate ndyneiedeios hwcrhyaie in lintouamal hdeofheithwwq vmof era
neheineithw nemdtedma ov era aszwirw whszdhza hsl era cmozmhf ths eras macome era fone wikawq
cwhiseaben ie rhn vodsl is mhskaal omlam

```

Question 38: Hill Cipher: Chosen-Plaintext Attack

Problem: The Hill cipher succumbs to a known-plaintext attack if sufficient plaintext-ciphertext pairs are provided (see Question 13). It is even easier to solve the Hill cipher if a chosen-plaintext attack can be mounted. Implement this.

Python Program:

```
# Q38. The Hill cipher succumbs to a known-plaintext attack given enough
# plaintext-ciphertext pairs (see Q13). It is even EASIER to break with a
# CHOSEN-plaintext attack: the attacker gets to pick convenient plaintext
# digraphs rather than hoping a natural message happens to provide a
# solvable (invertible) plaintext matrix.
#
# The cleanest choice: choose plaintext blocks equal to the standard basis
# vectors, e.g. for a 2x2 key, choose P1 = "BA" ([1,0]) and P2 = "AC"
# ([0,2])... more simply, choose the plaintext MATRIX to be the identity
# matrix I. Then C = K*P = K*I = K -- the ciphertext IS the key, read off
# directly with no linear algebra at all!

import numpy as np

def mod_inverse(a, m=26):
    a = a % m
    for x in range(1, m):
        if (a * x) % m == 1:
            return x
    return None

def matrix_mod_inverse(matrix, modulus=26):
    det = int(round(np.linalg.det(matrix)))
    det_inv = mod_inverse(det % modulus, modulus)
    if det_inv is None:
        return None
    adj = np.array([[matrix[1][1], -matrix[0][1]],
                    [-matrix[1][0], matrix[0][0]]])
    return (det_inv * adj) % modulus

def text_to_vecs(text):
    nums = [ord(c) - 65 for c in text.upper()]
    return np.array(nums).reshape(-1, 2).T

def vecs_to_text(vecs):
    flat = vecs.T.flatten()
    return ''.join(chr(int(v) % 26 + 65) for v in flat)

def hill_encrypt_vecs(P, key):
    return np.dot(key, P) % 26

if __name__ == "__main__":
    print("=== Hill Cipher: Chosen-Plaintext Attack (Even Easier Than Known-Plaintext) ===\n")

    # The real key, unknown to the attacker -- but the attacker gets an
    # encryption oracle: they may submit ANY plaintext and observe its ciphertext.
    true_key = np.array([[9, 4], [5, 7]])
    print("(unknown to attacker) True key matrix:")
    print(true_key, "\n")

    print("--- Known-plaintext attack (recap from Q13): needs an invertible P ---")
    print("The attacker must WAIT for or already possess plaintext-ciphertext pairs")
    print("whose plaintext matrix P happens to be invertible mod 26 -- not")
    print("guaranteed for arbitrary intercepted text.\n")
```

```

print("--- Chosen-plaintext attack: attacker actively PICKS the plaintext ---")
print("Choose the plaintext matrix P = the 2x2 IDENTITY matrix -- i.e. submit")
print("the two plaintext digraphs whose numeric column vectors are [1,0] and")
print("[0,1] (letters 'B A' and 'A B'):")

chosen_plain_text = "BAAB" # digraph1 = "BA" = [1,0]; digraph2 = "AB" = [0,1]
P = text_to_vecs(chosen_plain_text)
print(f"Chosen plaintext: {chosen_plain_text} -> P = \n{P}\n")

# The attacker submits P to the encryption oracle (i.e. gets it encrypted
# under the real, unknown key) and observes C.
C = hill_encrypt_vecs(P, true_key)
chosen_cipher_text = vecs_to_text(C)
print(f"Oracle returns ciphertext: {chosen_cipher_text} -> C = \n{C}\n")

print("Since P = I (the identity matrix), C = K*P = K*I = K directly --")
print("the ciphertext matrix IS the key matrix. No matrix inversion, no")
print("linear algebra at all is needed:")
recovered_key_trivial = C
print(recovered_key_trivial)
print(f"Recovered key matches the true key: {np.array_equal(recovered_key_trivial,
true_key)}\n")

print("--- For comparison: the general (known-plaintext) formula still works too ---")
P_inv = matrix_mod_inverse(P)
recovered_key_general = np.dot(C, P_inv) % 26
print(f"K = C * P^-1 (mod 26) = \n{recovered_key_general}")
print(f"Matches true key: {np.array_equal(recovered_key_general, true_key)}\n")

print("--- Conclusion ---")
print("A chosen-plaintext attacker doesn't need to search for or wait for a")
print("conveniently invertible plaintext matrix, and doesn't need to perform")
print("any modular matrix inversion at all -- choosing P = I makes the")
print("ciphertext equal to the key by construction. This is why chosen-")
print("plaintext attacks are strictly more powerful than known-plaintext")
print("attacks against the (linear, and therefore fundamentally weak) Hill cipher.")

```

Output:

```
Q38 - Hill Cipher Chosen-Plaintext Attack

=== Hill Cipher: Chosen-Plaintext Attack (Even Easier Than Known-Plaintext) ===

(unknown to attacker) True key matrix:
[[9 4]
 [5 7]]

--- Known-plaintext attack (recap from Q13): needs an invertible P ---
The attacker must WAIT for or already possess plaintext-ciphertext pairs
whose plaintext matrix P happens to be invertible mod 26 -- not
guaranteed for arbitrary intercepted text.

--- Chosen-plaintext attack: attacker actively PICKS the plaintext ---
Choose the plaintext matrix P = the 2x2 IDENTITY matrix -- i.e. submit
the two plaintext digraphs whose numeric column vectors are [1,0] and
[0,1] (letters 'B A' and 'A B'):
  Chosen plaintext: BAAB -> P =
[[1 0]
 [0 1]]

Oracle returns ciphertext: JFEH -> C =
[[9 4]
 [5 7]]

Since P = I (the identity matrix), C = K*P = K*I = K directly --
the ciphertext matrix IS the key matrix. No matrix inversion, no
linear algebra at all is needed:
[[9 4]
 [5 7]]
Recovered key matches the true key: True

--- For comparison: the general (known-plaintext) formula still works too ---
K = C * P^-1 (mod 26) =
[[9 4]
 [5 7]]
Matches true key: True

--- Conclusion ---
A chosen-plaintext attacker doesn't need to search for or wait for a
conveniently invertible plaintext matrix, and doesn't need to perform
any modular matrix inversion at all -- choosing P = I makes the
ciphertext equal to the key by construction. This is why chosen-
plaintext attacks are strictly more powerful than known-plaintext
attacks against the (linear, and therefore fundamentally weak) Hill cipher.
```

Question 39: Automated Frequency Attack on an Additive Cipher

Problem: (Identical to Question 15.) Perform a letter frequency attack on an additive cipher without human intervention, producing possible plaintexts in rough order of likelihood, with a 'give me the top 10 possible plaintexts' option.

This is the same question as Question 15; the identical program and output are reused here.

Python Program:

```
# Q15. Automated letter-frequency attack on an additive (Caesar) cipher.
# Tries all 26 shifts, scores each candidate plaintext against standard
# English letter frequencies using a chi-squared statistic, and returns
# the top-N most likely plaintexts (lower chi-squared = more English-like).
```

```
ENGLISH_FREQ = {
    'a': 8.2, 'b': 1.5, 'c': 2.8, 'd': 4.3, 'e': 12.7, 'f': 2.2, 'g': 2.0,
    'h': 6.1, 'i': 7.0, 'j': 0.15, 'k': 0.77, 'l': 4.0, 'm': 2.4, 'n': 6.7,
```

```

'o': 7.5, 'p': 1.9, 'q': 0.095, 'r': 6.0, 's': 6.3, 't': 9.1, 'u': 2.8,
'v': 0.98, 'w': 2.4, 'x': 0.15, 'y': 2.0, 'z': 0.074
}

def shift_text(text, k):
    result = []
    for ch in text:
        if ch.isalpha():
            base = 65 if ch.isupper() else 97
            result.append(chr((ord(ch) - base + k) % 26 + base))
        else:
            result.append(ch)
    return ''.join(result)

def chi_squared_score(text):
    text = text.lower()
    n = sum(1 for ch in text if ch.isalpha())
    if n == 0:
        return float('inf')
    counts = {c: 0 for c in ENGLISH_FREQ}
    for ch in text:
        if ch in counts:
            counts[ch] += 1
    score = 0.0
    for c, expected_pct in ENGLISH_FREQ.items():
        observed = counts[c]
        expected = expected_pct / 100 * n
        score += (observed - expected) ** 2 / expected
    return score

def top_n_candidates(ciphertext, n=10):
    candidates = []
    for k in range(26):
        plain = shift_text(ciphertext, k)
        score = chi_squared_score(plain)
        candidates.append((score, k, plain))
    candidates.sort(key=lambda x: x[0])
    return candidates[:n]

if __name__ == "__main__":
    print("=== Automated Frequency Attack on an Additive Cipher ===")

    original_shift = 11
    plaintext = "meet me after the toga party"
    ciphertext = shift_text(plaintext, original_shift)

    print(f"(demo) Original plaintext : {plaintext}")
    print(f"(demo) Encrypted with k={original_shift}: {ciphertext}\n")

    print("Give me the top 10 possible plaintexts:\n")
    results = top_n_candidates(ciphertext, n=10)
    for rank, (score, k, plain) in enumerate(results, start=1):
        marker = " <-- most likely" if rank == 1 else ""
        print(f"{rank:2d}. shift={k:2d} chi^2={score:7.2f} {plain}{marker}")

```

Output:

```
Q15 - Frequency Attack on Additive Cipher

=== Automated Frequency Attack on an Additive Cipher ===
(demo) Original plaintext : meet me after the toga party
(demo) Encrypted with k=11: xppe xp lqepc esp ezrl alcej

Give me the top 10 possible plaintexts:

1. shift=15  chi^2= 22.35  meet me after the toga party  <-- most likely
2. shift= 3  chi^2= 38.19  assb as othsf hvs hcuo dofhm
3. shift=22  chi^2= 45.74  tlla tl hmaly aol avnh whyaf
4. shift= 4  chi^2= 55.19  btti bt puitg iwt idvp epgin
5. shift=11  chi^2= 82.62  iaap ia wbpan pda pkcw lwnpu
6. shift=13  chi^2= 92.87  kccr kc ydrpc rfc rmey nyprw
7. shift=18  chi^2= 104.17 phhw ph diwhu wkh wrjd sduwb
8. shift= 9  chi^2= 167.10 gyyn gy uznyl nby niau julns
9. shift=25  chi^2= 196.79 wood wo kpdbp dro dyqk zkbdi
10. shift=16 chi^2= 207.01 nffu nf bgufs uif uphb qbsuz
```

Question 40: Automated Frequency Attack on a Monoalphabetic Substitution Cipher

Problem: (Identical to Questions 16 and 37.) Perform a letter frequency attack on any monoalphabetic substitution cipher without human intervention, producing possible plaintexts in rough order of likelihood, with a 'give me the top 10 possible plaintexts' option.

This is the same question as Questions 16 and 37; the identical program and output are reused here.

Python Program:

```
# Q16. Letter-frequency attack on a general MONOALPHABETIC SUBSTITUTION cipher,
# without human intervention.
#
# A monoalphabetic substitution has 26! possible keys, so brute force is
# impossible. Instead we use:
# 1. Frequency matching -> a good starting guess (map ciphertext letters to
#    plaintext letters by matching frequency rank order).
# 2. Simulated annealing -> repeatedly try swapping two letters in the
#    candidate key; accept improving swaps always, and occasionally accept
#    worsening swaps (with probability that shrinks over time) to escape
#    local optima. Fitness = log-probability of the decrypted text under an
#    English bigram+trigram model.
# 3. Multiple random restarts, keeping the best distinct plaintexts found,
#    to report the "top N most likely plaintexts."

import math
import random
import string
from collections import Counter

# Approximate relative frequencies of common English bigrams/trigrams
# (hand-curated, small tables -- enough signal to guide the search for a lab demo).
ENGLISH_BIGRAMS = {
    "th": 27, "he": 23, "in": 17, "er": 17, "an": 16, "re": 14, "on": 13,
    "at": 12, "en": 12, "nd": 11, "ti": 11, "es": 11, "or": 10, "te": 10,
    "of": 10, "ed": 9, "is": 9, "it": 9, "al": 9, "ar": 9, "st": 9, "to": 9,
    "nt": 8, "ng": 8, "se": 8, "ha": 8, "as": 8, "ou": 8, "io": 7, "le": 7,
    "ve": 7, "co": 6, "me": 6, "de": 6, "hi": 6, "ri": 6, "ro": 6, "ic": 6,
    "ne": 6, "ea": 6, "ra": 6, "ce": 5, "li": 5, "ch": 5, "ll": 5, "be": 5,
    "ma": 5, "si": 5, "om": 5, "ur": 5, "wi": 4, "ss": 4, "ho": 4, "wa": 4,
    "ns": 4, "ot": 4, "no": 4, "fo": 4, "ta": 4, "ee": 4, "sa": 4,
}

ENGLISH_TRIGRAMS = {
    "the": 34, "and": 15, "ing": 15, "ion": 11, "ent": 9, "her": 8, "for": 7,
```



```

    "tha": 7, "nth": 6, "int": 6, "ere": 6, "tio": 6, "ter": 6, "est": 6,
    "ers": 6, "ati": 6, "hat": 5, "ate": 5, "all": 5, "eth": 5, "hes": 5,
    "ver": 5, "his": 5, "oft": 4, "ith": 4, "fth": 4, "sth": 4, "oth": 4,
    "res": 4, "ont": 4, "com": 4, "men": 4,
}

LOG_TOTAL_BI = math.log(sum(ENGLISH_BIGRAMS.values()) * 4)
LOG_TOTAL_TRI = math.log(sum(ENGLISH_TRIGRAMS.values()) * 4)
# Unseen n-grams are assumed rarer than anything in our table (assumed
# frequency 0.5, well below the smallest listed value of 4) so that matching
# a *known* common bigram/trigram always scores strictly better than missing.
FLOOR_BI = math.log(0.5) - LOG_TOTAL_BI
FLOOR_TRI = math.log(0.5) - LOG_TOTAL_TRI

def score_text(text):
    text = ''.join(ch for ch in text.lower() if ch.isalpha())
    score = 0.0
    for i in range(len(text) - 1):
        bg = text[i:i + 2]
        f = ENGLISH_BIGRAMS.get(bg)
        score += (math.log(f) - LOG_TOTAL_BI) if f else FLOOR_BI
    for i in range(len(text) - 2):
        tg = text[i:i + 3]
        f = ENGLISH_TRIGRAMS.get(tg)
        score += 1.5 * ((math.log(f) - LOG_TOTAL_TRI) if f else FLOOR_TRI)
    return score

def decrypt_with_key(ciphertext, key):
    result = []
    for ch in ciphertext:
        if ch.isalpha():
            dec = key.get(ch.lower(), ch.lower())
            result.append(dec.upper() if ch.isupper() else dec)
        else:
            result.append(ch)
    return ''.join(result)

def frequency_initial_key(ciphertext):
    english_order = "etaoinshrdlcumwfgypbvkjxqz"
    counts = Counter(ch.lower() for ch in ciphertext if ch.isalpha())
    cipher_order = [c for c, _ in counts.most_common()]
    for c in string.ascii_lowercase:
        if c not in cipher_order:
            cipher_order.append(c)
    key = {}
    for i, c in enumerate(cipher_order):
        key[c] = english_order[i] if i < len(english_order) else c
    return key

def simulated_annealing(ciphertext, start_key, iterations, rng):
    key = dict(start_key)
    score = score_text(decrypt_with_key(ciphertext, key))
    best_key, best_score = dict(key), score
    letters = list(string.ascii_lowercase)

    T0, Tmin = 3.0, 0.05
    for step in range(iterations):
        T = max(Tmin, T0 * (1 - step / iterations))
        a, b = rng.sample(letters, 2)
        new_key = dict(key)
        va, vb = new_key.get(a, a), new_key.get(b, b)

```

```

        new_key[a], new_key[b] = vb, va
        new_score = score_text(decrypt_with_key(ciphertext, new_key))

        if new_score > score or rng.random() < math.exp((new_score - score) / T):
            key, score = new_key, new_score
            if score > best_score:
                best_key, best_score = dict(key), score
    return best_key, best_score

def attack(ciphertext, restarts=10, iterations=6000, top_n=10, seed=1):
    rng = random.Random(seed)
    base_key = frequency_initial_key(ciphertext)
    candidates = {}
    for r in range(restarts):
        key = dict(base_key)
        letters = list(string.ascii_lowercase)
        shuffle_amount = 0 if r == 0 else rng.randint(1, 4)
        for _ in range(shuffle_amount):
            a, b = rng.sample(letters, 2)
            key[a], key[b] = key.get(b, b), key.get(a, a)
        final_key, score = simulated_annealing(ciphertext, key, iterations, rng)
        plain = decrypt_with_key(ciphertext, final_key)
        candidates[plain] = max(candidates.get(plain, float("-inf")), score)

    ranked = sorted(candidates.items(), key=lambda kv: kv[1], reverse=True)
    return ranked[:top_n]

if __name__ == "__main__":
    print("=== Automated Frequency Attack on a Monoalphabetic Substitution Cipher ===")

    random.seed(7)
    plain_alpha = list(string.ascii_lowercase)
    cipher_alpha = plain_alpha.copy()
    random.shuffle(cipher_alpha)
    enc_map = dict(zip(plain_alpha, cipher_alpha))

    secret_plaintext = (
        "this cryptographic attack recovers the original plaintext without "
        "any human intervention by combining letter frequency analysis with "
        "a simulated annealing search over the entire key space so that the "
        "correct substitution alphabet is discovered automatically from the "
        "statistical structure of the english language and the program can "
        "then report the most likely plaintexts it has found in ranked order"
    )
    ciphertext = ''.join(enc_map.get(c, c) for c in secret_plaintext)

    print(f"(demo) Original plaintext: {secret_plaintext}")
    print(f"(demo) Ciphertext       : {ciphertext}\n")

    print("Give me the top 10 possible plaintexts:\n")
    results = attack(ciphertext, restarts=10, iterations=6000, top_n=10, seed=42)
    for rank, (plain, score) in enumerate(results, start=1):
        marker = " <-- best candidate" if rank == 1 else ""
        print(f"{rank:2d}. score={score:9.1f} {plain}{marker}")

```

Output:

Q16 - Frequency Attack on Monoalphabetic Cipher

=== Automated Frequency Attack on a Monoalphabetic Substitution Cipher ===

(demo) Original plaintext: this cryptographic attack recovers the original plaintext without any human intervention by combining letter frequency analysis with a simulated annealing search over the entire key space so that the correct substitution alphabet is discovered automatically from the statistical structure of the english language and the program can then report the most likely plaintexts it has found in ranked order

(demo) Ciphertext : rtzd wlegraoljgtzw jrrjwn lhwabhld rth alzoqjx gxjzqrhmr uzrtacr jqe tcpjq zqrhlbhrzaq fe wapfzqzo xhrrhl ylhvchqwe jqjxedzd uzrt j dzpcxjrhi jqqhjxzqo dhjltw abhl rth hqrzlh nhe dgjwh da rtjr rth wallhwr dcfdrzrcrzaq jxgtjfhz zd izdwabhlhi jcrapjrzwxje ylap rth drjrzdrzwx drlcwrcjh ay rth hqoxzdt xjqocjoh jqi rth glaoljp wjq rthq lhgair rth padr xznhxe gxjzqrhmrdr zr tjz yacqi zq ljqnhi alihi

Give me the top 10 possible plaintexts:

1. score= -5893.0 this crxjtograjhic attacb recovers the original jlaintext withomt anx hmfan intervention yx cofyining letter ureqmcncx analxsis with a sifmlated annealing search over the entire bex sjace so that the correct smystitmtion alhayet is discovered amtofaticallx urof the statistical strmtcmre ou the english langmage and the jrograf can then reort the fost libelx jlaintexts it has uomnd in ranbed order <-- best candidate

2. score= -6121.7 esrh iapdecmandsri neenif aticxtah est carmrng dgnroetve yrescle nop slbno roetaxtoerco zp icbzzorom gteeta watqltoip nongphrh yres n hrbngnetu nootngrom htnais cxta est toerat ftp hdnit hc esne est icaatie hlzherelerco ngdsnzte rh urhicxtatu nlecbneringgp wacb est henerhering healielat cw est tomgrhs gnomlnmt nou est dacmanb ino esto atdcae est bche grftgp dgnroetveh re snh wclou ro anoftu cauta

3. score= -6126.0 envr iaufecwatfnri teetij asicysav ens carwrhto fotrhesbe prencde thu ndxth rhesayscherch gu icxgrhrhw oseesa maszdshiu thtouvrv pren t vrxdotsl thhstorhw vstain cysa ens sheras jsu vftis vc ente ens icaasie vdgverederch tofntgse rv lrvicysasl tdecxteritooou maxc ens veterverito veadiedas cm ens shworvn othwdtws thl ens facwatx ith ensh asfcae ens xcve orjsou fotrhesbev re ntv mcdhl rh athjls calsa

4. score= -6136.2 ernd shufegchifrn ieseip hasgwahd era ghncntio fointeabe ynergme itu rmzit nteahwatengt ku sgzkntntc oaeeah vxaxmatsu itioudnd yner i dnmzoieal ittaiontc daihsr gwah era atenha pau dflsa dg erie era sghhase dmkenemenet iofrikae nd lndsgwahal imegziensioou vhz era deiendensio dehmsemha gv era atcondr oitcmica itl era fhgchiz sit erat hafghe era zgde onpaou fointeabed ne rid vgmth nt hitpal ghlah

5. score= -6139.9 esro aifbelwinbsra neenaz italptio est lirwrhnd bdnrhetve xreslme nhf smgnh rhetiptherlh qf algqrhrhw dteeti citjmthaf nhndforo xres n orgmdnetu nhhtndrhv otnias lpti est therit ztf obnat ol esne est aliitae omqoeremerlh ndbsnqte ro uroalptitu nmelgneranddf cilg est oeneroerand oeimaemit lc est thwdros dnhwmnwt nhu est bilwing anh esth itblie est gloe drztdf bdnrhetveo re sno clmhu rh inhztu liuti

6. score= -6143.6 ernd cipweogiawrnc aeaeack iscousid ers oingntal wlantesbe znerofe atp rfmatt ntesiuštenot yp comyntntg lseesi visxfstcp atalpdnd zner a dnmflaesh attsalntg dsacir ousi ers stenis ksp dwacs do erae ers coisce dfydenefenot alwayse nd hndcousish afeomaencallp viom ers deaendencal deifcefis ov ers stglndr latgfags ath ers wiogiam cat erst iswoie ers mode lnkslp wlantesbed ne rad vofth nt iatksh oihsi

7. score= -6146.3 eram liufechinfral neenly islcqsim ers ciahatno fonatesbe waernde ntu rdgnt atesiqsteact ju lcgjatath oseesi visxdstlu ntoumnam waer n magdonesz nttsnoath msnlr cqi ers steais ysu mfnls mc erne ers lciisle mdjmeadeact nofrnjse am zamlcqsizs ndecgnealnoou vicg ers meneamealno meidledis cv ers sthoamr onthdnhs ntz ers fiching lnt erst isfcie ers gcme oaysou fonatesbem ae rnm vcdtz at intysz cizsi

8. score= -6151.0 ertm ciguewaiourtc oeeocx iscwzsim ers witathon unothsbe jterwde ohg rdvoh thesizshetwh yg cwvyththa nseesi lisqdshcg ohongmtm jter o mtvdnoesf ohhsontha msoicr wzsi ers shetis xsg muocs mw eroe ers cwiisce mdymetedetwh onuroyse tm fmcwzsisf odewvoetconng liwv ers meoetmetcon meidcedis wl ers shantmr nohadoas ohf ers uiwaiov coh ersh isuwie ers vwme ntxsng unothsbem te rom lwdhf th iohxsf wifsi

9. score= -6153.0 tehs aomvtlconveha ntnnaz oraluros ter lohching vgnhitryt bhteldt nim edwni hitrouarithli pm alwphihih grttro forxdriam ningmshs bhte n shwdgntrk niirnghic srnoae luro ter rithor zrm svnar sl tent ter aloorat sdpsthtdthli ngvenprt hs khsalurork ndtlwnthanggm folw ter stnthsthang stodator lf ter ricghse gnicdnr nik ter volconw ani teri orvlot ter wlst ghzrgm vgnhitryts ht ens fldik hi onizrk lokro

10. score= -6186.8 erin tmqceozmhcrit heehtk matouamn era omizishw cwhiseabe gierode hsq rdfs iseamuaseios yq tofyisizs waeeam vmapdastq hshwqnin gier h nifdwheal hssahwisz nahmtr ouam era aseima kaq nchta no erhe era tommate ndyneiedeios hwcrhya in lintouamal hdeofheithwwq vmof era neheineithw nemtedma ov era aszwir whszdhza hsl era cmozmf ths eras macome era fone wikawq cwhiseaben ie rhn vodsl is mhskaal omlam